VIVA

Programming Environment

PUBLICATION ISSUED BY:

Via Kennedy, 24 - 10019 STRAMBINO (TO)
Phone: +39 0125 63 68 11
Fax.: +39 0125 63 68 99
e-mail: seica@seica.com
Website: http://www.seica.com

Copyright © 2010
All rights reserved

Microsoft Windows is a registered trademark of Microsoft Corporation

Document Code: MA-VI-VIVAPEEN-00
Release software: VIVA Rel. 2.0.3.x for Windows XP
Date of issue: 2-17-2010

Seica S.p.A. reserves the right to change the technical specifications without notice.

CONTENTS

CONTENTS ..... **I**
LIST OF FIGURES III
LIST OF TABLES III
ACRONIMS IV
1. Introduction **1-1**
1.1.1 Wizard Environment (CTRL+W) 1-2
1.1.2 Graphic Environment (CTRL+G) 1-4
1.1.3 Camera Environment (CTRL+I) 1-4
1.1.4 Debug Environment (CTRL+D) 1-4
1.1.5 Programming Environment (CTRL+P) 1-6
1.1.6 Functional Programming Environment (CTRL+F) 1-7
1.1.7 Board Configuration (CTRL+J) 1-8
1.1.8 System Configuration (CTRL+Q) 1-8
1.1.9 Self Test (CTRL+S) 1-8
1.1.10 Homing Procedure (CTRL+H) 1-8
1.1.11 Exit (CTRL+E) 1-8
2. Programming Environment **2-1**
2.1 Create a function 2-6
2.2 Associate macros and components to a function 2-8
2.3 Create a Macro 2-9
2.4 Define the parameters of a macro 2-10
2.5 Define the local variables of a macro 2-12
2.6 Define the global variables of a macro 2-14
2.7 Define the code of a macro 2-16
2.7.1 Define the code of a label 2-18
3. System tools **3-1**
TEST 3-2
TEST.Clear 3-3
TEST.Set 3-4
TEST.Measure 3-5
TEST.GetParN 3-6
TEST.GetParS 3-6
TEST.GetParP 3-7
TEST.SetParN 3-7
TEST.SetParS 3-8
TEST.SetParP 3-8
TEST.WriteMessage 3-9
TEST.WriteReport 3-9
VBS 3-10
VBS.Clear 3-10
VBS.Execute 3-11
EXPRESSION 3-12
Expression.Assign 3-12
IF 3-13
IF.Evaluate 3-13
ELSE 3-14
ELSE.Dummy 3-14
ELSE_IF 3-15
ELSE_IF.Evaluate 3-15

Resistor 3-16
Resistor.Clear 3-16
Resistor.Measure 3-16
Capacitor 3-17
Capacitor.Clear 3-17
Capacitor.Measure 3-17
Inductor.Clear 3-18
Inductor.Measure 3-18
Diode 3-19
Diode.Clear 3-19
Diode.Measure 3-19
Transistor 3-20
Transistor.Clear 3-20
Transistor.Measure 3-20
Fnode 3-22
Fnode.Clear 3-22
Fnode.Measure 3-22
Autic, Jscan, OpenFix, Continuity, Isolation, Short, Contact, Discharge 3-24
SYSTEM 3-25
SYSTEM.Cluster 3-26
4. Appendix 4-1
4.1 Permissions for Functions and Macros 4-2
Parameter type 4-2
Generation type 4-3
Lam type 4-3
4.2 Attributes of the macro parameter 4-4
Parameter Class 4-4
Parameter Unit 4-6
4.3 Tools- Methods - Parameters 4-12
5. Bibliography 5-1

LIST OF FIGURES[1]

Fig. 1-1 - VIVA: Prepare, verify and program execution 1-3
Fig. 1-2 - VIVA: Syntax diagrams of a VIVA program (development environment) 1-6
Fig. 1-3 - VIVA: Syntax diagrams of a VIVA program (runtime environment) 1-7
Fig. 2-1 - VIVA: Program structure and association Function-Components 2-2
Fig. 2-2 - VIVA: Programming Environment – tab "General"..... 2-3
Fig. 2-3 - VIVA: Patterns 2-4
Fig. 2-4 - VIVA: Dialog "Function"..... 2-6
Fig. 2-5 - VIVA: Dialog "Permissions" 2-7
Fig. 2-6 - VIVA: Dialog "Macro" 2-9
Fig. 2-7 - VIVA: Programming Environment – page "Parameters" 2-10
Fig. 2-8 - VIVA: Programming Environment – page "Local Variables" 2-12
Fig. 2-9 - VIVA: Programming Environment – page "Global Variables"..... 2-14
Fig. 2-10 - VIVA: Programming Environment – page "Code" 2-16
Fig. 2-11 - VIVA: Programming Environment – dialog: "Code – macroname"..... 2-18
Fig. 2-12 - VIVA: Programming Environment – dialog: "Tool Selection" 2-19
Fig. 2-13 - VIVA: Programming Environment – dialog: "Method Selection"..... 2-20
Fig. 2-14 - VIVA: Programming Environment – Selection of Locals and Parameters 2-22
Fig. 2-15 - VIVA: Programming Environment – Selection of Globals and Expressions 2-22
Fig. 2-16 - VIVA: Programming Environment – Selection of Label and Macro properties 2-22
Fig. 3-1 - VIVA: Programming Environment – dialog: "Debug Environment"..... 3-2
Fig. 3-2 - VIVA: Programming Environment – Digital – Analog Display 3-3

LIST OF TABLES

Tab. 3-1 - VIVA: Name mapping from Transistor.Measure and macro TRANSISTOR..... 3-21
Tab. 3-2 - VIVA: Name mapping from Fnode.Measure and macro FNODE..... 3-23
Tab. 4-1 - VIVA: Permissions 4-2
Tab. 4-2 - VIVA: Parameters Class..... 4-4
Tab. 4-3 - VIVA: Parameters Unit 4-6
Tab. 4-4 - VIVA: Tools- Methods- Parameters..... 4-12

[1] The figures included in this manual may not correspond to the last release of the product. Their aim is to support the topics developed in the manual.

ACRONIMS

ADC *Analog to Digital Converter*: hardware component which accepts in input an analog signal and outputs a digital value.

BJT *Bipolar Junction Transistor:* type of bipolar junction transistor made up by different layers of semiconductors drugged in different ways to form NPN or PNP transistor.

BNC *Bayonet Neill Concelman:* family of bayonet connectors use for the heading of coaxial cables.

CPCI *CompactPCI*.

DAC *Digital to Analog Converter*: hardware component which accepts as an input a digital signal and outputs an analog value

DFT *Discreet Fourier Transform*

DDFS *Direct Digital Frequency Synthesis* or point-skipping at constant clock-rate or simply DDS: technique to generate repetitive waveforms with sample scanning at a constant frequency. (point-skipping technique).

DDS *Direct Digital Synthesis*.

DSP *Digital Signal Processing*.

DUT *Device Under Test*.

FGE *Functional Guided Editor*.

FET *Field Effect Transistor*: type of *Field Effect Transistors*, where the conduction is performed through channels which may be of the P or N.

FFT *Fast Fourier Transform*: algorithm to calculate the Fourier transform of the Fourier Discrete.

GPIB *General Purpose Interface Bus*, IEEE 488.

GUI *Graphical User Interface:* allows the user to interact with the computer managing the items graphically, without requiring the learning of a series of commands through keyboard as it happens with the *Command Line Interface* (CLI).

ICT *In Circuit Test*: checks a component inserted in a circuit with other components.

I_{IH} *High Level Input Current:* Maximum high level input current of a digital device, with signal at a high logic level. (e.g.: 40 µA).

I_{IL} *Low Level Input Current:* Maximum low level input current of a digital device, with signal at a low logic level. (e.g..: -1,6mA).

I_{OH} *High Level Output Current:* Maximum high level output current from a digital device, with signal at a high logic level. (e.g.: -0,4mA).

I_{OL} *Low Level Output Current:* Maximum low level output current from a digital device, with signal at a low logic level. (e.g.: 16mA).

IEEE 488 Interface Bus to communicate with external instruments.

ITA *Interface Test Adapter*: Routing of signals from the channels of the test station to the nodes of the board. (Fixture).

IMM *Internal Multi Meter*.

JTAG *Join Test Action Group*: technology developer to facilitate the automatic test of SoB, now a standard and codified as IEEE 1149.

LED *Light Emitting Diode*

MIL-STD-1553 Military standard which defines the electrical features and the protocol of a serial bus for data exchange.

MOSFET *Metal Oxide Semiconductor Field Effect Transistor*: type of *Metal Oxide Semiconductor Field Effect Transistor*. Mainly used in the digital circuits. The conduction is performed through channels which may be of the P or of the N type.

OOP *Object Oriented Programming*: Programming technique allowing the management of instruments through modules which drive the instrument

	and on the other hand supply the procedures (or methods) which are the only controlled way to access the instrument.
PCI	*Peripheral Component Interface*. (4 peripheral slots).
PLL	*Phase Locked Loop*.
PPC	*Point per Clock* synthesis at variable clock rate or sample buffer playback synthesis: technique to generate repetitive waveforms with sample scanning at a variable frequency. (point-per-clock technique).
PXI	*PCI eXtension* for instrumentation. (7 peripheral slots).
SoB	*System on Board*: System inserted in a board.
SoC	*System on Chip*: System inserted in a chip, or VLSI circuit.
UUT	*Unit Under Test*.
VBS	*Visual Basic Script*, environment allowing the execution of procedural code under VBS syntax.
VISA	*Virtual Instrument Software Architecture*, software architecture to communicate with peripheral modules: serial, VXI and GPIB.
VL	*VIVA Language*: language used in the VIVA environment, of Seica S.p.A. to build up macro used by the VIVA functions.
VLSI	*Very Large Scale Integration*: indicates the high level of miniaturization of an electronic component.
VME	*Versa Module Europe*; Vme bus specification managed by VSO.
VNL	*VIVA Native Language*: language used in the VIVA, environment by Seica S.p.A. to build up functions and macros executed by the VIVA programs.
VXI	*VME eXtension* for instrumentation.
$\mathbf{V_{IH}}$	*High Level Input Voltage:* Minimum voltage level to consider the input signal of a digital device at a high logic level. (e.g.: 2V).
$\mathbf{V_{IL}}$	*Low Level Input Voltage:* Maximum voltage level to consider the input signal of a digital device at a low logic level. (e.g.: 0,8V).
$\mathbf{V_{OH}}$	*High Level Output Voltage:* Minimum voltage level to consider the output signal of a digital device at a high logic level. (e.g.: 2,4V).
$\mathbf{V_{OL}}$	*Low Level Output Voltage:* Maximum voltage level to consider the output signal of a digital device at a low logic level. (e.g.: 0,4V).

1. Introduction

The VIVA Software has been developed to develop and execute test programs on electronic devices (single components, boards, devices, etc.) using the systems manufactured by Seica S.p.A.

In the main dialog of the VIVA Software there is a toolbar made up by a set of buttons which allow to select one of the environments available or to exit the program:

Wizard Environment	(CTRL+W)
Graphic Environment	(CTRL+G)
Camera Environment	(CTRL+I)
Debug Environment	(CTRL+D)
Programming Environment	(CTRL+P)
Functional Programming Environment	(CTRL+F)
Board Configuration	(CTRL+J)
System Configuration	(CTRL+Q)
Self Test	(CTRL+S)
Homing Procedure	(CTRL+H)
Exit	(CTRL+E)

In the next paragraphs, before describing the Programming Environment, which is the main topic of this manual, is provided a description of the environments available, also included in the documentation quoted in the bibliography[2].

This manual is addressed to VIVA users belonging to the group "Administrators": the information supplied with this manual allow to add new macros or functions in the "Function and Macro database" (Fig. 1-1). The users belonging to the group "Programmers" are those who create, modify and verify the test programs through the Wizard Environment and use the items available in the "Function and Macro database".

[2] In the text, a number in square brackets indicates a crossed reference to another document listed in the bibliography (see the corresponding section). For example [2] indicates the following bibliographical reference:

[2] ACL Module– Internal Instruments
Code: MA-VI-ACLI00EN-06 (23-07-2009) Seica S.p.A.

1.1.1 *Wizard Environment (CTRL+W)*

The **Wizard Environment** is a guided procedure to develop test programs through eight dialogs displayed in sequence. The operator can supply to the system all the data required to develop the test program. After the first complete execution of the procedure, the operator can select the dialogs random with no need of starting the procedure anew.
The eight dialogs correspond to eight pages present in the dialog of the Wizard Environment:

Page **Start**: allows to create a new program, select an existing program, access the system maintenance environment, explore the installed manuals or the help on line.

Page **Import Data**: allows to supply the data which describe the layout and the components of the UUT.

Page **Board Data**: allows to supply additional information related to the physical features of the UUT. Here it is possible to modify the data defined in the Import Data phase and add new ones.

Page **Create Program**: allows to create the test program. For each component to be tested, the systems creates the values associated to the parameters of the software macros developed to execute the test.

Page **Multiply**: allows to multiply the program, to manage more than one UUT when these are arranged in groups to be tested before being physically separated.

Page **Board Setup**: allows to define the information required to align the coordinates of the board placed in the work area with the coordinates included in the board description recorded by the software. The operations scheduled in this page are specific for Flying Probe systems, while the page is not available for the Bed of Nails systems. The page schedules four activities which must be executed in sequence. The first two are mandatory:

1. Definition of Reference Points
2. Translate Board on Reference Points
3. Coordinates Check and Autolearn
4. Definition of Markers

Page **Verify**: allows to execute the created program with possibility of debug and autolearn. This page is the same as the page RUN but contains the icon to enter the Debug Environment. Once the activities available in the page Verify have been completed, the development phase is ended and the program is ready to be executed in production activities. The production activities can be activated in the page RUN, which can be selected directly for the subsequent executions of the program.

Page **Run**: allows to execute the created program. For the Flying Probe system, before starting the test program. It is possible to select the management mode of the markers. Instead of starting the test program, it is possible to select the Debug Environment and start the program from that environment.

During the creation of the new program, the pages allowing the operator to supply the information required to develop the test program, are available in sequence so that the subsequent page is available as soon as all the activities scheduled in the previous pages have been completed. Then, the pages are all available and can be selected randomly.
In the VIVA software, the name of the board identifies the test program; in the dialogs the name of the board indicates both the database which contains the data of a board and the test program associated to that board . The flowchart below, summarizes the activities of the Wizard Environment.

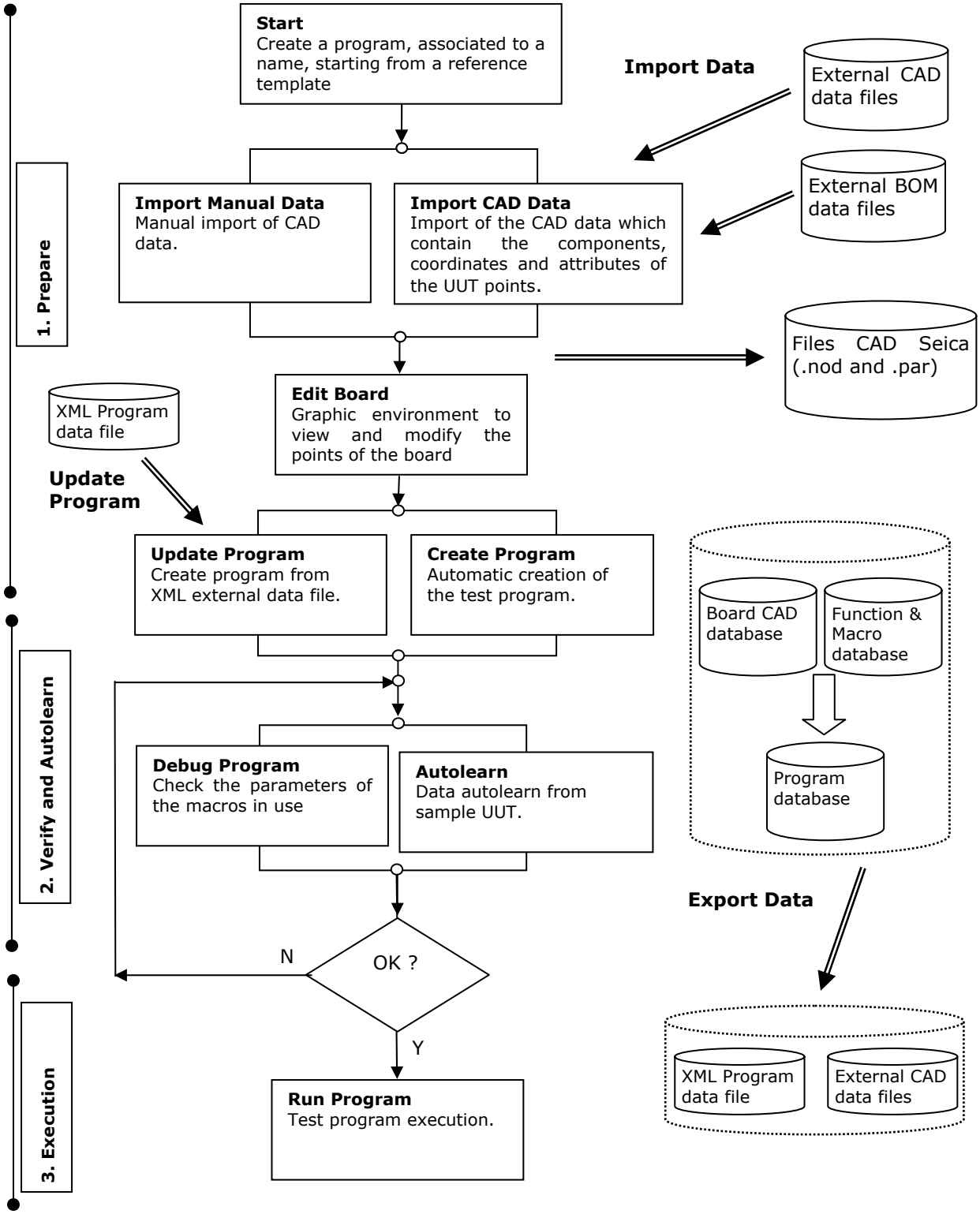


Fig. 1-1 - VIVA: Prepare, verify and program execution

1.1.2 Graphic Environment (CTRL+G)

The **Graphic Environment** allows to manage the layout of the points[3] and components which make up the unit under test. The display of the board layout can be related either to the top or the bottom side; the image of the board can be flipped or rotated all the ways round. The main dialog of the environment contains a toolbar and two selectable pages: Edit board and Rotate board. In this environment it is possible to define or modify the features of the nodes by changing the values of some attributes referred to each node. In particular:

- Side possible values: Top or Bottom
- Testability possible values: Test/Don't Test
- Technology possible values: SMD or Trough Hole
- Shape possible values: Round or Square/Rectangle or Hole
- External diameter possible values: of the external diameter of the point
- Internal diameter possible values: of the internal diameter of the point

For Flying Probe systems, at the board level, it is possible to define two points as translation points: these will take the values of the physical coordinates acquired with the camera when translating the board. If the value of the attributes modified affects the program generation, the button which allows to save the changes tells that when saving the data the test program is generated.

1.1.3 Camera Environment (CTRL+I)

For Flying Probe systems, the **Camera Environment**, through a view system, allows to:

- manage the acquisition of the physical coordinates of the board placed into the work area
- manage the acquisition of the markers used as fiducial points for the automatic translation during program execution
- execute the manual translation of the board coordinates using the two translation points defined in the Graphic Environment
- manually insert the points in the layout of the board to be tested.

The main dialog of this environment contains a toolbar and three selectable pages: Translation, Set Markers and Autolearn.

1.1.4 Debug Environment (CTRL+D)

After program creation, the **Debug Environment** allows to:

- Define the execution mode.
- enable/disable the execution on a specific section, component or macro executed in the section.
- enable/disable the repetition of the executon on a specific section, component and on a specific macro executed in the section.
- enable/disable the graphic view of the values related to the execution on a specific point
- view the result of the execution on a specific point.
- View and modify the value of settings used in the software procedure (macro) which executes the test procedure on a specific point.
- enable/disable the step by step program execution.
- enable/disable the loop execution on the program.
- enable/disable the report printing after program execution.
- enable/disable the use of markers for automatic translation when program executon starts.
- enable/disable the probe view when executing the test
  start/stop program execution.

[3] For further details on points, channels, hardware and software context, please refer to [1]

The **Debug Environment** can be used to view:

- the **sections** of a test program (corresponding to the execution of a specific kind of function)
- the **components** tested in a specific section
- the **macro used** to test each component
- for each macro, **the current macro settings**

the Debug Environment dialog is divided into several areas:

- in the upper left part, it is possible to select the test mode
- immediately below, on the left, a grid gives the hierarchical order of the items of a test program (sections, components of the section and macros used for each component)
- On the lower left part, it is possible to select summarizing information about the program
- The area on the upper right side, is used to view information related to the item selected in the second area on the left (Section, Component, Macro)

The area in the lower right part allows to view graphical information concerning program execution or the component related to the item selected in the second area on the left.

1.1.5 Programming Environment (CTRL+P)

The **Programming Environment** is used to make new functions available or to modify the existing to create VIVA program. In this environment it is possible to:

- Define the link between a kind of component and the functions which can be executed on that component through VIVA software
- Add new functions to those provided with the installation
- Define, for a specific function, the related macros
- Define, for a specific macro, the related labels
- Define, for a specific label, of a specific macro, the execution code, which consists in recalling the system instruments with the associated operation (or method) which the instrument can execute.

The description of the Programming Environment is the aim of this manual. The notes and warnings provided are aimed to give a general description of the architecture of a VIVA program and of the environment which allows to define the components of such architecture. Also the diagrams below aim to provide an idea of the structure of VIVA programs. The Fig. 1-2 shows the diagrams of a VIVA program defined in the Programming Environment. The Fig. 1-3 shows the diagrams of a VIVA program in the execution environment.

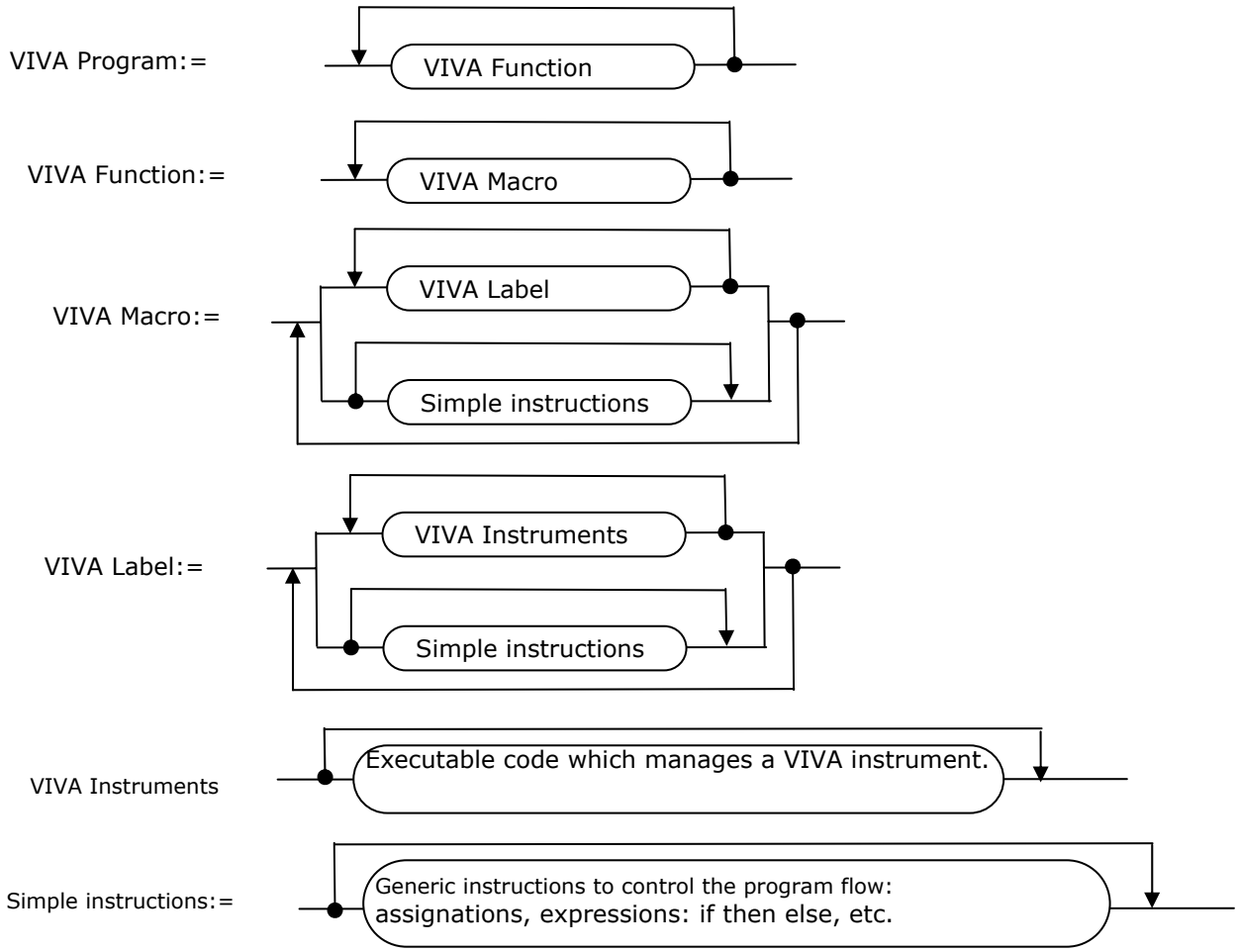


Fig. 1-2 - VIVA: Syntax diagrams of a VIVA program (development environment)

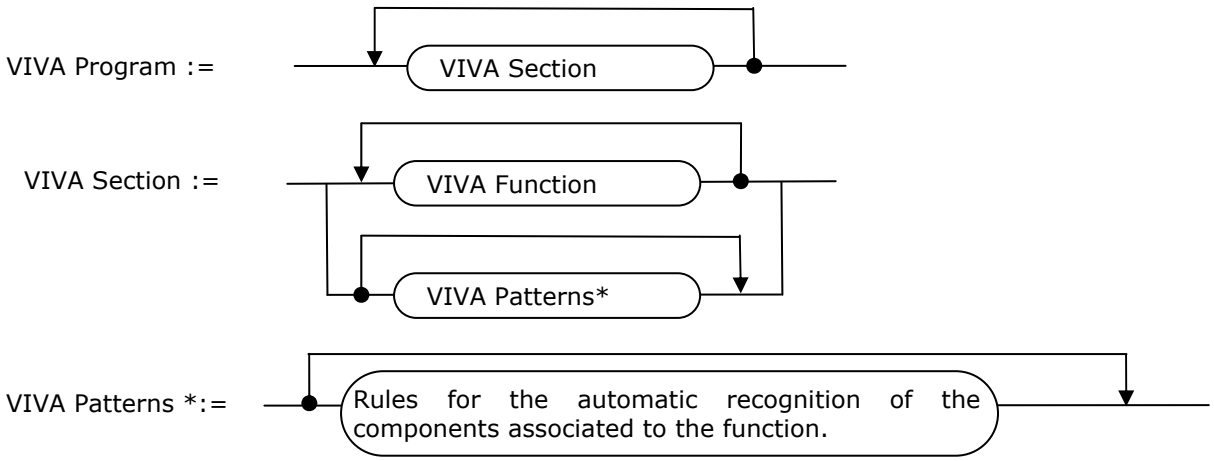


Fig. 1-3 - VIVA: Syntax diagrams of a VIVA program (runtime environment)

*Patterns VIVA: are rules present in the Programming Environment and used in the Create Program phase to create the program.

Once created, the program appears as a series of sections, where "section" means the execution of a function associated to a component.

1.1.6 *Functional Programming Environment (CTRL+F)*

The **Functional Programming Environment** is used on the TEST systems managed by the VIVA software to create functional programs. The functional programs, also named patterns, are made up by clusters; the cluster is a sequence of code written to check specific functions of a UUT.

Through a text editor, the environment allows to:

- Build single clusters
- Organize the clusters into pattern files (*.pat)
- compile the pattern files

The description of the Functional Programming Environment is not the aim of this manual. For further information on this topic, please refer to [4] or to the on-line documentation of the VIVA Software.

This environment has been quoted since some system macros have been created in this environment. These macros are defined, with the parameter "ModuleType" are defined of "VL" and "VL_DYNAMIC", type where:

VL The macro in use is written in VIVA Language (VL). The macro is executed in the static mode.
VL_DYNAMIC The macro in use is written in VIVA Language Dynamic (VL_DYNAMIC). The macro is executed in the dynamic mode.

Two other possible values for "ModulType" are:

NONE The macro in use is written in VIVA native language[4].
VBS The macro in use is written in Visual Basic Script. (VBS).

[4] The VIVA native language is a graphic language which can be used to build VIVA macros. With this language, it is possible to use parts of codes written in VL, C, VBScript languages and access directly to the software stratum of the drivers.

1.1.7 Board Configuration (CTRL+J)

The **Board Configuration Environment** allows to configure some default parameters used in the Create Program phase.

1.1.8 System Configuration (CTRL+Q)

The **System Configuration Environment** contains several information related to the system installation configuration and status. The information are available on ten pages which form the dialog of the environment.

1.1.9 Self Test (CTRL+S)

The **Self Test Environment** contains a dialog to parameterize the VIVA system self-diagnostic program.

1.1.10 Homing Procedure (CTRL+H)

For the Flying Probe systems, the button **Homing** executes the homing procedure of the axes, moving the probes out of the work area.

1.1.11 Exit (CTRL+E)

The button **Exit** stops the execution of the VIVA software.

2. Programming Environment

The Programming Environment allows to define the functions necessaries in the "Function & Macro database" (Fig. 1-1) which, combined with the structures available in "Board CAD database", are used to create test programs and check UUTs made up by **components** grouped in **devices**.

There's a **function** associated to each device, used to check the components belonging to that specific device.

The functions are made up by one or more **macros**, which form the code executed to check the components.

It is possible to parameterize the macros according to the features of the component to be checked (value of the scale to be measured, accepted tolerances, identification of the component pins, etc.)

*Note: The macros are made up by groups of code named: **labels**. The labels can be executed either in sequence or with a flow control based on the instruction: "Jump: <condition> To: <label>", executable at the end of each label.*

The labels are made up by instructions which manage the **tools** available in the VIVA Environment.

The tools are **software objects** aimed to manage the resources[5] in the VIVA Environment. The tools are identified by a name and can be accessed through an interface made up by methods identifying the operations required to the software object; it is not possible instead to access the operations executed by the software object. The instructions to manage a tool are identified by the couple <**Tool.method**>

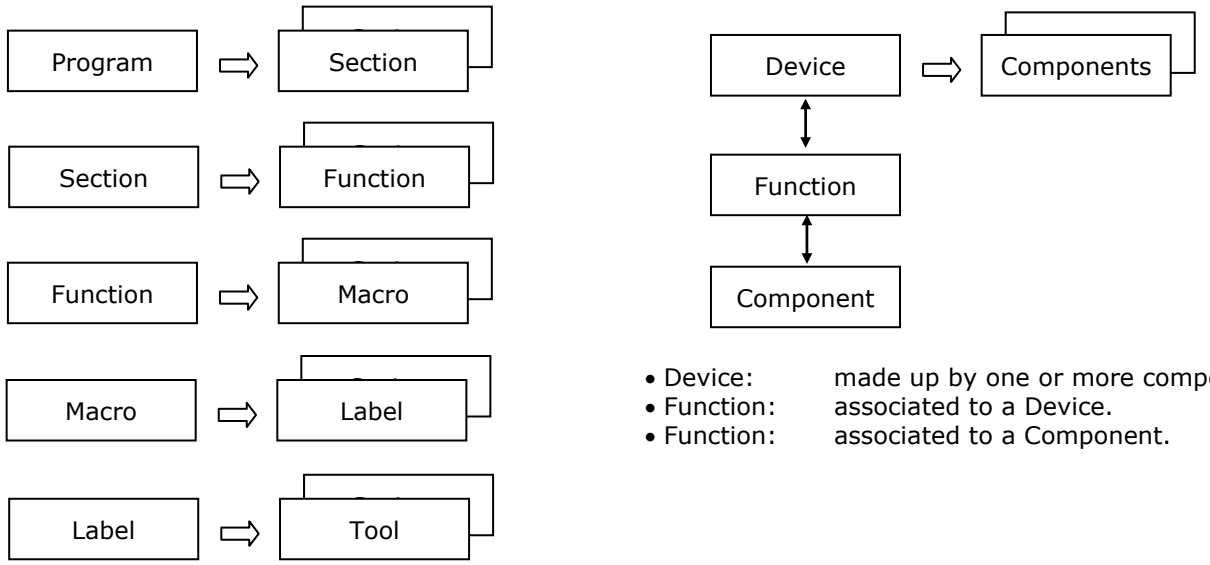
The method identifying a function to access the tool, schedules a variable number of parameters according to the method itself. A generic instruction to manage a tool will have the following format:
<**Name_Tool.Name_Method(par*1*, par*2*, ... Par*n*)**>

In a label, the control of the instructions flow is performed by the instructions of the **VIVA native language**, which can be created through a graphic interface present in the Programming Environment.

For every function present in the Function & Macro database, the attribute **Section** allows to group one or more functions with a section name which can be defined by the user. The section name will appear in the Debug Environment to identify the set of components checked with the same macros. In the Debug Environment the name of the function does not appear; it is used only in the Create Program phase to associate the appropriate macros to the components to be checked.

The drawings in Fig. 2-1 show the links of the different structures related to a program executable in the VIVA Environment. The Programming Environment allows to define the composition and links of the structures.

[5] The term resources does not merely identify the HW instruments used to manage the measurements, but also the SW instruments implementing the VIVA native language.



- Program: made up by one or more Sections.
- Section: made up by one or more Functions.
- Function: made up by one or more Macros.
- Macro: made up by one or more Label.
- Label: made up by one or more Tool.

- Device: made up by one or more components.
- Function: associated to a Device.
- Function: associated to a Component.

Fig. 2-1 - VIVA: Program structure and association Function-Components

The page "Create Program" in the Wizard Environment will allow to create the test program according to the data present in Board CAD database, which describe the UUT, and the structures present in Function & Macro database defined in the dialogs described in the next paragraphs.

As shown in Fig. 2-1, a macro is a sequence of code executed to check the components of a UUT. A macro is normally used to check the functionality of a component. When the component presents more than one functionality, it might be necessary to use more than one macro for the complete check of the component. (e.g., a diode has the function for direct polarity and the function for reverse polarity). The definition of the functions to be checked, i.e. of the macros to be executed for a specific component, can be defined by creating a function which groups one or more macros and is associated to one or more components.

The button "Programming Environment" in the toolbar of the main VIVA dialog, opens the Programming Environment (see Fig. 2-2). The controls available in the tab: "General" allow to manage three areas:

- Management of functions and patterns
- Management of macros
- Management of complex devices

Besides, on the bottom right side of the dialog, there are four buttons used to manage the transactions associated to the five pages of the dialog. ("General", "Parameters", "Local Variables", "Code" and "Global Variables)

OK : - If pressed, saves all the changes and exits the dialog Programming Environment.

Cancel : - If pressed, exits the dialog Programming Environment without saving the changes.

Apply : - If pressed, applies the changes without exiting the dialog Programming Environment. (Commit of the transaction)

Restore : - If pressed, all the changes made with the last selection of buttons: "OK" or "Apply" are deleted without exiting the dialog Programming Environment. (RollBack of the transaction)

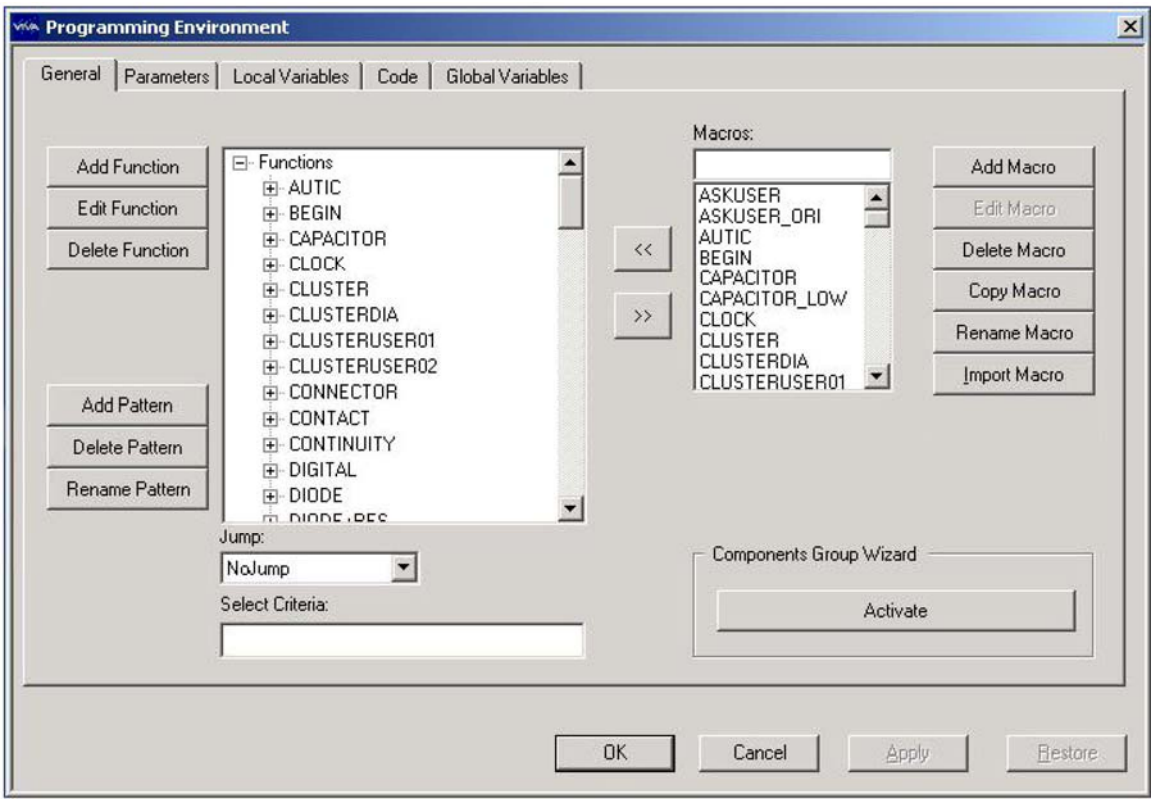


Fig. 2-2 - VIVA: Programming Environment – tab "General"

<u>Management of functions and patterns:</u>

Add Function : - opens the dialog "Function", which allows to create a new function.

Edit Function : - opens the dialog "Function", which allows to modify the attributes of a function.

Delete Function : - if pressed, a message appears to confirm the action". This button is enabled only with the "System Library" template. Once the program (board) has been selected, the button is not enabled.

Add Pattern : - this button opens a dialog where it is possible to add a pattern associated to the selected function. The pattern indicates the rule which defines the association of the function to the components described in the board database. The rule is used in the phase "Create program". The rule is defined by an alpha-numeric string with the following syntax:

aaaaaaaa	identifies the device to whom the function is associated. (all the components belonging to the device named. aaaaaaaa).
aaa*	identifies the device to whom the function is associated. (all the components belonging to the device with name. beginning with: aaa).
##aaaaa	identifies the component to whom the function must be associated. (component name:aaaaa)
##aaa*	identifies the component to whom the function must be associated. (all the components whose name begins with: aaa)

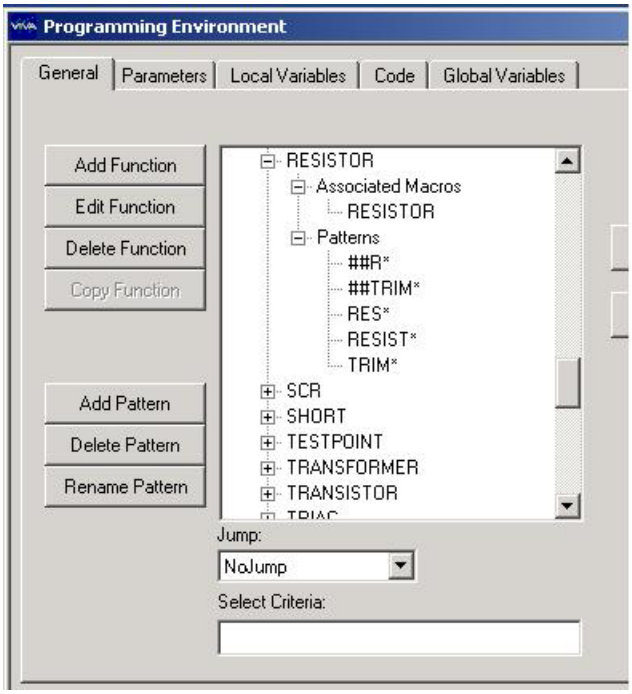


Fig. 2-3 - VIVA: Patterns

It is possible to define more than one pattern for a function, as explained in Fig. 2-3, which displays the patterns associated to the function "RESISTOR", present in the VIVA Software database.

Delete Pattern : - if pressed, the system displays a message and the user is asked to confirm the action (delete the pattern selected in the list on the right side of the dialog).

Rename Pattern : - if pressed, opens a dialog which requires to confirm the action (rename the selected pattern).

Jump: - The field "Jump:" allows to define the sequence of execution of a function. It is relevant only for functions made up by more than one macro.. In the field "Jump:" the following values can be selected (dropdown menu).

NoJump	after the execution of the current macro the program will continue with the subsequent macro.
Jump	after the execution of the current macro the program will go to the end of the function, i.e. to the step next to the last macro of the function.
JumpOnOk	after the execution of the current macro with no errors the program will go to the end of the function, i.e. the step next to the last macro of the function.
JumpOnError	after the execution, with errors, of the current macro, the program will go to the end of the function, i.e. to the step next to the last macro of the function.

Execution with no errors means that the different checks executed by the macro ended with pass, i.e. the values measured are included in the accepted range of the expected value.

WARNING: Select Criteria: - The field allows to indicate the criteria used in the phase "Create Program" with regard to the section that will make up the test program. The criteria is formed by an instruction in *SQL language. The modification of this field is reserved to VIVA Software developers or to expert users who have knowledge of VIVA database and *SQL language.

<< : - this button is used to add macros to the list of macros which make up a function.

>>: - this button is used to remove macros from the list of the macros which make up a function.

Management of macros:

Add Macro : - To activate the button, it is necessary to type in the field: "Macros:", located above the control ListBox which displays the macros available, the name of the new macro. The button opens the dialog to create a new macro.
Edit Macro : - The button opens the dialog "Macro" to modify the attributes of a macro.

Delete Macro : - The button opens a dialog where the user is asked to confirm the action (delete the selected macro). This button is enabled only with the "System Library" template and not when a program is selected.

Copy Macro : - The button opens a dialog where the user is asked to confirm the action (copy the selected macro).

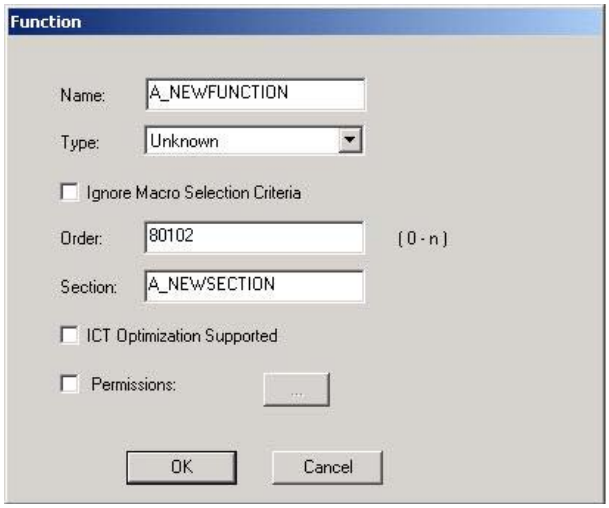
Rename Macro : - The button opens a dialog where the user is asked to confirm the action (rename the selected macro).

Import Macro : - The button opens a dialog where it is possible to select the database of the program (extension .mdb) from where the macro shall be imported. Then, select the macro to import from the list of macros displayed.

Management of complex devices:

In the section "Component Group Wizard" the button "Activate" allows to add a complex device to the board database, as explained in[4]

2.1 Create a function



A function is a group of one or more macros which can be associated to a component, or to a type of components, to check its functionalities.

To create a new function press the button "Add Function" in the page: "General" of the dialog: "Programming Environment". To fill in the fields of the dialog displayed, follow the instructions below.

The fields available in the dialog "Function" (see fig. 2-4) are:

Fig. 2-4 - VIVA: Dialog "Function"

Name: - in the field "Name" type the name of the function to be created (e.g. A_NEWFUNCTION).

Type: - in the field "Type" select, for the current function, the kind of management executed in the phase "Create Program". The selectable values are identified with the names of the functions available with the VIVA Software. The field allows to select the same rules of creation used for the function present in the field "Type". (ICT tests, tests previously requiring the execution of the macro POWER, Fan, Autic, Jscan, Openfix tests, etc.)

Ignore Macro Selection Criteria: - if flagged, the criteria defined with the field "Select Criteria" is not taken into consideration in the phase "Create Program"

Order: - the field "Order" is used to define the position of the section in the list of sections of the test program.

Section: - in the field "Section" type the name of the section to be assigned to the function identified with parameter "Name:".

ICT Oprtimization Supported: - if flagged, optimizes the function in the test phase

Permissions: - if flagged, allows to define the permissions for the function identified by parameter "Name:". The "Permission" associated to a function identify the management mode of the function itself. If the checkbox is flagged, the button **...** opens the dialog: "Permissions" where it is possible to define the execution mode.

...: - opens the dialog "Permissions", Fig. 2-5, where it is possible to define the permissions described in Tab. 4-1.

Ok: - The button exits the dialog "Function". If all the fields of the dialog have been filled in correctly, the new function will appear in the corresponding list.

Cancel : - If pressed, the dialog "Function" is closed and the possible values assigned to the field will not be applied.

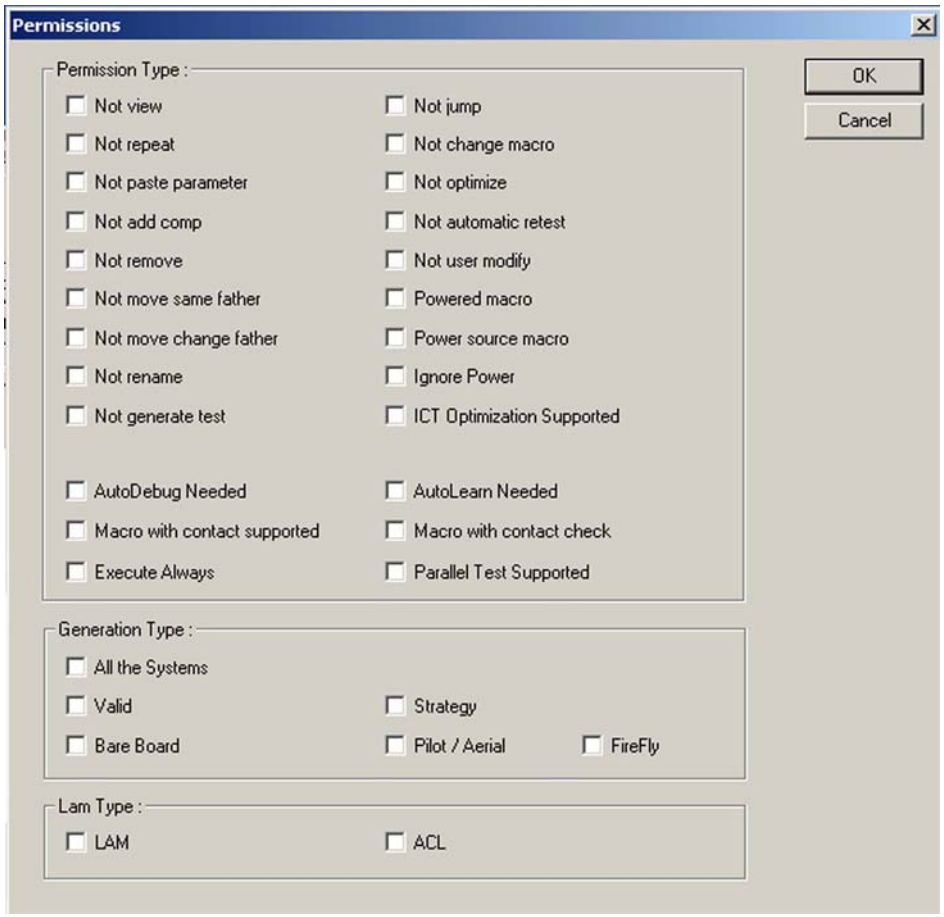


Fig. 2-5 - VIVA: Dialog "Permissions"

2.2 Associate macros and components to a function

Once the function has been created, and its name and attributes appear in the "Function & Macro database", it is necessary to define the macros which make up the function and, possibly, the components which can be checked with that function.

How to associate a macro to a function:

The buttons **<<** and **>>** available in the dialog "Programming Environment" allow to define the macros which make up a function.

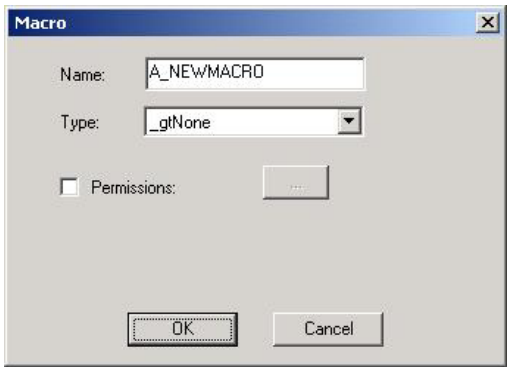
From the list of the existing functions:

- Select the function (e.g. A_NEWFUCTION).
- Select the item: "Associated Macros" in the selected function.
- Select a macro in the list of the macros available. The name of the selected macro is displayed in the field: "Macros:".
- Select the button: << to assign the macro to the selected function.
- To add other macros, repeat the two previous steps.
- Once the composition of the function has been defined, if necessary, select the item: "Patterns" to add the rules to associate the created function to the components or to the UUT devices.
- The button: >> can be used to remove a macro from the list of the macros which make up a function.

How to associate components to a function:

The structure "Pattern" allows to associate the components to a function "Patterns". The buttons available are: "Add Pattern", "Delete Pattern" and "Rename Pattern".

2.3 Create a Macro



A macro is a sequence of code used to check the functionality of a component.

To create a new macro, press the button: "Add Macro" in the page "General" of the "Programming Environment".

To create a macro correctly, fill in the fields of the dialog following the instructions below:

Fig. 2-6 - VIVA: Dialog "Macro"

Name: - in the field "Name" type the name of the new macro (e.g. A_NEWMACRO).

Type: - in the field "Type" select, for the current macro, the kind of management executed in the phase "Create Program". (ICT tests, tests requiring the previous execution of the macro POWER, Fan, Autic, Jscan, Openfix tests, etc.)

Permissions: - if selected, the checkbox indicates if it is necessary to define "Permissions" for the macro identified with parameter "Name:". The "Permission" associated to a macro, identify the execution mode of the macro itself. When the checkbox is flagged, the button **...** opens the dialog "Permissions" where it is possible to define the execution mode.

...: - The button opens the dialog: "Permissions", Fig. 2-5, where it is possible to define the permissions at the macro level (see Tab. 4-1).

Ok: - If pressed, exits the dialog "Macro" and, if the fields of the dialog have been filled in correctly, the new macro is displayed in the list.

Cancel : - If pressed, exits the dialog "Macro", and the possible values typed in the fields of the dialog will not be applied.

When a new macro is created, its code contains, by default, a "label" named "DUMMY". This "label", contains three instances of the tool "TEST" activated with the methods: Clear, Set and Measure.

```
- DUMMY
+------TEST.Clear()
+------TEST.Set(V,0[V],0[V],0[V])
+------TEST.Measure(1[V],SET ERROR,NONE)
```

2.4 Define the parameters of a macro

Once the macro has been created and its name and attributes appear in the "Function & Macro database", it is necessary to define the parameters accepted by the code which forms the macro itself.
The parameters are variable which contain values managed by the code of the macro. The dialog displayed in the figure below shows all the controls used to defined the parameters of a macro.

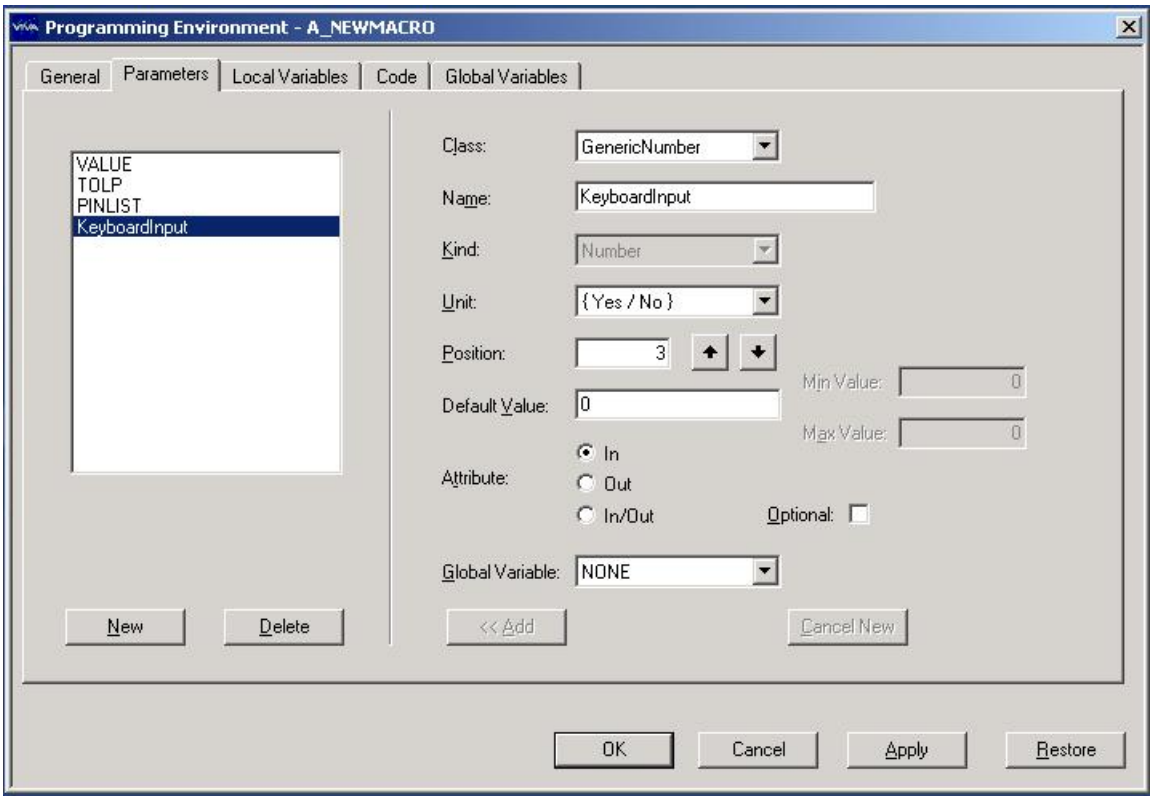


Fig. 2-7 - VIVA: Programming Environment – page "Parameters"

The page "Parameters", in the dialog "Programming Environment", allows to define the parameters used by a macro. Once a new macro has been created, there are three default parameters which may be used, modified or removed. The list of the parameters used is displayed on the left side of the dialog. If a parameter in the list is selected, on the right side of the dialog will be displayed the parameters which can be modified using the controls available in the dialog. Besides, these controls also allow to add, remove and define the parameters used by a macro.

New : - if selected, the area on the right side of the dialog is deleted and it is possible to define the attributes of the new parameter (the buttons "<< Add" and "CancelNew" are enabled to catalog or cancel the new parameter.

Delete : - If pressed, it deletes the selected parameter.

Add : - If selected, adds the parameter to the list of the parameters available for the specific macro.

CancelNew : - Deletes the right side of the dialog which contains the attributes of the new parameter. The operation is completed by automatically selecting the first parameter of the macro and displaying the related attributes.

Class: - in the field "Class" select the type of data assigned to the parameter. The classification is related to the scales of the external mode, which is in this case, the test environment with reference to the electric values or measurement values with thresholds (e.g. resistive, capacitive, voltage, etc. or expected value, positive tolerance, negative tolerance, etc). The values used by the VIVA Software in the phase "Create Program" to update the values of the parameter are included in the CAD data of the UUT: With regard to the field "Class" a particular attention is required: it is necessary to select data compatible with the information contained in the CAD data of the board. The Tab. 4-2 contains the complete list of the type of data available to define the parameters of a macro. For further information, please refer to the online documentation of the VIVA Software.

Name: - In the field "Name" type the name of the variable identifying the parameter. This is the name displayed in the Debug Environment when the user selects the page "Macro View".

Kind: - identifies, with reference to the defined variable, the type of data processed by the code. The type of data identified by the field "Kind" represent groups of the types of data which can be defined with parameter "Class". This field is output only, the user can not modify it. The value will depend on the value assigned to the field "Class". The possible values for this field are:

String the parameter will contain a string, i.e. a sequence of alpha-numeric characters.
Number the parameter will contain numeric values.
Pinlist the parameter will contain a list of channels, whose syntax is formed by one or more whole numbers separated by (-).

Unit: - indicates the measurement unit of the scale identified by the variable. The complete list of the measurement units used to define the macro parameters is available in Tab. 4-3. For further information, please refer to the online documentation of the VIVA Software.

Position: - indicates the position of the parameter in the list of parameters. The two buttons on both sides of the field "Position", allow to change (upwards or downwards) the position of the current parameter in the list.

Default Value: - indicates the default value assigned to the variable which identifies the parameter.

Min Value: - indicates the minimum value of the variable. It is enabled only when the selected measurement unit is associated with values with a minimum and maximum.

Max Value: - indicates the maximum value of the variable. It is enabled only when the selected measurement unit is associated with values with a minimum and maximum..

Attribute: - Indicates the type of parameter from the point of view of the execution. The possible values are:

In The parameter contains values used only in input by the code of the macro.
Out The parameter contains values used only in output by the code of the macro.
In/Out The parameter contains values used both in input and output by the code of the macro.

Optional: - If flagged, it means that the parameter is optional. For further information, please refer to [1].

Global variable: - identifies the global variable used to assign a default value to the parameter. If the user types a value in this field, the value assigned to "Default Value" is not relevant; hence, to grant a good readability of the program, when the field "Global variable" contains values other than "NONE" the field "Default Value" shall not be filled in.

2.5 Define the local variables of a macro

The local variables are used during the execution of a macro. The local variables can be defined in the dialog displayed in Fig. 2-8, and are valid only for the code of the macro. To use variables valid for the whole program, it is necessary to use the global variables.

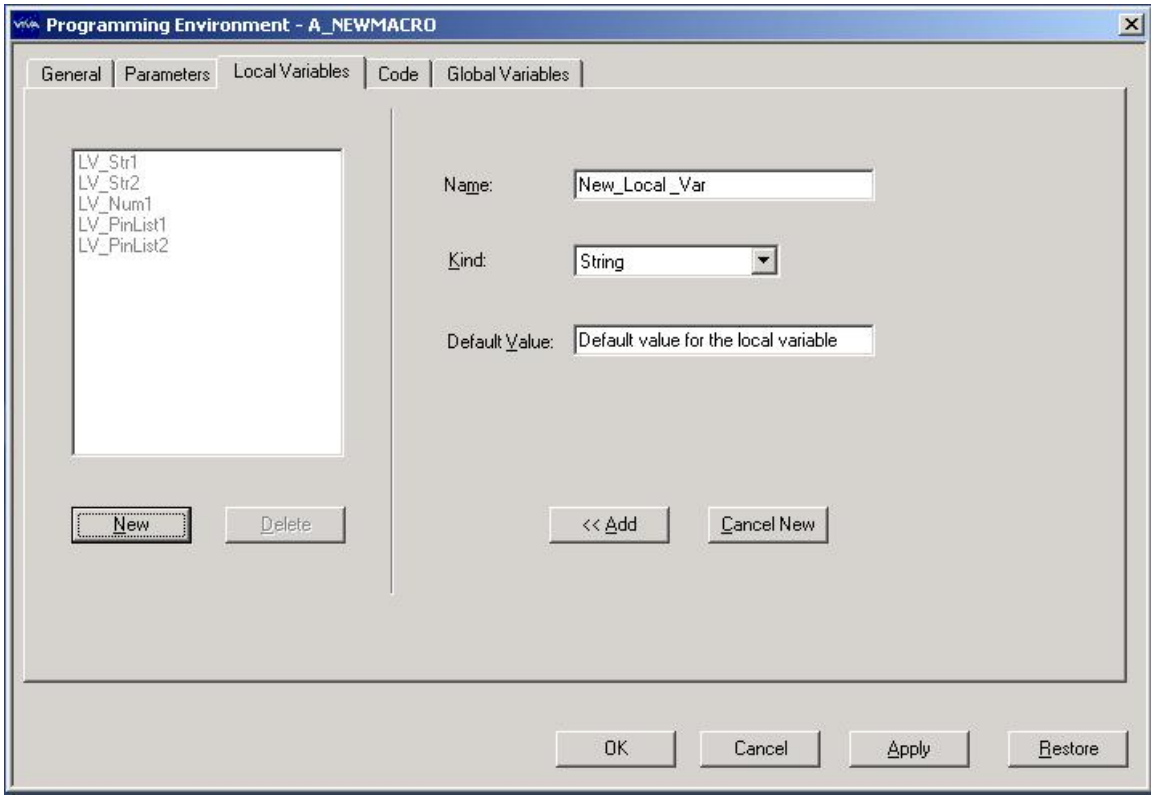


Fig. 2-8 - VIVA: Programming Environment – page "Local Variables"

The page "Local Variables" in the Programming Environment, allows to define the local variables used in a macro. The list of the local variables is displayed on the left side of the dialog. When a local variable is selected from the list on the left, on the right side will be displayed the attributes of the variable which can be modified using the controls available in the dialog. The same controls allow to add, remove and define the local variables used by a macro

New : - The button "New" deletes the value in the field "Name" on the right side of the dialog; then it is possible to define the variable (the two buttons "<< Add" and "CancelNew" are enabled to catalog or to delete the new variable

Delete : - When pressed, deletes the selected variable from the list of the local variables.

Add : - to add the local variable to the list of the variables scheduled for the specific macro.

CancelNew : - If selected, the area on the right side of the dialog is deleted. The operation is completed by automatically selecting the first variable present in the list, showing the corresponding attributes.

Name: - in the field “Name” type the name of the local variable.

Kind: - indicates, with regard to the selected variable, the type of data processed by the code. The possible values are:

String	the variable contains a string, i.e. a sequence of alpha-numeric characters.
Number	the variable contains a numeric value.
Pinlist	the variable contains a list of channels, whose syntax is formed by one or more whole numbers separated by (-).

Default Value: - indicates the default value assigned to a local variable. The default value for the value must be compatible with the type of data defined by parameter “Type”.

2.6 Define the global variables of a macro

The global variables are valid for the whole program and not only for the specific macro. The global variables can be defined in the dialog displayed in Fig. 2-9, similar to that in Fig. 2-8, which container in the dialog name the name of the macro meaning that the variable defined in that dialog was related to the macro only.

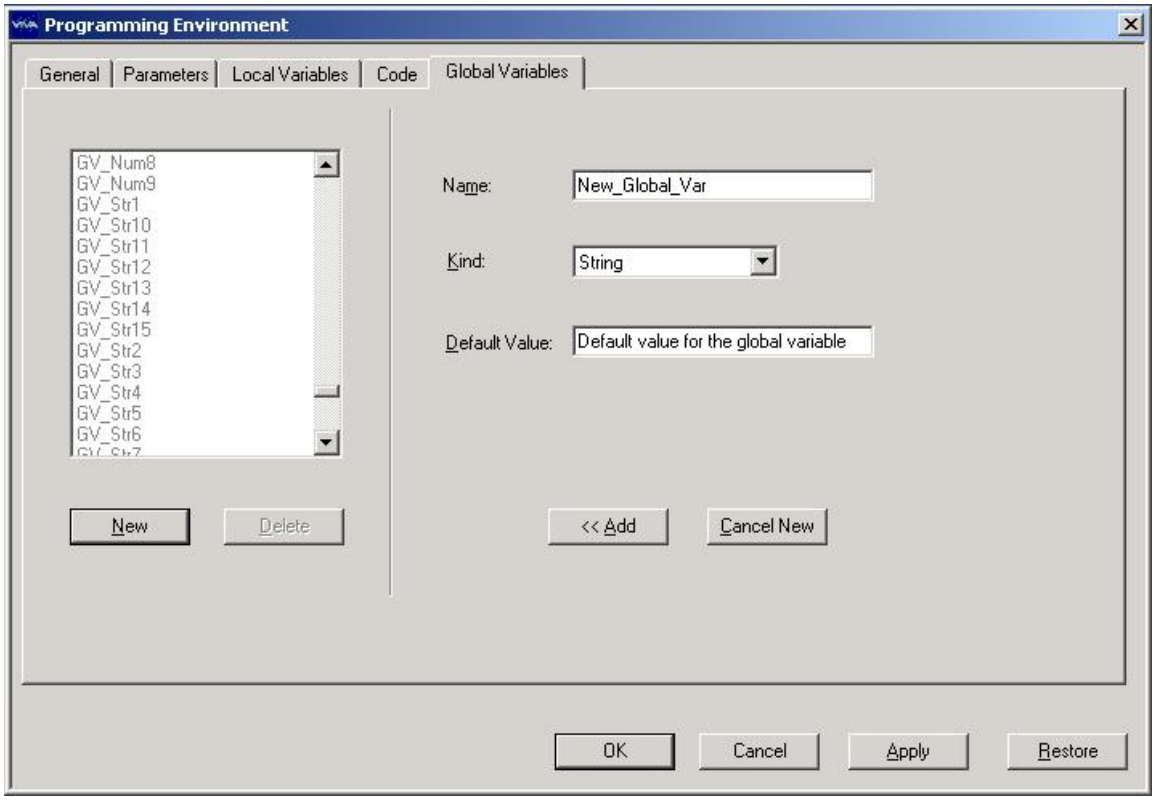


Fig. 2-9 - VIVA: Programming Environment – page "Global Variables"

The page "Global Variables" in the Programming Environment, allows to define the global variables used in a test program. The list of the defined global variables is displayed on the left side of the dialog. When a global variable is selected from the list, on the right side are displayed the attributes of the variable which can be modified using the controls available in the dialog. The same controls allow to add, remove and define the global variables used by a program.

New : - The button "New" deletes the value of the field "Name" on the right side of the dialog and allows the definition of the variable (the two buttons "<< Add" and "CancelNew" are enable to catalog or cancel the new variable).

Delete : - The button "Delete" the selected variable is deleted from the list of the global variables.

Add : - The button "Add" adds the variable to the list of the global variables scheduled for the program.

CancelNew : - The button "CancelNew" deletes the area on the right side of the dialog. The operation is completed by automatically selecting the first variable present on the list, showing the corresponding attributes.

Name: - in the field "Name" type the name of the global variable.

Kind: - indicates, with regard to the selected macro, the kind of data processed by the code. The possible values are:

String the variable contains a string, i.e. a string of alpha-numeric characters.
Number The variable contains numeric data.
Pinlist The variable contains a list of channels, whose syntax is formed by one or more whole numbers separated by (-).

Default Value: - indicates the default value assigned to the selected variable. The default value must correspond to the type of data defined by the parameter kind.

The variable available with the VIVA software supplied are used by the system macros and the basic software for the test systems management. The user who needs global variables to develop his own macros should define new global variables without modifying the variables available with the VIVA software supplied by Seica S.p.A. The variables whose name begins with "_" (underscore) are used as constants and never modified, neither by the basic VIVA software.

2.7 Define the code of a macro

As already mentioned above, the code which makes up a macro is a group of blocks of codes named "Label". The labels can be executed in sequence or with a flow control based on the instruction: "Jump: <condition> To: <label>", executable at the end of each label. The labels are formed by instructions which manage the **tools** available in the VIVA environment.

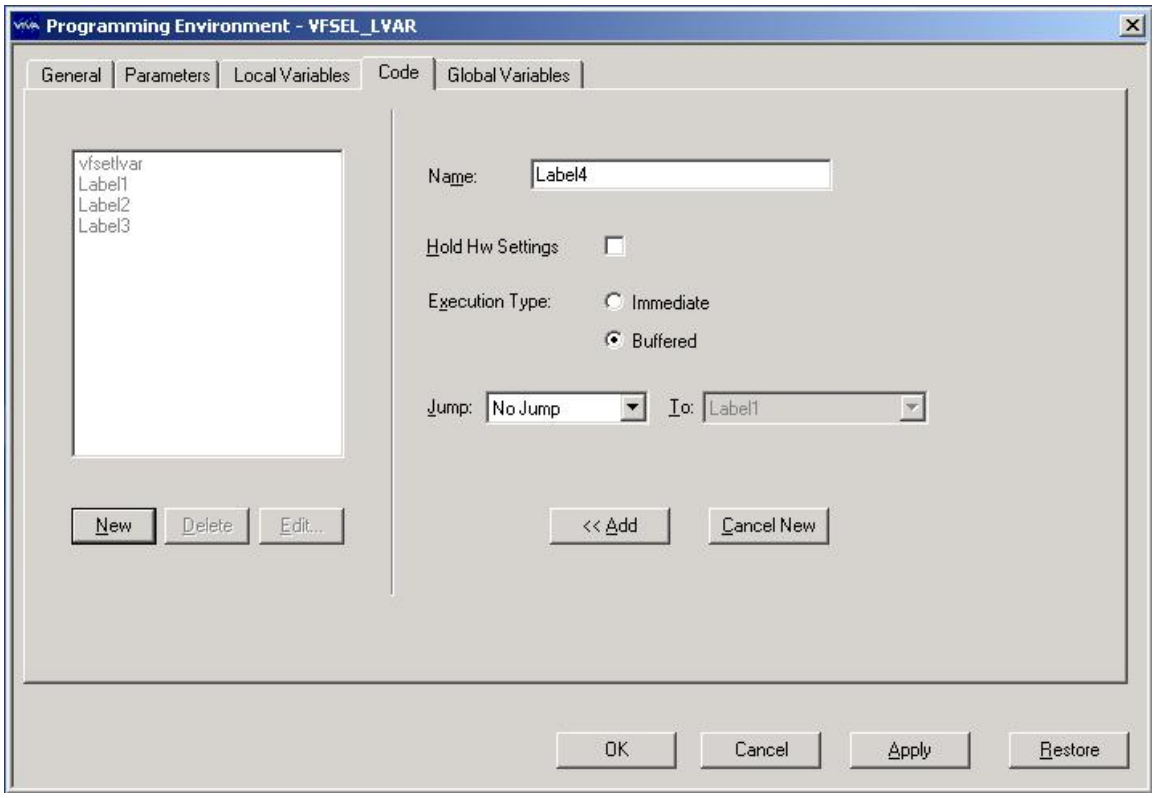


Fig. 2-10 - VIVA: Programming Environment – page "Code"

The page "Code" in the Programming Environment, allows to write the code which makes up the macro. The list of the "Labels" is displayed in the area on the left side of the dialog. When a "label" is selected from the list, on the right side are displayed the attributes of the "label" which may be modified with the controls available in the dialog. The same controls allow to add, remove and define the "labels" which make up a macro.

New : - The button "New" deletes the content of the field "Name" on the right side of the dialog and then it is possible to define the "label" (the two buttons "<< Add" and "CancelNew" are enabled to catalog or to delete the new "label").

Delete : - The button "Delete" deletes the selected label from the list of the labels available.

Edit... : - The button "Edit…" opens the dialog "Code – macro-name" where it is possible to type the code of the label expressed in VNL – VIVA Native Language.

Add : - The button "Add" adds the "label", to the list of the labels scheduled for that macro. When the button **Add** is pressed, the buttons **Delete** and **Edit...** are enabled.

CancelNew : - The button "CancelNew" deletes the area on the right side of the dialog, which contains the attributes of the label. The operation is completed by automatically selecting the first label present in the list, showing its attributes.

Name: - in the field "Name" type the name of the new "label" to be defined, or the new name of a label selected from the list.

Hold HW Settings: - this checkbox allows to define if, during the execution of the current instruction, for the instructions managing the test instruments, it is necessary to maintain the instrument settings defined with the previous instructions. **This option is relevant only for the instruments present on the LAM module**. See the instruction HLD described in [5].

Execution Type: - Indicates the type of execution scheduled for the label code, with regard to the instructions which manage the test instruments present on one of the two modules (LAM or ACLAM). The possible values are:

Immediate	The instruction is interpreted and immediately executed by the instrument.
Buffered	The instruction is interpreted and buffered but not executed by the instrument. The buffered instructions are sent to the instruments when it is interpreted the execution of an instruction to execute a measurement.

Jump: - the field "Jump:" allows to define the flow of execution of the labels which make up a macro. If the field "Jump" is selected, it is possible to define the type of jump to be executed after executing the code of the current label. The possible values for the field "Jump" are:

NoJump	After the execution of the current label the program flow continues with the execution of the subsequent label.
Jump	After the execution of the current label, the program flow will pass to the label typed in the field "To:" next to the field: "Jump:".
JumpOnOk	after the execution, with no errors, of the current label, the program flow will pass to the label typed in the field "To:" next to the field: "Jump:".
JumpOnError	After the execution, with errors, of the current label, the program flow will pass to the label typed in the field "To:" next to the field: "Jump:".
JumpOnPrevious	if during the execution of any of the labels previous to the current an error occurred, the program flow will pass to the label typed in the field "To:" next to the field: "Jump:".

Execution with no errors means that the checkbox has not been flagged to indicate the presence of errors. For further information, please refer to the description of the parameter "TestResult" in [1].

2.7.1 Define the code of a label

After selecting a label from the list of the labels scheduled for a macro, the button "Edit…" or the double click on the selected label, open the dialog "Code – macroname", displayed in Fig. 2-11. The dialog provides a graphic interface to add the code, expressed in VNL – VIVA Native Language.

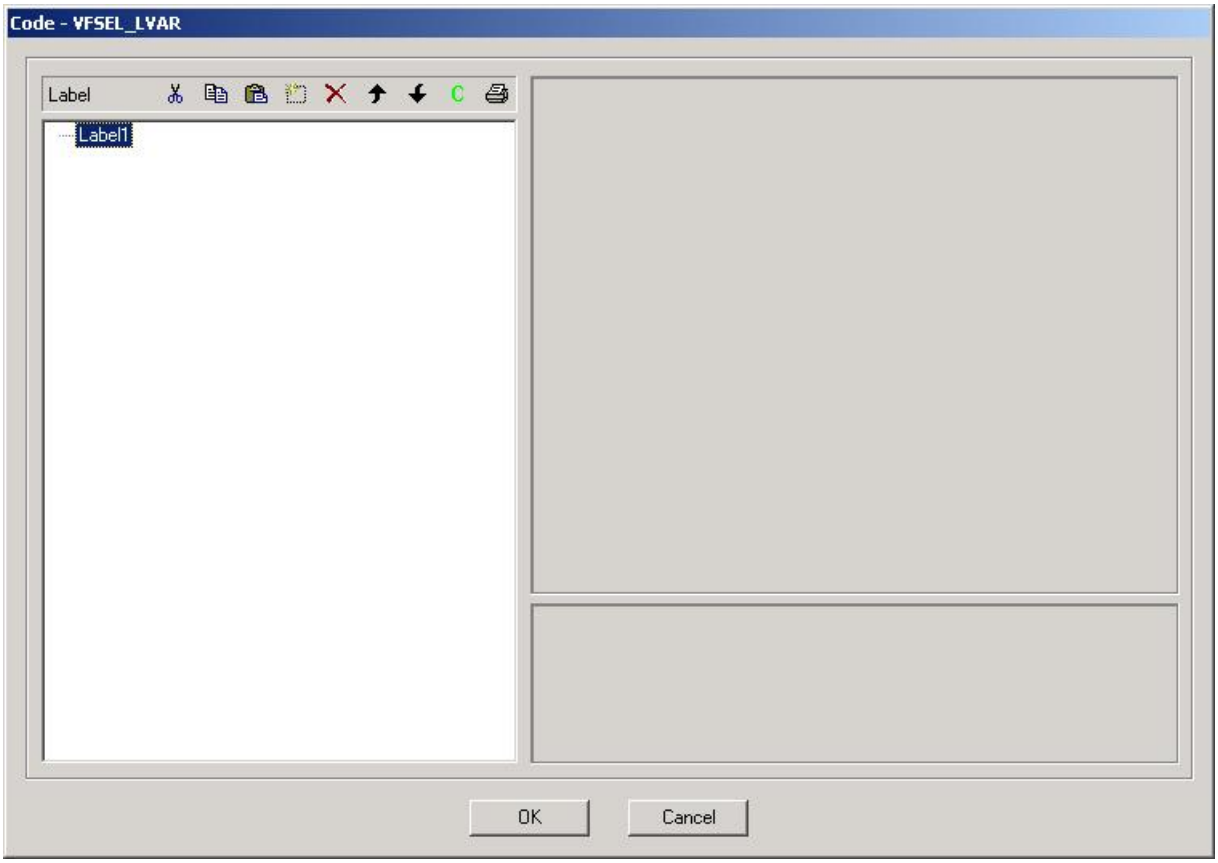


Fig. 2-11 - VIVA: Programming Environment – dialog: "Code – macroname"

On the top left side of the dialog a toolbar allows the following operations:

Cut	The selected tool is removed from the list of the instruments of the label and added to the system clipboard.
Copy	The selected tool is copied to the system clipboard.
Paste	The tool present in the system clipboard is copied and is placed below the tool currently selected.
New	Opens the dialog "Tool Selection", where it is possible to select a tool to be added to the list of the tools used by the label. It is also possible to choose a method among the available for the selected tool. The Fig. 2-12 , show san example of the tools displayed in the dialog: "Tool Selection".
Delete	The selected tool is removed from the list of tools of the Label. The removed tool is not copied in the system clipboard as it happens with button "Cut".
MoveUp	The selected tool is moved up in the list of the tools in use.
MoveDown	The selected tool is moved down in the list of the tools in use.
Comment	the line of code which uses the tools is turned into a comment, i.e. will not be executed.

Print Prints all the tools of the label. The printed document will contain the following information:

```
Macro: "macro_name"        Label: "label_name"

Tool_1.method_1(arguments of the method_1,…)
…
Tool_i.method_j(arguments of the method _j,…)
…
```

Below, an example of the dialog displayed to select the tools used by the label which make up the system macros.

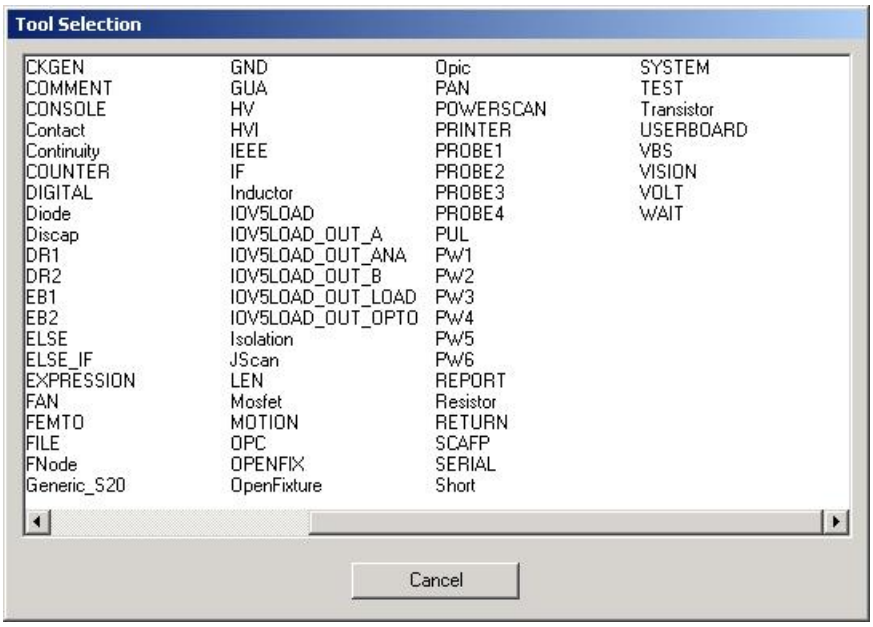


Fig. 2-12 - VIVA: Programming Environment – dialog: "Tool Selection"

Once a tool has been selected, a dialog opens to select the methods available and define the value to be assigned to the possible arguments of the method. (Fig. 2-13).

The definition of values to be associated to the possible arguments is performed through seven buttons located on the bottom left side of the dialog.

These seven buttons are the only mode to create the sequence of instructions, written in VNL, for a "label". The instructions are validated, it is not necessary to interpret or compile them before execution.

Six of the buttons allow selection only; the button [Value:……] instead, allows input from keyboard; hence, it is important to pay attention to the values input, which must correspond to the three types of data scheduled for the information processed by the VIVA software, i.e.: String, Number and PinList.

Once you have selected a method, an argument of the method and defined the values for the argument, the button Done adds the instruction (in VNL) in the area on the left below the toolbar available to manage the labels.

When the tools schedule more than one method, the button Done will display the instruction with the following syntax :

<ToolName.MethodName(current value argument1, current value argument2, etc.)>

When the tool schedules a single method, the VNL syntax may not indicate the tool name and/or method. For example:

EXPRESSION.Assign(Var,Expression) becomes:	Var = Expression
IF.Evacuate(Condition) becomes:	IF (condition)

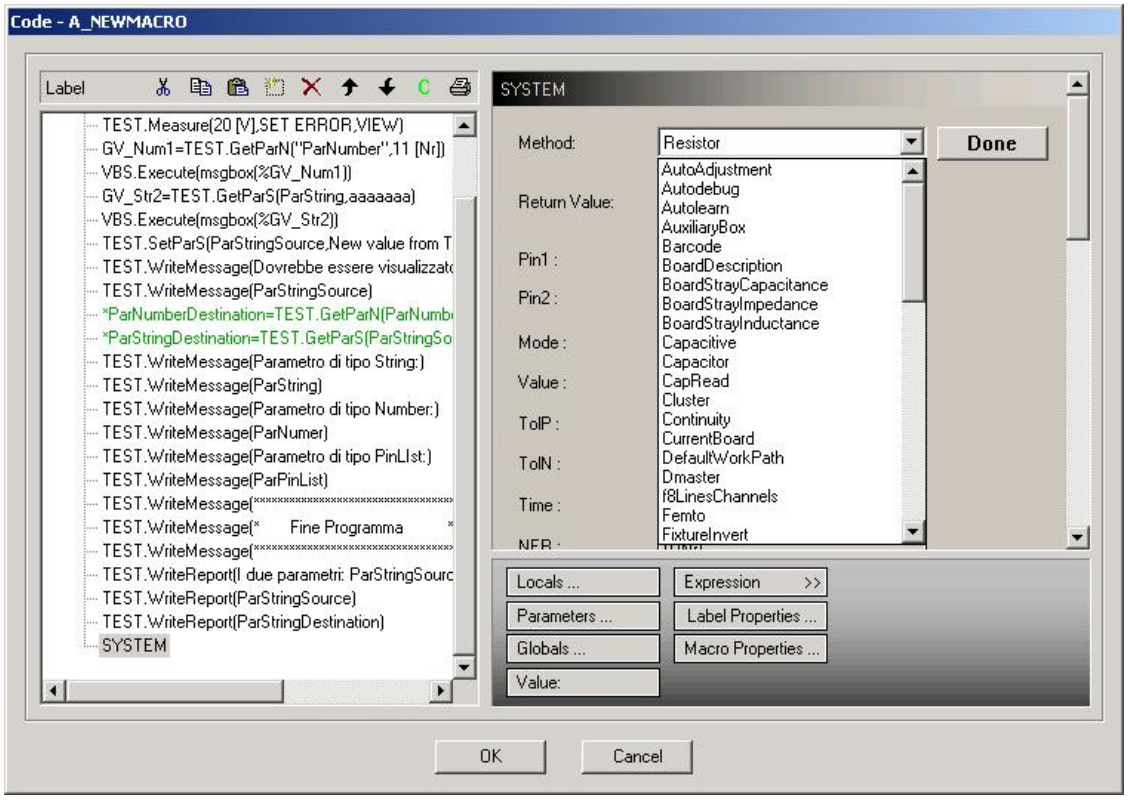


Fig. 2-13 - VIVA: Programming Environment – dialog: "Method Selection"

Below, a description of the buttons available in the dialog:

Locals ... : to select the local variable from the list of local variables defined for the macro. (Fig. 2-14).

Parameters ... : to select the parameter from the list of parameters defined for the macro. (Fig. 2-14).

Global ... : to select the global variable from the list of local variables defined for the macro. (Fig. 2-15).

Value: : The button enables a field where it is possible to add values from keyboard. The values added must correspond with the destination of the input value. Press "Enter" to confirm the input and button **Done** to add the value in the VNL instruction.

Label Properties ... : to define the label properties, i.e. how ends the execution of the label (FAIL/RETVAL). (Fig. 2-16)

Macro Properties ... : to define the macro properties, i.e. how ends the execution of the "macrol". (FAIL). (Fig. 2-16)

Expressions >>> : to define the operator used to calculate an expression whose terms can be selected with the other buttons. (Fig. 2-15). The possible operators are:

+ sum of the two terms
- Subtraction of the second term from the first term
***** multiplication of the two terms
/ Ratio between the first and the second term
^ first term raised to the power indicated of the second term
(open bracket to define a sub-expression in a complex expression.
) closed bracket to define a sub expression in a complex expression.
== operator used to compare two expressions. Outputs the true value if the value of the two expressions are equal, otherwise it outputs a false value.
!= operator used to compare two expressions. Outputs the true value if the value of the two expressions is different, otherwise, it outputs the false value.
> operator used to compare two expressions. Outputs the true value if the value of the first expression is higher than the value of the second expression, otherwise, it outputs the false value.
>= operator used to compare two expressions. Outputs the true value if the value of the first expression is higher or equal to the value of the second expression, otherwise it outputs a false value.
< operator used to compare two expressions. Outputs the true value if the value of the first expression is lower than the value of the second expression, otherwise it outputs a false value.
<= operator used to compare two expressions. Output the true value if the value of the first expression is lower or equal to the value of the second expression, otherwise it outputs a false value.
! operator which acts on the result of a comparison. Denies the value of the term.

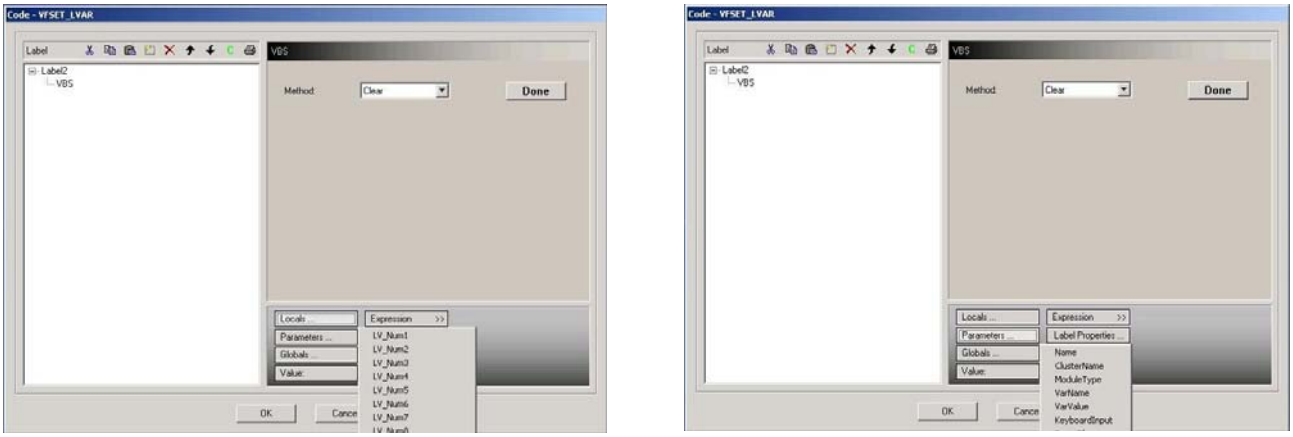


Fig. 2-14 - VIVA: Programming Environment – Selection of Locals and Parameters

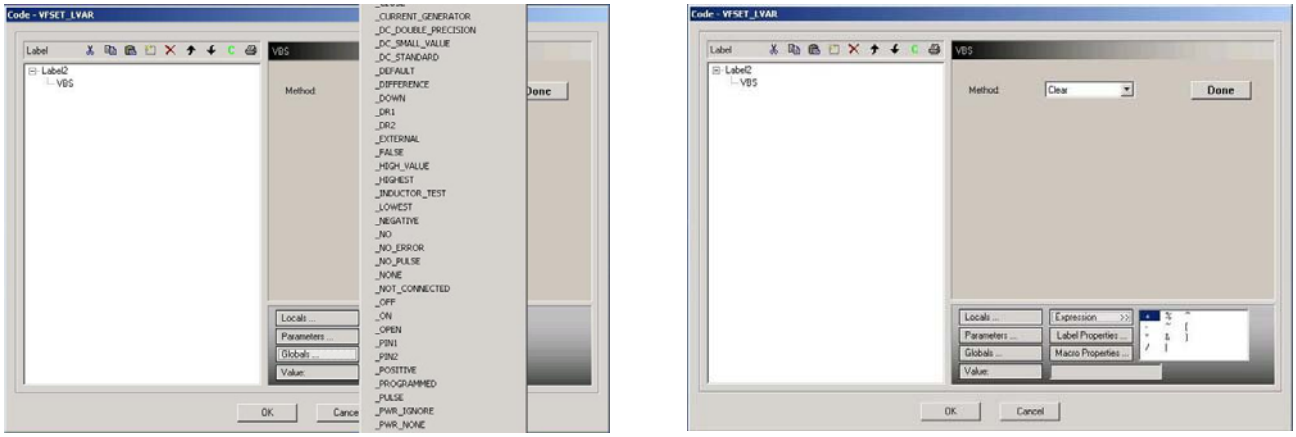


Fig. 2-15 - VIVA: Programming Environment – Selection of Globals and Expressions

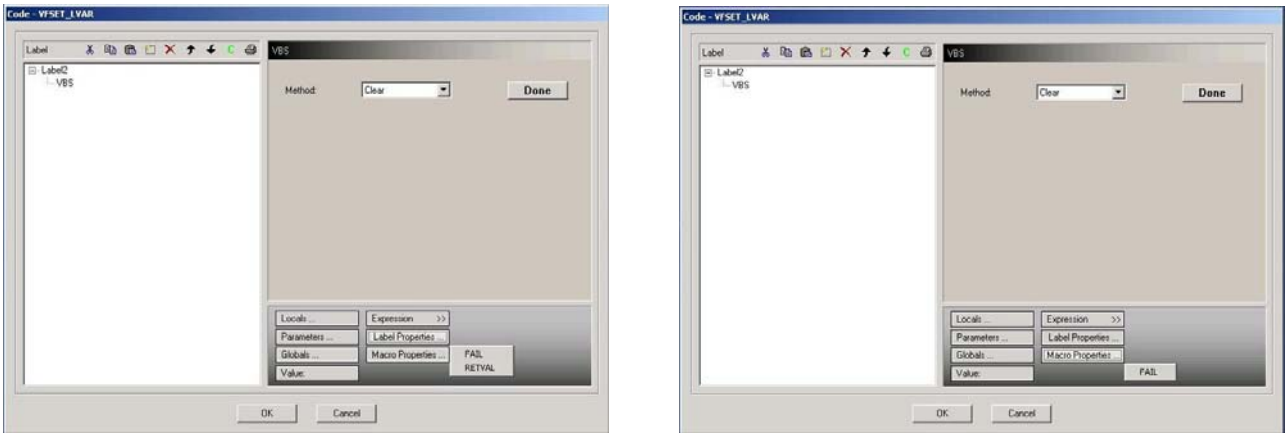


Fig. 2-16 - VIVA: Programming Environment – Selection of Label and Macro properties

3. System tools

This section describes the main tools available in the Programming Environment of the VIVA Software. The tools are managed through the instructions of the labels, written in VIVA Native Language (VNL)

The tools are **software elements** dedicated to manage the resources in the VIVA Environment. They are identified by a name and can be accessed through an interface made up by methods, identifying the operations required to the software element, while it's not possible to access the operations executed by the software element. The instructions for the tool management are identified by the couple <**Tool.method**>

The method schedules a variable number of parameters according to the method itself. A generic instruction to manage a tool will have the following format:

<**Name_Tool.Name_Method(par*1*, par*2*, ... Par*n*)**>

The next paragraphs will describe the main tools used, their methods and the arguments of the methods.

TEST

The tool TEST is used to manage: the macro parameters, the output of messages for the operator and the analog digital indicator displayed in the page "Program", during the program execution.

The methods available with this tool are:

TEST.Clear	TEST.Measure	TEST.SetParP
TEST.GetParN	TEST.Set	TEST.WriteMessage
TEST.GetParS	TEST.SetParN	TEST.WriteReport
TEST.GetParP	TEST.SetParS	

The Fig. 3-1, shows the dialog of the Debug Environment, containing the areas managed by the tool TEST, i.e. the analog-digital display, the macro parameters and the area: "Info" and "Report".

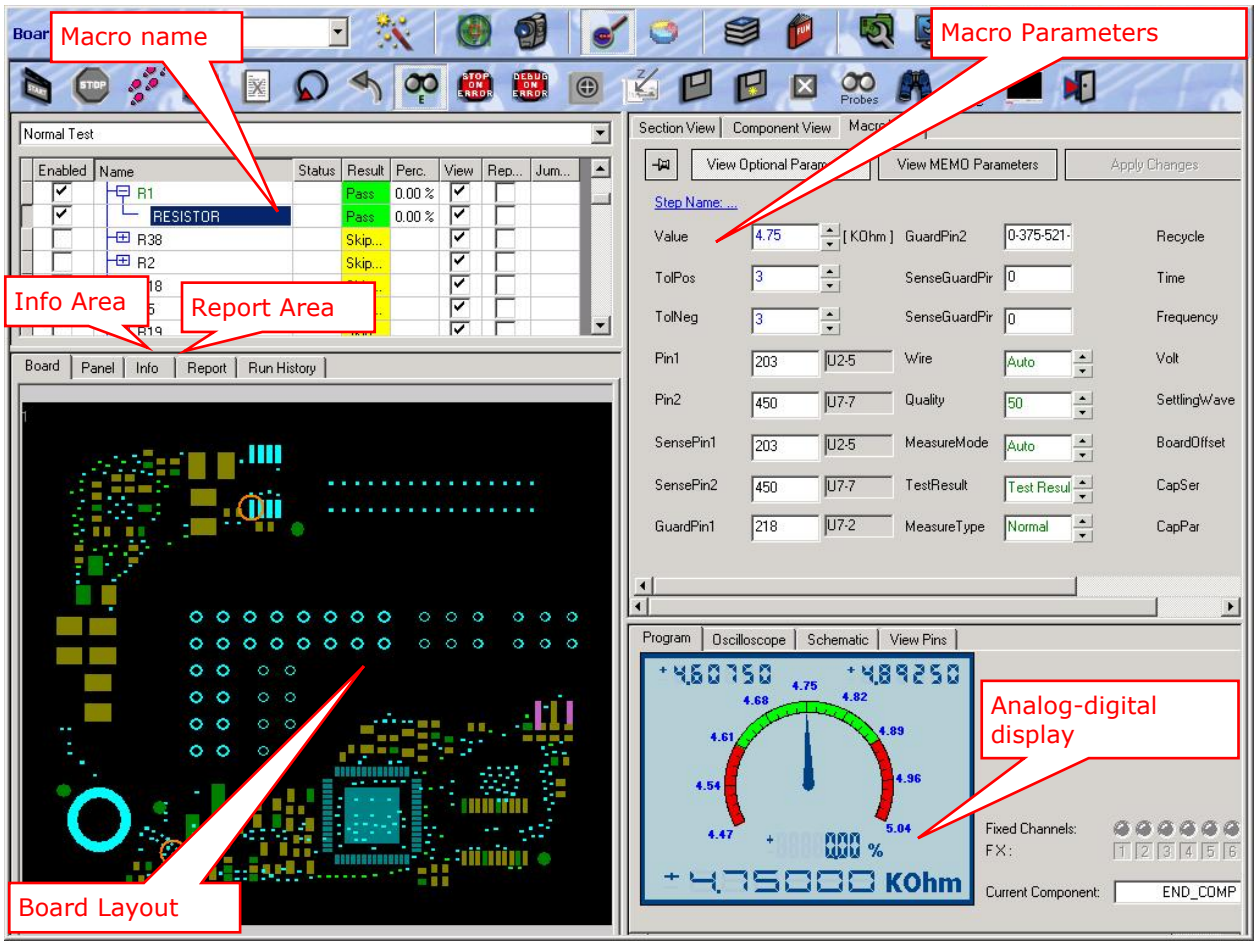
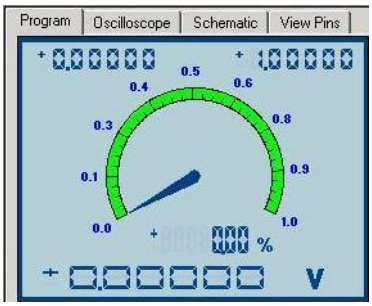


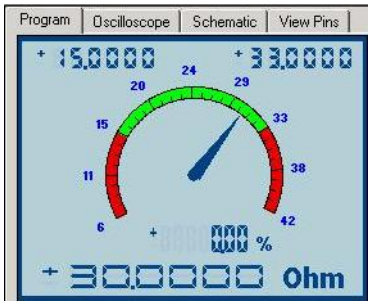
Fig. 3-1 - VIVA: Programming Environment – dialog: "Debug Environment"

The Fig. 3-2, show the condition of the analog-digital display with the execution of methods Clear, Set and Measure of the tool TEST.

TEST.Clear()



TEST.Set(O, 15, 30, 33)



TEST.Measure(25, SET ERROR, NONE)

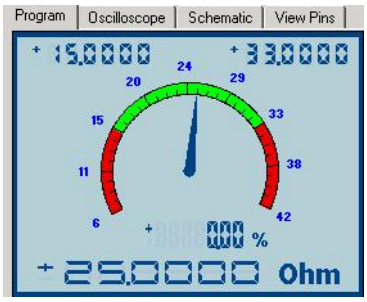


Fig. 3-2 - VIVA: Programming Environment – Digital – Analog Display

TEST.Clear

Resets the analog-digital display shown in the page "Program", during the execution of the test program. The values displayed can be set using the methods TEST.Set and TEST.Measure, have the following values.

unit := V
low := 0.00000
high := 1.00000
meas := 0.00000
value := 0.00000

Syntax

```
TEST.Clear()
```

Arguments

The method does not require arguments.

Examples

TEST.Clear()

TEST.Set

Defines the mode to display the expected value.

Syntax

```
TEST.Set(unit, low, meas, high)
```

Arguments

unit
Indicates the value of the measurement unit of the scale displayed with the analog-digital display in the page "Program", after the execution of the measurement. (V, A, O, F, H, Hz, s, etc.).

low
Indicates the minimum value accepted for the measurement executed. The value is reproduced in digital form above the analog-digital display (top-left side). In the analog part of the display, the area which indicates the values lower than the value defined with the argument "*low*" is colored in red.

meas
indicates the expected value for the measurement executed.

high
Indicates the maximum accepted value for the executed measurement. The value is found in the digital form above the analog-digital display (top-right side). In the analog part of the analog-digital display, the area which indicates the values higher than the values defined with the argument: "*high*" is colored in red.

Notes

The value associated to the argument "*meas*" is displayed, in the analog part of the instrument, with the position of the index on the instrument range. The digital display, instead, can be found on the last line of the analog-digital display, indicating the measurement unit defined with the argument "*unit*".

In the analog part of the display, the range which indicates the values included in the interval defined with the values associated to arguments "*low*" and "*high*" is colored in gree. If the value assigned to "*value*", of the method TEST.Measure, is a value included in the green color on the range, the executed measurement is classified as "Pass". Otherwise, the executed measurement is classified as "Fail".

Examples

TEST.set(O, 15[V], 30[V], 33[V])

TEST.Measure

Defines the mode to display the measured value.

Syntax

```
TEST.Masure(value, error,view)
```

Arguments

value

Indicates the measured value displayed in the analog-digital display (page "Program".

error

Indicates the management mode of the measured value assigned to the argument "*value*". The following values can be associated to the argument "*error*":

DEFAULT	Same meaning as value: "SET ERROR".
SET ERROR	If the value assigned to argument "*value*" is included in the validity range defined with arguments "*low*" and "*high*" of the method TEST.Set, the executed measurement is classified as "Pass". If the value is not included in the validity range, the executed measurement is classified as "Fail".
NO ERROR	the executed measurement is classified as "Pass", regardless the value assigned to the argument "*value*" .
FORCE ERROR	the executed measurement is classified as "Fail", regardless the value assigned to the argument "*value*".

view

The argument indicates the view mode of the measured value assigned to the argument: "*value*". The following values can be associated to the argument: "*view*": NONE, VIEW, REPEAT, VIEW. The values associated to the parameter "*view*" are not relevant in the VIVA environment: they are present for a matter of compatibility with the former test environment "S20 Windows". The view mode in the VIVA test environment is managed through the controls available in the Debug Environment.

Notes

The value associated to the argument "*value*" is displayed in the analog part of the tool, with the index on the tool scale. The digital display, instead, can be found in the last line of the analog-digital display. The view of the value assigned to the argument "value", which corresponds to the measured value, will replace the view of the value assigned to the argument "*meas*" of the method TEST.Set, which corresponds to the expected value.

If the value assigned to parameter "*value*" is include in the interval colored in green on the scale range, the executed measurement is classified as "Pass". If the value assigned to parameter "*value*" is not included in the range colored in green on the scale, the executed measurement is classified as "Fail". the values "NO ERROR" or "FORCE ERROR", assigned to the argument "*error*", precede the behavior described in this note, which is valid when the values "DEFAULT" or "SET ERROR" are assigned to the argument "error".

The line above the digital display of the measured value contains the percentage variation (%) of the measured value with regard to the expected value.

Before the execution of the method TEST.Measure, it is necessary to execute the method: TEST.Set which allows to define the validity range on the analog scale with the green color.

Examples

TEST.Measure(25[V], SET ERROR, NONE)

TEST.GetParN

Returns the value of a parameter of the type "Number".

Syntax

```
Var=TEST.GetParN(Name, Default Value)
```

Arguments

Var
Indicates the variable or the parameter of destination where is copied the current value of the parameter with the argument "*Name*".

Name
Indicates the name of the parameter whose value is copied in the destination, indicated with "Var".

Default Value
Indicates the value returned if the method: TEST.GetParN can not obtain the current value of the parameter indicated by the argument "*Name*".

Notes

Both the copied parameter and the destination must be of the type "Number".

Examples

```
GV_Num1=TEST.GetParN(ParNumber, 11[Nr])
```

TEST.GetParS

Returns the value of a parameter of the type "String".

Syntax

```
Var=TEST.GetParS(Name, Default Value)
```

Arguments

Var
Indicates the variable or destination parameter where is copied the current value of the parameter indicated by parameter "*Name*".

Name
Indicates the name of the parameter whose value is copied in the destination indicated with "Var".

Default Value
Indicates the value returned if the method: TEST.GetParS does not obtain the current value of the parameter indicated by argument "*Name*".

Notes

Both the copied parameter and the destination must be of the type "String".

Examples

```
GV_Str2=TEST.GetParS(ParString, aaaaaaa)
```

TEST.GetParP

Returns the value of a parameter of the type "Pinlist".

Syntax

```
Var=TEST.GetParP(Name, Default Value)
```

Arguments

Var
Indicates the variable or destination parameter where is copied the current value of the parameter indicated by argument "*Name*".

Name
Indicates the name of the parameter whose value is copied in the destination indicated with "Var".

Default Value
Indicates the value returned if the method: TEST.GetParP does not obtain the current value of the parameter indicated by argument "*Name*".

Notes

Both the copied parameter and the destination must be of the type "PinList".

Examples

```
GV_PinL1=TEST.GetParP(ParString, 0-0-0)
```

TEST.SetParN

Assigns the value of a parameter of the type "Number".

Syntax

```
TEST.SetParN(Name, Source)
```

Arguments

Name
Indicates the destination parameter where is copied the value indicated by argument "*Source*".

Source
Indicates the source of the value assigned to the parameter identified by "*Name*".

Notes

Both the assigned value and the destination parameter must be of the type "Number".

Source may be a constant, a parameter or a variable of the type Number.

Examples

```
TEST.SetParN(ParNumber, 11111111)
```

TEST.SetParS

Assigns the value of a parameter of the type "String".

Syntax

```
TEST.SetParS(Name, Source)
```

Arguments

Name
Indicates the destination parameter where is copied the value indicated with argument "*Source*".

Source
Indicates the source of the value assigned to the parameter identified by "*Name*".

Notes

Both the assigned value and the destination parameter must be of the type "String".

Source can be a constant, a parameter or a variable of the type String.

Examples

TEST.SetParS(ParString, New value from TEST.SetParS!)

TEST.SetParP

Assigns the value of a parameter of the type "Pinlist".

Syntax

```
TEST.SetParP(Name, Source)
```

Arguments

Name
Indicates the destination parameter where is copied the value indicated by the argument "*Source*".

Source
Indicates the source of the value assigned to the parameter identified by "*Name*".

Notes

Both the assigned value and the destination parameter must be of the type "PinList".

Source can be a constant, a parameter or a variable of the type PinList.

Examples

TEST.SetParP(ParString, 10-20-35)

TEST.WriteMessage

Displays a message in the INFO Area, which contains runtime information for the operator.

Syntax

```
TEST.WriteMessage(Message)
```

Arguments

Message
Indicates the message displayed in the Info Area.

Notes

The info area contains runtime messages displayed during the test program execution.

To view the info area, select tab "Info" in the Debug Environment.

Message can be a string, a parameter or a variable of any kind (String, Number, PinList).

Examples

TEST.WriteMessage(run time info message)

TEST.WriteReport

Displays a message in the Report area which contains end execution information for the operator.

Syntax

```
TEST.WriteReport(Message)
```

Arguments

Message
Indicates the message displayed in the Report Area

Notes

To view the report area, select tab "Report" in the Debug Environment.

Message can be a string, a parameter or a variable of any kind (String, Number, PinList).

Examples

TEST.WriteReport(End program report message)

VBS

The tool VBS makes available the environment "Visual Basic Script", where it is possible to execute instructions written in VBS.

The methods available with this tool are:

VBS.Clear VBS.Execute

VBS.Clear

Initializes the VBS environment.

Syntax

```
VBS.Clear()
```

Arguments

The method does not require arguments.

Notes

The initialization of the environment brings the VBS to the initial conditions. Hence, possible directives such as: #include"C:\My.vbs" previously typed, are deleted.

Examples

VBS.Clear()

VBS.Execute

Executes instructions written in VBS

Syntax

VBS.Execute(VBS instruction(par*1*, par*2*, …par*n*), VBS instruction(par*1*, par*2*, …par*n*), …)

Arguments

VBS Instructions
Instructions written in VBS language and added in the "run" input area of the dialog opened when the method "Execute" of the VBS tool is selected.

par1,*par2*, *etc*
Possible parameters scheduled for VBS instructions

Notes

The instructions added in the input area "run:" are executed during the program execution.

The instructions added to the input area "code:" are not executed in the phase run program, but are buffered before executing the instructions added in the area "run". For example, an instruction of the type #include"C:\My.vbs" will imply the buffering of the file "C:\My.vbs". The input area "run:" may contain calls to procedures or functions present in the file "C:\My.vbs"

Examples

Area "run":

```
Call GetPar(%ParName)
Call SetPar(%ParName, NewParvalue)
```

Area "code":

```
#include"C:\My.vbs"
```

The file "C:\My.vbs" must contain the two procedures: GetPar and SetPar

EXPRESSION

The tool EXPRESSION allows to evaluate expressions on the data available for the macro.

The methods available with the tool are:

EXPRESSION.Assign

Expression.Assign

Assigns to the variable placed to the left of the symbol "=" the result of the evaluation of "*Expression*".

Syntax

```
Var = Expression
```

Arguments

Var Indicates the variable to whom is associated the result of the evaluation of Expression

Expression
Any expression obtained with constants and data available for the macro, combined through the operators selected with button **Expressions >>>**.

Notes

"Expression" may indicate a simple variable or constant, a simple expression made up by a single operator or a complex expression made up by several operators with the possible use of brackets to define the priority of calculation.

The method "EXPRESSION.Assign(Expression)" in the instruction written in VNL has the syntax: Var=(Expression).

Examples

```
LV_NUm1=ParNumberSource
LV_NUm1=ParNumberSource+12
GV_NUm1=ParNumberDestination+10*(5+5)
```

IF

The tool IF allows to execute under condition a group of instructions, according to the value of an expression.

The methods available with this tool are:

IF.Evaluate

IF.Evaluate

Evaluates an expression which can return the Boolean values "True" or "False". If the result of the evaluation is "True" the instructions contained in the lower level are executed. Lower level indicates the instructions indented below the instruction IF in the list, in VNL format, of the instructions which make up the label.

Syntax

```
IF(condition)
```

Arguments

condition

Expression made up by constants and/or data available for the macro, combined through the operators selectable with the button **Expressions >>>**.

Notes

The instructions executed if the result of the evaluation is "True" correspond to the instructions which in other languages are called "THEN" type instructions. Here they are indented with regard to the previous line. The instructions not belonging to the THEN type, are written at the same level of the instruction IF.

If the result of the evaluation is "False" the instructions written at the lower level can be executed only if they are preceded by the instruction "ELSE" located at the same level of the instruction "IF".

If the instruction "ELSE" is not present, the instructions of the THEN type must be at the same level of the instruction IF, these are always executed regardless the evaluation of "*condition*".

The expression indicated with the argument "*condition*" may contain a single operator, hence two terms only.

The terms may be of the type "Number" or "String".

Once selected the method "Evaluate" and the argument "*condition*", it is possible to write the expression using the buttons available to edit the VNL instructions.

The method "IF.Evaluate(condition)" in the VNL instruction, is expressed with the syntax: IF(condition).

Examples

```
…
IF(ParNumber >=123)
+----VBS.Execute(MsgBox"Parameter greater than or equal to 123",0,"IF-THEN path"
+----   …
VBS.Execute(MsgBox"Statement after the IF instruction" ,0, "After the IF instruction"
…
```

ELSE

The tool ELSE allows to end the group of instructions executed after the method IF (or ELSEIF) where the evaluation of "*condition*" returned the result "False".

The methods available with the tool are:

ELSE.Dummy

ELSE.Dummy

Indicates the beginning of the instructions executed if the result of "*condition*" of the instruction IF returned the result "False". The instructions executed are at the lower level then the instruction ELSE. Lower level indicates the instructions indented under the instruction IF.

Syntax

```
ELSE
```

Arguments

The instruction has no arguments and is used to identify the beginning of the instructions executed in case of a result "False" in the evaluation of the argument "*expression*" of the instruction IF which is at the same level than the instruction ELSE.

Notes

The instructions executed if the result of the evaluation is "False" correspond to the instructions which, in other languages are called "ELSE type" instructions. The instructions not belonging to the else type are written at the same level of the instruction ELSE.

The tool ELSE combined with the tool IF, allows to obtain an execution under condition according to the two values, "True" and "False", that the evaluation of "*condition*" of the tool IF may have. The syntax of the instruction written in VNL is:

```
IF (condition)
   +--Statements for the "THEN path                statements performed when condition == "True"
   +--…
ELSE
   +--Statements for the "ELSE path                statements performed when condition == "False"
   +--…
Statements not depending on condition value        statements  always performed
…
```

Examples

```
…
- IF(ParNumber >=123)
   +-VBS.Execute(MsgBox"Parameter greater than or equal to 123",0,"IF-THEN path"
   +-   …
- ELSE
   +-VBS.Execute(MsgBox"Parameter less than 123",0,"IF-ELSE path"
   +-   …
VBS.Execute(MsgBox"Statement after the if instruction" ,0, "After the IF instruction"
…
```

ELSE_IF

The tool ELSE_IF allows to execute under condition a group of instructions, according to the value of an expression.

The methods available with the tools are:

ELSE_IF.Evaluate

ELSE_IF.Evaluate

Evaluates an expression which can return the Boolean values "True" or "False". If the result of the evaluation is "True" the instructions contained in the lower level to the instructions ELSE_IF are executed. Lower level indicates the instructions indented with regard to the instruction ELSE_IF in the list of the instructions which form the label.

Syntax

```
ELSE_IF(condition)
```

Arguments

condition

Expression made up by constants and/or data available for the macro, combined through the operators selectable with the button **Expressions >>>**.

Notes

The tool ELSE_IF can be used after the tool IF for further evaluations subsequent to a value "False" of the "*condition*" of the previous IF tool.

A repeated use of the tool ELSE_IF allows to implement the instance "Case", not available as simple instruction in VNL. The syntax of the VNL instruction is, in this case:

```
IF (condition)
   +--Statements performed when condition=="True"          Case 0
ELSE_IF (condition-1)
   +-- Statements performed when condition-1=="True"       Case 1
ELSE_IF (condition-n)
   +-- Statements performed when condition-n=="True"       Case n
ELSE
   +-- Statements performed when all =="False"             Case otherwise
Statements not depending on condition value          statements  always performed
```

Examples

```
- IF(ParNumberSource ==456)
   +-VBS.Execute(MsgBox"Case0: ParNumberSource == 456",0,"Case 0: first condition"
   +-   …
- ELSE_IF(ParNumberSource ==457
   +-VBS.Execute(MsgBox"Case1: ParNumberSource == 457",0,"Case 1: second condition"
   +-   …
- ELSE_IF(ParNumberSource ==nnn
   +-VBS.Execute(MsgBox"Casen: ParNumberSource == nnn",0,"Case n: nnn.th condition"
   +-   …
- ELSE
   +-VBS.Execute(MsgBox"Case otherwise: all condition are "False"",0,"Case otherwise: all condition false"
   +-   …
VBS.Execute(MsgBox"1)Statement after the conditional statements" ,0, "always performed"
…
```

Resistor

The tool Resistor allows to check a resistive value and that this value is included in a specified validity range.

The methods available with the tool are:

Resistor.Clear Resistor.Measure

Resistor.Clear

The method Resistor.Clear initializes the value of the arguments used by the method Resistor.Measure.

Syntax

```
Resistor.Clear()
```

Arguments

The method does not require arguments.

Notes

The method Resistor.Measure, always executes, as first activity, the instructions scheduled for the method Resistor.Clear. So, it is usually unnecessary to execute the method Resistor.Clear

Example

```
Resistor.Clear()
```

Resistor.Measure

The method Resistor.Measure checks that a resistive value on the UUT is included within a specified validity range.

Syntax

```
Resistor.Measure (Value, TolPos, TolNeg, Pin1, Pin2, SensePin1, SensePin2, GuardPin1, GuardPin2,
                  SenseGuardPin1, SenseGuardPin2, Wire, Quality, MeasureMode, TestResult,
                  MeasureType, Recycle, Time, Frequency, Volt, SettlingWave, BoardOffset, CapSer,
                  CapPar, IndSer, IndPar, ResGuardPin1, CapGuardPin1, ResGuardPin2, CapGuardPin2)
```

Arguments

The arguments are described in [1] in the description of macro RESISTOR and Resistor tool.

Notes

The different arguments allows defining the value to be measured, the measurement modes, and the output mode for the results and the possible parasite parameters which can affect the measurement.

Examples

```
Resistor.Measure (VALUE, TOLP, 0[Nr], Pin1, Pin2, SensePin1, SensePin2, 0[<unknown>], 0[<unknown>],
                  0[<unknown>], 0[<unknown>], Wire, Quality, DC_Active, TestResult, Normal, 1[Nr], 0.1[s],
                  15[KHz], 5[V], 85[Nr], 0[O], 0[F], 0[F], 0[H], 0[H], 0[O], 0[F], 0[O], 0[F])
```

Capacitor

The tool Capacitor allows to check that a capacitive value is included within a specified validity range

The methods available with the tool are:

Capacitor.Clear Capacitor.Measure

Capacitor.Clear

The method Capacitor.Clear initializes the value of the arguments used by the method Capacitor.Measure.

Syntax

```
Capacitor.Clear()
```

Arguments

The method does not require arguments.

Notes

The method Capacitor.Measure, always executes, as first activity, the instructions scheduled for the method Capacitor.Clear. So, it is normally unnecessary to execute the method Capacitor.Clear

Examples

```
Capacitor.Clear()
```

Capacitor.Measure

The method Capacitor.Measure, checks that a capacitive value on the UUT is included within a specified validity range.

Syntax

```
Capacitor.Measure(Value, TolPos, TolNeg, Pin1, Pin2, SensePin1, SensePin2, GuardPin1, GuardPin2,
                  SenseGuardPin1, SenseGuardPin2, Wire, Quality, MeasureMode, TestResult,
                  MeasureType, Recycle, Time, Frequency, Volt, SettlingWave, BoardOffset, ResSer,
                  ResPar, ResGuardPin1, CapGuardPin1, ResGuardPin2, CapGuardPin2)
```

Arguments

The arguments are described in [1] in the description of system macro CAPACITOR and tool Capacitor.

Notes

The different arguments allow to define the value to be measured, the executon mode of the measurement, and the mode to output the results and the possible parasite parameters which can affect the measurement.

Examples

```
Capacitor.Measure (VALUE, TOLP, 0[Nr], Pin1, Pin2, SensePin1, SensePin2, 0[<unknown>], 0[<unknown>],
                   0[<unknown>], 0[<unknown>], Wire, Quality, DC_Active, TestResult, Normal, 100[Nr], 0.1[s],
                   1500[Hz], 5[V], 85[Nr], 0[F], 0[O], 0[O], 0[O], 0[F], 0[O], 0[F])
```

Inductor

The tool Inductor allows to check has a value included in a specified validity range

The methods available with the tool are:

Inductor.Clear Inductor.Measure

Inductor.Clear

The method Inductor.Clear initializes the value of the arguments used by the method Inductor.Measure.

Syntax

```
Inductor.Clear()
```

Arguments

The method does not require arguments.

Notes

The method Inductor.Measure, always executes, as first activity, the instructions scheduled for the method Inductor.Clear. So, it is normally unnecessary to execute the method Inductor.Clear.

Examples

```
Inductor.Clear()
```

Inductor.Measure

The method Inductor.Measure, checks that an inductive value on the UUT is included within a specified validity range.

Syntax

```
Inductor.Measure (Value, TolPos, TolNeg, Pin1, Pin2, SensePin1, SensePin2, GuardPin1, GuardPin2,
                  SenseGuardPin1, SenseGuardPin2, Wire, Quality, TestResult, MeasureType, Recycle,
                  Frequency, Volt, SettlingWave, BoardOffset, ResSer, ResPar, ResGuardPin1,
                  CapGuardPin1, ResGuardPin2, CapGuardPin2)
```

Arguments

The arguments are described in [1] in the system macro INDUCTOR and the tool Inductor.

Notes

The different arguments allow to define the value to be measured, the executon mode of the measurement, and the mode to output the result and the possible parasite parameters which can affect the measurement.

Examples

```
Inductor.Measure (VALUE, TOLP, 0[Nr], Pin1, Pin2, SensePin1, SensePin2, 0[<unknown>], 0[<unknown>],
                  0[<unknown>], 0[<unknown>], Wire, Quality, TestResult, Average, 100[Nr], 1500[Hz], 5[V],
                  85[Nr], 0[H], 0[O], 0[O], 0[O], 0[F], 0[O], 0[F])
```

Diode

The tool Diode allows to check that the voltage drop at the pins of a diode with direct polarity is included in a specified validity range.

The methods available with the tool are:

Diode.Clear Diode.Measure

Diode.Clear

The method Diode.Clear initializes the value of the arguments used by the method Diode.Measure.

Syntax

```
Diode.Clear()
```

Arguments

The method does not require arguments.

Notes

The method Diode.Measure, always executes, as first activity, the instructions scheduled for the method Diode.Clear. So, it is normally unnecessary to execute the method Diode.Clear.

Examples

Diode.Clear()

Diode.Measure

The method Diode.Measure checks that the voltage drop at the pins of a diode with direct polarity is included in a specified validity range.

Syntax

```
Diode.Measure (Value, TolPos, TolNeg, Anode, Catode, SenseAnode, SenseCatode, GuardAnode, GuardCatode, SenseGuardAnode, SenseGuardCatode, Wire, Quality, JunctionMode, TestResult, MeasureType, Recycle, Time, Current, ResPar, CapPar, ResGuardPin1, CapGuardPin1, ResGuardPin2, CapGuardPin2)
```

Arguments

The arguments are described in [1] with the system macro DIODE and the tool Diode.

Notes

The different arguments allow to define the value to be measured, the measurement executon mode, and the modes to output the results and the possible parasite parameters which can affect the measurement.

Examples

```
Diode.Measure (VALUE, TOLP, 0[Nr], Pin1, Pin2, SensePin1, SensePin2, 0[<unknown>], 0[<unknown>], 0[<unknown>], 0[<unknown>], Wire,, Quality, Direct, TestResult, Average, 100[Nr], 0.5[s], 1200[Hz], 0.2[A], 0[O], 0[F], 0[O], 0[F], 0[O], 0[F])
```

Transistor

The tool Transistor allows to check the transistor type components present on the UUT.

The methods available with the tool are:

Transistor.Clear Transistor.Measure

Transistor.Clear

The method Transistor.Clear initializes the value of the arguments used by the method Transistor.Measure.

Syntax

Transistor.Clear()

Arguments

The method does not require arguments.

Notes

The method Transistor.Measure, always executes, as first activity, the instructions scheduled for the method Transistor.Clear. So, it is normally unnecessary to execute the method Transistor.Clear.

Examples

Transistor.Clear()

Transistor.Measure

The method Transistor.Measure, checks the transistor type components present on the UUT.

Syntax

Transistor.Measure	(**Type**, Technology, **TolPos**, **TolNeg**, Emitter_Source, Base_Gate, Collector_Drain, GuardBase, GuardCollector, Wire, Quality, **MeasMode**, TestMode, TestResult, MeasureType, Recycle, Time, **Volt**, **Current**, **ResPar**, **CapPa**r, **ResGuaBase**, **CapGuaBase**, **ResGuaCollector**, **CapGuaCollector**)

Arguments

The arguments of the method Transistor.Measure are described in [1] in the system macro TRANSISTOR.

Notes

The syntax of the method Transistor.Measure and of the system macro TRANSISTOR are different, even though the semantic is the same. In fact, the syntax of the macro TRANSISTOR is:

TRANSISTOR (Technology, ValueOFF, ValueON, Emitter_Source, Base_Gate, Collector_Drain, GuardBase, GuardCollector, Wire, Quality, TestMode, TestResult, MeasureType, Recycle, Time, Volt_C_D, Current_C_D, ResPar_C_D, CapPar_C_D, Volt_B_G, Current_B_G, ResPar_B_G, CapPar_B_G)

The Tab. 3-1, lists the mapping of the argument names of the method Transistor.Measure and the names of the parameters of the TRANSISTOR system macro.

Two arguments, Type and MeasMode, are present in Transistor.Measure, but not in the list of the parameters available with the macro TRANSISTOR. These two arguments are no longer used with the method Transistor.Measure.

Examples

Transistor.Measure (**0[Nr]**, BJT NPN, ValueOFF, ValueON, Pin1, Pin2, Pin3, SensePin1, SensePin2, Wire, Quality, **0[Nr]**, TestMode, TestResult, Normal, 100[Nr], 0.3[V], 5[V], 0.2[A], 0[O], 0[F], 0[O], 0[F], 0[O], 0[F])

Tab. 3-1 - VIVA: Name mapping from Transistor.Measure and macro TRANSISTOR

Names on Transistor.Measure	Names on Macro TRANSISTOR	Value on Transistor. Measure
Type		0[Nr]
Technology	Technology	
TolPos	ValueOFF	
TolNeg	ValueON	
Emitter_Source	Emitter_Source	
Base_Gate	Base_Gate	
Collector_Drain	Collector_Drain	
GuardBase	GuardBase	
GuardCollector	GuardCollector	
Wire	Wire	
Quality	Quality	
MeasMode		0[Nr]
TestMode	TestMode	
TestResult	TestResult	
MeasType	MeasureType	
Recycle	Recycle	
Time	Time	
Volt	Volt_C_D	
Current	Current_C_D	
ResPar	Volt_B_G	
CapPar	Current_B_G	
ResGuaBase	ResPar_C_D	
CapGuaBase	CapPar_C_D	
ResGuaCollector	ResPar_B_G	
CapGuaCollector	CapPar_B_G	

Fnode

The tool Fnode allows to check the behavior of the nets associated to a signal and connected to one or more points of the UUT, considered as an electric bipole.

The methods available with the tool are:

Fnode.Clear Fnode.Measure

Fnode.Clear

The method Fnode.Clear initializes the value of the arguments used by the method Fnode.Measure.

Syntax

```
Fnode.Clear()
```

Arguments

The method does not require arguments.

Notes

The method Fnode.Measure, always executes, as first activity, the instructions scheduled for the method Fnode.Clear. So, it is normally unnecessary to execute the method Fnode.Clear.

Examples

Fnode.Clear()

Fnode.Measure

The method Fnode.Measure checks that the current floating in the node identifying the electric bipole as a value, in width and phase, which is included in a defined validity range.

Syntax

Fnode.Measure (TolPosI, TolNegI, TolPosPha, TolNegPha, **Pin1**, **Pin2**, **GuardPin1**, Wire, Quality, **ViewMode**,TestResult, MeasureType, Recycle, Volt, PointWF, **NumberWF**, BurstNr, StartFreq, StopFreq, SettlingWave, Scale, **RangeI**, ScopeMode, SelectView,RelMacro)

Arguments

The arguments are described in [1] with the system macro FNODE and the tool Fnode.

Notes

The method Fnode.Measure is used in the label “Fan” of the FNODE macro. The macro FNODE is used by the function FNODE. The function FNODE does not contain directives in the section “Patterns” since it is not associated to single components or devices, but is associated to all the signals present in the UUT in the phase “Create Program” if in the dialog “Advanced Create program Settings” the field “OTPN” contains the value “Fnode”.

With regard to the note above, it is recommended not to use the method Fnode.Measure from user macro to avoid unpredictable results.

The syntax of Fnode.Measure method and FNODE macro are different even though the semantic is the same. In fact, the syntax of the FNODE macro is:

FNODE (TolPosI, TolNegI, TolPosPha, TolNegPha, SignalPin, RefPin, SignalGuard, Wire, Quality, TestResult, MeasureType, Recycle, Volt, PointWF, BurstNr, StartFreq, StopFreq, SettlingWave, Scale, ViewMode, ScopeMode, ViewWFNumber, SelectView)

The Tab. 3-2, contains the mapping of the argument names of the method Fnode.Measure and the names of the parameters of the system macro FNODE.

Examples

Fnode.Measure (10[Nr], 10[Nr], 10[Nr], 10[Nr], Pin1, Pin2, Pin3, Wire, Quality, Current, TestResult, Normal, 100[Nr], 3.5[V], -1[Nr], -1[Nr], 10[Nr], 1[KHz], 100[KHz], 85[Nr], 0.5[Nr], 0.2[A], Time, 10[Nr], 0[Nr])

Tab. 3-2 - VIVA: Name mapping from Fnode.Measure and macro FNODE

Names on Fnode.Measure	Names on Macro FNODE	Notes
TolPosI	TolPosI	
TolNegI	TolNegI	
TolPosPha	TolPosPha	
TolNegPha	TolNegPha	
Pin1	**SignalPin**	
Pin2	**RefPin**	
GuardPin1	**SignalGuard**	
Wire	Wire	
Quality	Quality	
ViewMode	ViewMode	
TestResult	TestResult	
MeasType	MeasType	
Recycle	Recycle	
Volt	Volt	
PointWF	PointWF	
NumberWF	**ViewFNumber**	
BurstNr	BurstNr	
StartFreq	StartFreq	
StopFreq	StopFreq	
SettlingWave	SettlingWave	
Scale	Scale	
RangeI	RangeI	
ScopeMode	ScopeMode	
SelectView	SelectView	
RelMacro		

Autic, Jscan, OpenFix, Continuity, Isolation, Short, Contact, Discharge

For information about tools: Autic, Jscan, OpenFix, Continuity, Isolation, Short, Contact, Discharge, please refer to [1] which describes the system macros with the same names of the corresponding tools.

The methods available for each instrument are:

ToolName.Clear ToolName.Measure

Arguments

For the description of the arguments related to the different tools, please refer to [1].

Notes

For each tool, the corresponding system macro uses only the method: ToolName.Measure.

Unless really necessary, it is recommended to use the system macros or copies of the system macros instead of writing the ToolName:Measure(arg1,arg2,…argn) in the Programming Environment.

When building user macros that use the system tools, it is recommended to pay attention to the class associated to each parameter of the macro and the proper association: <macro parameters – arguments of the method used for the tool>.

The recommendation supplied for the tools: Autic, Jscan, OpenFix, Continuity, Isolation, Short, Contact, Discharge, is applied to all the tools for which a corresponding system macro is available.

Examples

For the examples, please refer to the instruction which executes the method: ToolName.Measure present in the label of the corresponding system macro.

SYSTEM

The tool SYSTEM is used for different VIVA Software managements, here included the porting from programs developed with the former test environment S20Win.

The methods available with the tool are:

SYSTEM.AutoAdjustment
SYSTEM.Autodebug
SYSTEM.Autolearn
SYSTEM.Auxiliarybox
SYSTEM.Barcode
SYSTEM.BoardDescription
SYSTEM.BoardStrayCapacitance
SYSTEM.BoardStrayImpedance
SYSTEM.BoardStrayIductance
SYSTEM.Capacitive
SYSTEM.Capacitor
SYSTEM.Capread
SYSTEM.Cluster
SYSTEM.Continuity
SYSTEM.CurrentBoard
SYSTEM.DefaultWorkPath
SYSTEM.Dmaster
SYSTEM.f8LinesChannels
SYSTEM.Femto
SYSTEM.FixtureInvert
SYSTEM.FixtureType
SYSTEM.HighVoltage
SYSTEM.HVIvalue
SYSTEM.Inductor
SYSTEM.Isolation
SYSTEM.LamPresent
SYSTEM.LamScale
SYSTEM.MoveProbes
SYSTEM.OpenFix
SYSTEM.ParassiteLearn
SYSTEM.PowerSupply
SYSTEM.ProgrammingStation
SYSTEM.Resistor
SYSTEM.RestoreParameters
SYSTEM.SaveParameters
SYSTEM.ScaFP
SYSTEM.Scampor
SYSTEM.ScannerBoards
SYSTEM.SideToTest
SYSTEM.Soldering
SYSTEM.Soldering1
SYSTEM.Soldering2
SYSTEM.Soldering3
SYSTEM.StrayCapacitance
SYSTEM.StrayImpedance
SYSTEM.StrayIductance
SYSTEM.SystemName
SYSTEM.SystemType
SYSTEM.UserBoard
SYSTEM.UserBoard2
SYSTEM.UserBoard3
SYSTEM.UserBoard4
SYSTEM.UserGen
SYSTEM.UserGen2
SYSTEM.UserGen3
SYSTEM.UserGen4
SYSTEM.Vision

The methods available with the tool SYSTEM are used by the system macros of the VIVA software supplied.

Below, the description of the method SYSTEM.Cluster used by the system macros CLUSTER and CLUSTERDIA.

SYSTEM.Cluster

The method SYSTEM.Cluster is used by the system macros CLUSTER and CLUSTERDIA to execute blocks of instructions in un cluster. The cluster, in the VIVA environment, is the code included between the instructions **START** "ClusterName" " Cluster Parameters….; and **ENDTEST**;.
The instructions are written in VIVA Language (VL) in ".pat" files. For further information, please refer to [1] and [3] and to the online documentation of the VIVA Software.

The clusters can be used in functional or ICT programs.

In the functional programs the cluster are used to check the correct operation of one or more components of a board, by injecting input signals and checking that the output signals correspond to the expected value. The signal configurations are called pattern and the files containing the pattern are called pattern files.

In the ICT programs, the clusters are used by the system macros to check the UUT components. As explained in the notes, the system macros schedule a parameter which contains the name of the cluster to be executed and another parameter which is updated in the phase "Create Program" with the name of the component on the UUT. This way, a single cluster is used to check several components of the same type on the UUT.

Syntax

```
SYSTEM.Cluster(ClusterName, FileName, Flags)
```

Arguments

ClusterName
The argument indicates the name of the cluster to be executed.

Filename
The argument indicates the name of the ".pat" file which contains the instructions of the cluster.

Flags
The argument allows to compile a numeric value which can be used in the code of the cluster to control the execution flow of the instructions which make up the cluster.

Notes

The method SYSTEM.Cluster is executed through the system macro CLUSTER (or CLUSTERDIA) which schedules the parameters: "Name", "ModuleType", "Value", "PinList".

In the label which makes up the macro, the argument "*ClusterName*" identifies the cluster executed and associated to parameter "Name" of the macro.

In the "Create Program" for ICT programs, the name of a component on the UUT is associated to parameter "Name" of the macro CLUSTER.

When the label contained in the macro is executed, it is executed the "*ClusterName*" which, in the phase "Create Program" got the name of the component on the UUT.

From the notes above, it is evident that are necessary as many clusters as are the components of the same type on the UUT, or it is necessary to manally modify the parqameter "Name" which was compiled automatically in the phase "Create Program", typing the name identifying the only cluster needed to check all the instances of the component.

Using a new parameter (*Clustername*) in the macro, which contains the name of the cluster to be executed, there is no need of manual operation after the program has been created. The cluster identified by this parameter is associated to the argument "*ClusterName*" which will execute the same cluster for every component. Through the instructions of the cluster it is possible, where necessary, the parameter "Name" which identifies the specific component on the UUT.

Examples

System.Cluster(Name, ,0[Nr])	with macros CLUSTER and CLUSTERDIA
System.Cluster(ClusterName, ,0[Nr])	with the macros for ICT programs

4. Appendix

This appendix contains:

- The Table of “Permissions” associated to functions and macros
- The Table of classes and measurement units used in the definition of the macro parameters.

4.1 Permissions for Functions and Macros

The table below lists the "Permissions" scheduled for the functions and macros present in the VIVA environment.

Tab. 4-1 - VIVA: Permissions

Parameter type

Parameter type	Description
Not view	If flagged, the view of the values with the analog-digital display is disabled in the page "Program" during the execution of the code associated to the function or to the macro.
Not repeat	If flagged, it disables the field "Repeat" in the grid of the Debug Environment during the execution of the code associated to the function or macro.
Not paste parameter	If flagged, it disables the item "Paste parameters" in the contextual menu of the Debug Environment.
Not add comp	If flagged, it disables the item "Add component" in the contextual menu of the dialog Debug Environment.
Not remove	If flagged, it disables the item "Remove" in the contextual menu of the dialog Debug Environment.
Not move same father	If flagged, it disables the item "Move" in the contextual menu of the Debug Environment which would allow to move the item into the grid of the dialog.
Not move change father	If flagged, it disables the item "Move" in the contextual menu of the Debug Environment which would allow to move the item into the grid of the dialog.
Not rename	If flagged, it disables the item "Rename" in the contextual menu of the dialog "Debug Environment".
Not generate test	If flagged, it disables the item "Generate test" in the contextual menu of the Debug Environment.
Not jump	If flagged, it disables the fill in of the field "Jump/Call " in the grid of the dialog Debug Environment during the execution of the code associated to the function or macro.
Not change macro	If flagged, it disables the fill in of the field "Macro " in the grid of the dialog "Debug Environment", when a component is selected.
Not optimize	If flagged, it disables the item "Optimize" in the contextual menu of the dialog Debug Environment.
Not automatic retest	If flagged, it disables the possibility of the automatic retest when an error is detected.
Not user modify	If flagged, any change by the user is not allowed
Powered macro	If flagged, it indicates the macro need power to be executed.
Power source macro	If flagged, it indicates that the macro supplies power for the macros requiring it.
Ignore Power	The macro does not check the presence or lack of power to be executed
ICT Optimization supported	The macro can be optimized to reduce the test time
Auto Debug needed	The macro needs autodebug
Autolearn needed	The macro needs autolearn
Macro with contact supported	macro with check of contact in case of test fail
Macro with contact check	macro with contact check executed always before measurement
Execute always	Not implemented yet
Parallel test supported	Not implemented yet

Generation type

Generation type	Description
ALL THE SYSTEMS	If flagged, the code created can be executed on all the product lines.
VALID	If flagged, the code created can be executed only “VALID” Line Systems.
BARE BOARD	If flagged, the code created can be executed only on Bare Board Line Systems.
FIREFLY	If flagged, the code created can be executed only on Firefly Line Systems.
STRATEGY- BON	If flagged, the code created can be executed only on Bed of Nails systems (Strategy Line).
PILOT/AERIAL-FP	If flagged, the code created can be executed only on the Flying Probes systems (Pilot Line).

Lam type

Lam type	Description
LAM	If flagged, the code created can be executed on systems which schedule the test instruments on the LAM module.
ACL	If flagged, the code created can be executed on systems which schedule the test instruments on the ACL module.

4.2 Attributes of the macro parameter

The following tables list the classes and measurement units related to the parameters used in the call of a macro. The class of a parameter is the attribute which the VIVA software uses to locate the data in the CAD files which identify the UUT, and will be used in the phase "Create Program" to assign the current value to the parameter of the macro.

Tab. 4-2 - VIVA: Parameters Class

Parameter Class

Class Name	Description
<unknown>	Identifies an unknown class. Usually not used.
GenericString	The class identifies the parameter as a generic string. The data are processed by the code as (Kind): "String". In the phase "Create Program" the parameter belonging to this class is not modified.
GenericNumber	The class identifies the parameter as a generic number. The data are processed by the code (Kind): "Number". In the phase "Create Program" the parameter belonging to this class is not modified.
GenericPinlist	The class identifies the parameter as a generic pinlist (channel numbers), which will be processed by the code as (Kind): "Pinlist". In the phase "Create Program" the parameter belonging to this class is not modified.
Value	The class identifies the parameter as the expected value to be checked by the macro. The data are processed by the code as type of data (Kind): "Number".
TolP	The class identifies the parameter as the positive tolerance used in the measurement of the expected value. The data are processed by the code as (Kind): "Number".
TolN	The class identifies the parameter as the negative tolerance used in the measurement of the expected value. The data are processed by the code as (Kind): "Number".
Pin1	The class identifies the parameter as number of the channel connected to Pin1 of the component. The data are processed by the code as (Kind): "Pinlist".
Pin2	The class identifies the parameter as number of the channel connected to Pin2 of the component. The data are processed by the code as (Kind): "Pinlist".
Name	The class identifies the parameter a san alpha-numeric string indicating the component to be checked. The data are processed by the code as (Kind): "String".
IsolationVoltage	The class identifies the parameter as the reference voltage value used for the insulation tests. The data are processed by the code as (Kind): "Number".
IsolationImpedance	The class identifies the parameter as the reference resistive value used for the insulation tests. The data are processed by the code as (Kind): "Number".
Pin1GUA	The class identifies the parameter as a generic pinlist (channel numbers) which, through the Guarding Generator and the Active Guarding technique are forced to the same potential. The data are processed by the code as (Kind): "Pinlist".
Pin2GUA	The class identifies the parameter as a generic pinlist (channel numbers) which, through the Guarding Generator and the Passive Guarding technique are forced to the same potential. The data are processed by the code as (Kind): "Pinlist".
ContinuityCurrent	The class identifies the parameter as the value of the reference current used for the continuity tests. The data are processed by the code as (Kind): "Number".
RPARAL	The class identifies the parameter as the resistive value parallel to the component to be checked. The data are processed by the code as: (Kind): "Number".
CPARAL	The class identifies the parameter as the capacitive value parallel to the component to be checked. The data are processed by the code as: (Kind): "Number".
LPARAL	The class identifies the parameter as the inductive value parallel to the component to be checked. The data are processed by the code as: (Kind): "Number".
Parasite	The class identifies parameters related to the parasite values of the component to be checked. The data are processed by the code as (Kind): "Number".
PinGnd	The class identifies the parameter as the indicator of the number of channel connected to the component pin connected to GND. The data are processed by the code as: (Kind): "Pinlist".
ContinuityImpedance	The class identifies the parameter as the reference resistive value used for the continuity tests. The data are processed by the code as: (Kind): "Number".
MeasureType	The class identifies the type of measurement executed for the component. With regard to the ACL module, it may vary according to the measurement unit and the macro used.
SensePin1	The class identifies the parameter as the pin (channel number) connected to the positive pole of the meter used for the measurement. Used for 4-wire measurements. The data are processed by the code as: (Kind): "Pinlist".

Class Name	Description
SensePin2	The class identifies the parameter as the pin (channel number) connected to the negative pole of the meter used for the measurement. Used for 4-wire measurements. The data are processed by the code as: (Kind): “Pinlist”.
ResGuardPin1	The class identifies the parameter as resistive value of the impedance of the components connected to the component under measurement. If not properly insulated these could absorb part of the current supplied by the generator. The data are processed by the code as: (Kind): “Number”. The data are related to the pin1 of the component.
CapGuardPin1	The class identifies the parameter as capacitive value of the impedance of the components connected to the component under measurement. If not properly insulated these could absorb part of the current supplied by the generator. The data are processed by the code as: (Kind): “Number”. The data are related to the pin1 of the component.
ResGuardPin2	The class identifies the parameter as resistive value of the impedance of the components connected to the component under measurement. If not properly insulated, these could absorb part of the current circulating in the component to be checked and not detected by the meter. The data are processed by the code as (Kind): “Number”. The data are related to pin2 of the component.
CapGuardPin2	The class identifies the parameter as the capacitive value of the impedance of the components connected to the component under measurement. If not properly insulated these may absorb part of the current circulating in the component to be checked and not detected by the meter. The data are processed by the code as: (Kind): “Number”. The data are related to pin2 of the component.
SignalPin	The class identifies the assigned signal as forced to the signal. The data are processed by the code as: (Kind): “Pinlist”. The forced channel is associated to the node connected to the fixture on bed of nails systems or where is placed the probe (Flying Probe Systems) For the macro FNODE, it identifies the channel connected to the positive pole of the electric bipole.
Signal1Guard	The class identifies the group of channels associated to guard 1. The data are processed by the code as: (Kind): “Pinlist”.
Signal2Guard	The class identifies the group of channels associated to guard 2. The data are processed by the code as: (Kind): “Pinlist”.
SensePinGnd	The class identifies the group of channels associated to GND or to the reference point for the FNODE macro. The data are processed by the code as (Kind): “Pinlist”.
Pin3	The class identifies the parameter as indicator of the channel number connected to pin3 of the component. The data are processed by the code as (Kind): “Pinlist”.
Pin4	The class identifies the parameter as indicator of the channel number connected to pin4 of the component. The data are processed by the code as (Kind): “Pinlist”.
Pinlist	The class identifies a generic list of channels. The data are processed by the code as (Kind): “Pinlist”.
SensorNum	The class identifies the fixed sensor number used for the tests executed with macro OPENFIX on bed of nails systems. The data are processed by the code as (Kind): “Number”.
SensorPin	The class identifies the channel related to the position of the capacitive probe in the tests executed with macro OPENFIX on flying probe systems. The data are processed by the code as (Kind): “Pinlist”.
Signal1SenseGuard	The class identifies the channel used as reference for guard1. Used for 6-wire measurement, the data are processed by the code as (Kind): “Pinlist”.
Signal2SenseGuard	The class identifies the channel used as reference for guard2. Used for 6 wire measurements the data are processed by the code as (Kind): “Pinlist”.
IsolatedPin	The class is used only for the JSCAN macro, parameter “RefPin”. The data are processed by the code as (Kind): “Pinlist”.
ModuleName	The class identifies the parameter as indicator of the name of the cluster used by the macro. The data are processed by the code as (Kind): “String”.
ModuleType	The class identifies the parameter as indicator of the kind of macro in use The data are processed by the code as (Kind): “Number”.
PowerMode	The class identifies the parameter as the indicator of the status set for the power supplies selected with the macro “POWER”. The data are processed by the code as (Kind): “Number”.
PowerName	The class identifies the parameter as indicator of the number of current section POWERON. The data are processed by the code as (Kind): “String”.
StartChn	The class identifies the 1st channel of the group of channels where short or contact must be checked. The data are processed by the code as (Kind): “Number”.
StopChn	The class identifies the last channel of the group of channels where short or contact must be checked. The data are processed by the code as (Kind): “Number”.
FamilyNrCont	The class identifies the parameter as indicator of a group of channels checked with the system macro CONTACT. The data are processed by the code as (Kind): “Number”.

Class Name	Description
FamilyNrSho	The class identifies the parameter as indicator of a group of channels checked with the system macro SHORT. The data are processed by the code as (Kind): “Number”.

The following table refers the measurement units of the scales used with the VIVA Software. In some cases the possible values of the measurement unit are represented in different modes, according whether the user is the processing code of the VIVA software or a user accessing through VIVA GUI to select the measurement unit and define the default value for the parameter.
When necessary, for the discrete measurement units, the list of the possible values is indicated by the couple <processing value – user value>. Example: 1 – DC_PASSIVE.

Tab. 4-3 - VIVA: Parameters Unit

Parameter Unit

Unit Name	Description
<unknown>	The measurement unit indicates a scale not compatible with the scales indicated below. Usually not used.
F	The measurement unit of the scale is: Farad.
H	The measurement unit of the scale is: Henry.
O	The measurement unit of the scale is: Ohm.
V	The measurement unit of the scale is: Volt.
s	The measurement unit of the scale is: Second.
A	The measurement unit of the scale is: Ampere.
Hz	The measurement unit of the scale is: Hertz.
m	The measurement unit of the scale is: Meter.
Nr	The measurement unit of the scale is: a generic number.
<MeasureMode>	The measurement unit is used on the systems which schedule the instruments present on the LAM module. The parameter indicates the measurement mode. (parameter MODE for macros: _CAPACITOR_STD, _RESISTOR_STD, _CAPACITOR, _RESISTOR). The possible values are: 0 – AUTO 3 – DC DOUBLE PRECISION 1 – DC STANDARD 4 – AC 2 – DC SMALL VALUE 5 – HIGH VALUE
<ErrorMode>	The measurement unit is used on the systems which schedule the test instruments present on the LAM module. The parameter indicates the type of result obtained when executing the macro. (parameter NER for macros: _CAPACITOR, _CAPACITOR_STD, CONTINUITY, CONTINUITY_STD, ISOLATION, ISOLATION_STD, _JUMPER, _RESISTOR , _RESISTOR_STD). The possible values are: 0 – NO ERROR 3 – CLEAR ALL 1 – SET ERROR 2 – FORCE ERROR
<RecycleMode>	The measurement unit is used on the systems which schedule the test instrument present on the LAM module. The parameter indicates the recycle mode on the measurement. (parameter MRC for macros: _JUMPER, _CAPACITOR_STD, _RESISTOR_STD, _CAPACITOR, _RESISTOR). The possible values are: – AVERAGE – HIGHEST – PROGRAMMED – LOWEST – POSITIVE – DIFFERENCE
<UsePin>	The measurement unit is used on the systems which schedule the test instrument present on the LAM module. The possible values are: 0 – AUTO 2 – PIN2 1 – PIN1
<On / Off>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The possible values are: 0 – ON 1 – OFF
<Yes / No>	The measurement unit is used on the systems which schedule the test instrument present on the ACL and on the LAM modules. The possible values are: 0 – NO 1 – YES

Unit Name	Description
<True / False>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The possible values are: 0 – FALSE 1 – TRUE
<InShortWith>	The measurement unit is used on the systems which schedule the test instrument present on the LAM module. The parameter indicates the possible values of the argument "inShortWith" of the method VOLT.Set. The possible values are: 0 – NONE 2 – DR2 1 – DR1
<PositiveNegative>	The measurement unit is used on the systems which schedule the test instrument present on the LAM module. The parameter indicates the possible values of the argument "*Mode*" of the method "*CKGEN.Set*". The possible values are: 1 – NEGATIVE 2 – POSITIVE
<DR1Mode>	The measurement unit is used on the systems which schedule the test instrument present on the LAM module. The parameter indicates the possible values of the argument "*Mode*" of the method "*DR1.Set*". The possible values are: 0 – CURRENT GENERATOR 2 – INDUCTOR TEST 1 – VOLTAGE GENERATOR
<PulseNoPulse>	The measurement unit is used on the systems which schedule the test instrument present on the LAM module. The parameter indicates the possible values of the argument "*Pulse*" of the methods "*DR1.Set*" and "*DR2.Set*". The possible values are: 0 – NO PULSE 1 – PULSE
<PULMode>	The measurement unit is used on the systems which schedule the test instrument present on the LAM module The parameter indicates the possible values of the argument "*Mode*" of the method "*PULL.Set*". The possible values are: 0 – NOT CONNECTED 2 – DOWN 1 – UP 3 – EXTERNAL
<GUAFollow>	The measurement unit is used on the systems which schedule the test instrument present on the LAM module The parameter indicates the possible values of the argument "*Mode*" of the "*GUA.Set*" method. The possible values are: 1 – DR1 3 – EXTERNAL
<ViewMode>	The measurement unit is used on the systems which schedule the test instrument present on the LAM module. The parameter indicates the possible values of the argument "*View*" of the method "*TEST.Measure*". The possible values are: 0– NONE 2 – REPEAT 1– VIEW 3 – VIEW and REPEAT
<OpenClose>	The measurement unit is used on the systems which schedule the test instrument present on both the LAM and ACL modules. The parameter indicates the status of connection between two points. The possible values are: 0 – CLOSE 1 – OPEN
<HighLow>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the status of the signal viewed on the system analog-digital display. The possible values are: 0 – LOW 1 – HIGH
<SyncAsync>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the possible values of the argument "*Mode*" of the methods "*FEMTO.Set*" and "*ACL_FEMTO.Set*". The possible values are: 0 – SYNCHRONOUS 1 – ASYNCHRONOUS
<FemtoBehavior>	The measurement unit is used on the systems which schedule the test instrument present on both the LAM and ACL modules. The parameter indicates the possible values of the argument "*Behavior*" of the methods "*FEMTO.Set*" and "*ACL_FEMTO.Set*". The possible values are: 0 – MEASURE 1 – LEARN
fF	The measurement unit of the scale is: femto Farad.
<CapReadBehavior>	The measurement unit is used on the systems which schedule the test instrument present on both the LAM and ACL modules. The parameter indicates the possible values of the argument "*MeasureType*" of the method "*SYSTEM.CapRead*". The possible values are: 0 – FEMTOONLY 3 – READPARASITE 1 – LAMONLY 4 – NOMEASURE 2 – AUTORANGE
W	The measurement unit of the scale is: Watt.

Unit Name	Description
<ACL Test Result>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the type of result obtained when the macro is executed. The possible values are: 0 – Force Pass 3 – Force Skip 1 – Test result 2 – Force Fail
<ACL Measure Type>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the type of processing executed on the acquired values. The possible values are: 0 – Normal 3 – Maximum 1 – Average 4 – Minimum 2 – RecOnError 6 – PeakToPeak
<ACL Measure Mode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the measure mode. The possible values are: 0 – DC_Active 2 – AC_Passive 1 – DC_Passive 3 – Auto
<ACL Wire>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the number of wires (or channels) used for the measurement. The possible values are: 2 – 2 Wire 4 – 4 Wire 3 – 3 Wire 6 – 6 Wire
<ACL Junction>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the polarization of the junction. The possible values are: 0 – Direct 1 – Invert
<ACL FAN ViewMode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the scale viewed on the analog-digital display. The possible values are: 0 – Current 4 – IndPar 1 – Phase 5 – IndPar 2 – CapPar 6 – CapSer 3 – Respar 7 – IndSer
<ACL FAN ScopeMode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the scale viewed on the VIVA “Oscilloscope”. The possible values are: 0 – Time 2 – I Module 1 – FFT 2 – I Phase
Dg	The measurement unit of the scale is: centesimal degrees/sexadecimal degrees.
<ACL_IND_MeasMode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the measure mode executed with the check of inductive values. The possible values are: 2 – AC_PASSIVE 3 – Auto
<ACL_CAP_MeasMode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the measure mode executed with the check of capacitive values. The possible values are: 0 – DC_Active 3 – Auto 2 – AC_Passive
<ACLWire_NO_DP>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the number of wires (or channels) used for the measurement. The possible values are: -1 – Auto 3 – 3 Wire 2 – 2 Wire
<ACLBJT_Technology>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the technology of the component to be checked. The possible values are: 0 – BJT NPN 3 – MOS P 1 – BJT PNP 4 – Auto 2 – MOS N
<ACLBJT_TestMode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the type of check executed on the component. The possible values are: 0 – On-Off 2 – Off 1 – On

Unit Name	Description
<ACL_Logic_Function>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the logic function checked. The possible values are: 0 – 2NAND 7 – 3NOR 14 – E_BUF 1 – 2NOR 8 – 3AND 15 – NE_NBUF 2 – 2AND 9 – 3OR 16 – EN_NBUF 3 – 2OR 10 – 3XOR 243 – BUF243 4– 2XOR 11 – 3XNOR 245 – BUF245 5 – 2XNOR 12 – NOT 6 – 3NAND 13 – NE_BUF
<ACL_Logic_Family>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the technological family of the component to be checked. The possible values are: 0 – Standard 32 – AHCT 69 – LVCH 1 – L 34 – FCT 71 – ALVC 3 – S 36 – AC 75 – LVT 5 – H 40 – ACT 77 – ALVCR 9 – LS 42 – ACQ 80 – ALVCH 11 – AS 46 – ACTQ 83 – LCX 13 – ALS 48 – ABT 86 – VCX 15 – F 50 – GTL 17 – C 52 – GTLP 22 – HC 54 – ALB 26 – HCT 56 – LV 28 – AHC 63 – LVCT
<ACL_Transf_MeasMode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the scale measured in the check of transformers. The possible values are: 0 – Current 1 – Voltage
<ACL_Triac_MeasMode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the check mode. The possible values are: 0 – On 2 – Inv_On 1 – Off 3 – Inv_Off
<ACL_Opto_MeasMode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the execution mode of the check. The possible values are: 0 – ON 2 – None 1 – OFF
<ModuleType>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the language used to implement the macro. The possible values are: 0 – NONEt 2 – VBS 1 – VL 3 – VL_DYNAMIC
<PowerModeType>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The possible values are: 0 – NONE 3 – POWERON 1 – DONTCARE 4 – POWEROFF 2 – POWERED
<ACLJunctionZener>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the measurement mode for a zener. The possible values are: 0 – Direct 1 – Zener
<ACLBBTMeasMode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The parameter indicates the capacitive measurements mode on BBT systems. The possible values are: 0 – Femto 2 – Autorange 1 – ACL
<ACLCalibrate>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. It is the measurement unit of the parameter of the parameter "*CalibrateMode*" of the macro "*TRIMMER*". The possible values are: 0 – FixedMeasure 2 – AutoExitCalibrate 1 – Calibrate

Unit Name	Description
<ACLOpenFixMode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. It is the measurement unit of the parameter "*MeasureMode*" of the macro "*OPENFIX*". The possible values are: 0 – RMS Low 2 – RMS High 1 – AC Low 3 – AC High
<ACLParametricFNode>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. It is the measurement unit of the parameter "*SelectView*" of the "*FNODE*" macro. The possible values are: 0 – Default 4 – Inductor 1 – ViewMode 5 – DirectJunction 2 – Resistor 6 – InvertJunction 3 – Capacitor
<MsgBoxIcon>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. It is the measurement unit of the parameter "*ShowedIcon*" of macros "*ASKUSER*" and "*MESSAGE*", written in VBS. The possible values are: 16 – Critical 48 – Exclamation 32 – Question 64 – Information
<MsgBoxButton>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. It is the measurement unit of the parameter "*ShowedButton*" of macros "*ASKUSER*" and "*MESSAGE*", written in VBS. The possible values are: 0 – OK 3 – YesNoCancel 1 – OkCancel 4 – YesNo 2 – AbortRetryIgnore 5 – RetryCancel
<MsgBoxDefButton>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. It is the measurement unit of the parameter "*DefaultButton*" of macros "*ASKUSER*" and "*MESSAGE*", written in VBS. The possible values are: 0 – Button1 512 – Butto3l 256 – Button2
<ACL_S20_MMD>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module to execute, in emulation mode, programs written in S20Windows environment. The parameter indicates the possible values of the argument "*MMD*" of the method "*GENERIC_S20.Measure*". The possible values are: 1 – V1 4 – N 2 – V4 5 – D 3 – R 6 – C
<ACL_S20_DRM>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module to execute, in emulation mode, programs written in S20Windows environment. The parameter indicates the possible values of the argument "*DRM*" of the method "*GENERIC_S20.Measure*". The possible values are: 1 – I 3 – L 2 – V
<ACL_S20_VIM>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module to execute, in emulation mode, programs written in S20Windows environment. The parameter indicates the possible values of the argument "*VIM*" of the method "*GENERIC_S20.Measure*". The possible values are: 1 – V 5 – L 2 – O 6 – H 3 – F 7 – T 4 – A
<ACL_S20_PUM>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module to execute, in emulation mode, programs written in S20Windows environment. The parameter indicates the possible values of the argument "*PUM*" of the method "*GENERIC_S20.Measure*". The possible values are: 1 – N 3 – D 2 – U 4 – E
<ACL_S20_POP>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module to execute, in emulation mode, programs written in S20Windows environment. The parameter indicates the possible values of the argument "*POP*" of the method "*GENERIC_S20.Measure*". The possible values are: 1 – N 5 – / 2 – + 6 – ! 3 – - 7 – ^ 4 – *

Unit Name	Description
<ACL_S20_SCA>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module to execute, in emulation mode, programs written in S20Windows environment. The parameter indicates the possible values of the argument "*SCA*" of the method "*GENERIC_S20.Measure*". The possible values are: 1 − H 2 – L
<ACL_Fnode_Scale>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The possible values are: 0 – Logarithmic 1 – Linear
<ACL_OpAmp_View>	The measurement unit is used on the systems which schedule the test instrument present on the ACL module. The possible values are: 0 – Output 2 – Input Neg 1 – Input Pos

4.3 Tools- Methods - Parameters

The following table lists all the tools available In the VIVA software environment. For each tool, is provided the list of the methods available and the parameters used for each method. Also the class associated to the parameter is indicated and, if relevant, the default value of the parameter.

Tab. 4-4 - VIVA: Tools- Methods- Parameters

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
ACL_CAP	Clear	NO			
ACL_CONTACT	Clear	NO			
ACL_COUNTER	Clear	NO			
ACL_DISCAP	Clear	NO			
ACL_DRA	Clear	NO			
ACL_DRB	Clear	NO			
ACL_DRC	Clear	NO			
ACL_FEMTO	Clear	NO			
ACL_FEMTO	Measure	YES	GND	NO	GenericPinlist
ACL_FEMTO	Measure	YES	testPoint	NO	GenericPinlist
ACL_FEMTO	Set	NO	behavior	Measure	GenericNumber
ACL_FEMTO	Set	NO	mode	Asynchronous	GenericNumber
ACL_FEMTO	Set	NO	scale	High	GenericNumber
ACL_FLASH	Clear	NO			
ACL_GND	Clear	NO			
ACL_GUAPOINT	Clear	NO			
ACL_HVI	Clear	NO			
ACL_HVI	Measure	YES	Channel		GenericPinlist
ACL_HVI	Measure	YES	Current		GenericNumber
ACL_HVI	Measure	YES	Mode		GenericString
ACL_HVI	Measure	YES	time	NO.NONO1	GenericNumber
ACL_HVI	Set	NO	Channel		GenericPinlist
ACL_HVI	Set	NO	Voltage		GenericNumber
ACL_IMM	Clear	NO			
ACL_IND	Clear	NO			
ACL_JUN	Clear	NO			
ACL_LINE	Clear	NO			
ACL_LINE_EXT	Clear	NO			
ACL_LOGIC	Clear	NO			
ACL_LOGIC_LEVEL	Clear	NO			
ACL_LOGIC_MODE	Clear	NO			
ACL_MONITOR	Clear	NO			

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
ACL_NODETEST	Clear	NO			
ACL_PULL	Clear	NO			
ACL_PW1	Clear	NO			
ACL_PW1	Set	NO	sense	On	GenericNumber
ACL_PW1	Set	NO	state	On	GenericNumber
ACL_PW1	Set	NO	valV		GenericNumber
ACL_PW2	Clear	NO			
ACL_PW2	Set	NO	sense	On	GenericNumber
ACL_PW2	Set	NO	state	On	GenericNumber
ACL_PW2	Set	NO	valV		GenericNumber
ACL_PW3	Clear	NO			
ACL_PW3	Set	NO	sense	On	GenericNumber
ACL_PW3	Set	NO	state	On	GenericNumber
ACL_PW3	Set	NO	valV		GenericNumber
ACL_PW4	Clear	NO			
ACL_PW4	Set	NO	sense	On	GenericNumber
ACL_PW4	Set	NO	state	On	GenericNumber
ACL_PW4	Set	NO	valV		GenericNumber
ACL_PW5	Clear	NO			
ACL_PW5	Set	NO	sense	On	GenericNumber
ACL_PW5	Set	NO	state	On	GenericNumber
ACL_PW5	Set	NO	valV		GenericNumber
ACL_PW6	Clear	NO			
ACL_PW6	Set	NO	sense	On	GenericNumber
ACL_PW6	Set	NO	state	On	GenericNumber
ACL_PW6	Set	NO	valV		GenericNumber
ACL_RAM_EXT	Clear	NO			
ACL_RD_HW	Clear	NO			
ACL_RES	Clear	NO			
ACL_SCAFP	Clear	NO			
ACL_SCAMPO	Clear	NO			
ACL_SHORT	Clear	NO			
ACL_SHUNT	Clear	NO			
ACL_WR_HW	Clear	NO			
ACL_BEEP	Clear	NO			
ACL_CALL	Clear	NO			
CALL	Execute	YES	label		GenericString
CALL	Execute	YES	macro		GenericString
Capacitor	Clear	NO			

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
Capacitor	Measure	YES	BoardOffset	NO	GenericNumber
Capacitor	Measure	YES	CapGuaP1	NO	GenericNumber
Capacitor	Measure	YES	CapGuaP2	NO	GenericNumber
Capacitor	Measure	YES	Frequency	-1	GenericNumber
Capacitor	Measure	YES	GuardPin1		GenericPinlist
Capacitor	Measure	YES	GuardPin2		GenericPinlist
Capacitor	Measure	YES	MeasMode	Default	GenericNumber
Capacitor	Measure	YES	MeasType	Default	GenericNumber
Capacitor	Measure	YES	Pin1		GenericPinlist
Capacitor	Measure	YES	Pin2		GenericPinlist
Capacitor	Measure	YES	Quality	1	GenericNumber
Capacitor	Measure	YES	Recycle	1	GenericNumber
Capacitor	Measure	YES	ResGuaP1	NO	GenericNumber
Capacitor	Measure	YES	ResGuaP2	NO	GenericNumber
Capacitor	Measure	YES	ResPar	NO	GenericNumber
Capacitor	Measure	YES	ResSer	NO	GenericNumber
Capacitor	Measure	YES	SenseGuardPin1		GenericPinlist
Capacitor	Measure	YES	SenseGuardPin2		GenericPinlist
Capacitor	Measure	YES	SenseP1		GenericPinlist
Capacitor	Measure	YES	SenseP2		GenericPinlist
Capacitor	Measure	YES	SettlingWave	-1	GenericNumber
Capacitor	Measure	YES	TestResult	Default	GenericNumber
Capacitor	Measure	YES	Time	-1	GenericNumber
Capacitor	Measure	YES	TolNeg		GenericNumber
Capacitor	Measure	YES	TolPos		GenericNumber
Capacitor	Measure	YES	Value		GenericNumber
Capacitor	Measure	YES	Volt	-1	GenericNumber
Capacitor	Measure	YES	Wire	Default	GenericNumber
CapRead	Clear	NO			
CapRead	Measure	YES	MeasMode	Default	GenericNumber
CapRead	Measure	YES	MeasType	Default	GenericNumber
CapRead	Measure	YES	Pin1		GenericPinlist
CapRead	Measure	YES	Pin2		GenericPinlist
CapRead	Measure	YES	Recycle	1	GenericNumber
CapRead	Measure	YES	TestResult	Default	GenericNumber
CapRead	Measure	YES	TolNeg		GenericNumber
CapRead	Measure	YES	TolPos		GenericNumber
CapRead	Measure	YES	Value		GenericNumber
CHANNEL	Clear	NO			

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
CKGEN	Clear	NO			
CKGEN	Set	NO	mode	Positive	GenericNumber
CKGEN	Set	NO	nImpuls	NO	GenericNumber
CKGEN	Set	NO	testPoint	NO	GenericPinlist
CKGEN	Set	NO	time	NO	GenericNumber
COMMENT	Set	NO	text		GenericString
CONSOLE	Beep	NO	Duration		GenericNumber
CONSOLE	Beep	NO	Frequency		GenericNumber
CONSOLE	ClearScreen	NO			
CONSOLE	GetProperty	YES	Name		GenericString
CONSOLE	HtmlEnableUI	NO	Enable	Yes	GenericNumber
CONSOLE	HtmlLoadPage	NO	URL		GenericString
CONSOLE	PutProperty	NO	Name		GenericString
CONSOLE	PutProperty	NO	Value		GenericString
CONSOLE	Reset	NO			
CONSOLE	SetColor	NO	Background		GenericNumber
CONSOLE	SetColor	NO	Foreground		GenericNumber
CONSOLE	SetCursor	NO	Column		GenericNumber
CONSOLE	SetCursor	NO	Row		GenericNumber
CONSOLE	Sleep	NO	Milliseconds		GenericNumber
CONSOLE	Write	NO	String		GenericString
Contact	Clear	NO			
Contact	Measure	YES	Family		GenericNumber
Contact	Measure	YES	StartCh		GenericNumber
Contact	Measure	YES	StopCh		GenericNumber
Contact	Measure	YES	TestResult	Default	GenericNumber
Contact	Measure	YES	Time	-1	GenericNumber
Contact	Measure	YES	Value		GenericNumber
Contact	Measure	YES	ViewCh	NO	GenericNumber
Contact	Measure	YES	Volt	-1	GenericNumber
Continuity	Clear	NO			
Continuity	Measure	YES	CapPar	NO	GenericNumber
Continuity	Measure	YES	MeasType	Default	GenericNumber
Continuity	Measure	YES	Pin1		GenericPinlist
Continuity	Measure	YES	Pin2		GenericPinlist
Continuity	Measure	YES	Recycle	1	GenericNumber
Continuity	Measure	YES	TestResult	Default	GenericNumber
Continuity	Measure	YES	Time	-1	GenericNumber
Continuity	Measure	YES	Value		GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
Continuity	Measure	YES	Volt	-1	GenericNumber
COUNTER	Clear	NO			
COUNTER	Measure	YES	mode	Average	GenericNumber
COUNTER	Measure	YES	recycle	1	GenericNumber
COUNTER	Measure	YES	testPoint	NO	GenericPinlist
COUNTER	Measure	YES	time	NO.NONO1	GenericNumber
DIGITAL	Clear	NO			
DIGITAL	Measure	YES	mode	AUTO	GenericString
DIGITAL	Measure	YES	time		GenericNumber
DIGITAL	Set	NO	highDrv		GenericNumber
DIGITAL	Set	NO	highThre		GenericNumber
DIGITAL	Set	NO	lowDrv		GenericNumber
DIGITAL	Set	NO	lowThre		GenericNumber
Diode	Clear	NO			
Diode	Measure	YES	Anode		GenericPinlist
Diode	Measure	YES	CapGuaP1	NO	GenericNumber
Diode	Measure	YES	CapGuaP2	NO	GenericNumber
Diode	Measure	YES	CapPar	NO	GenericNumber
Diode	Measure	YES	Catode		GenericPinlist
Diode	Measure	YES	Current	-1	GenericNumber
Diode	Measure	YES	GuardAnode		GenericPinlist
Diode	Measure	YES	GuardCatode		GenericPinlist
Diode	Measure	YES	MeasType	Default	GenericNumber
Diode	Measure	YES	Quality	-1	GenericNumber
Diode	Measure	YES	Recycle	1	GenericNumber
Diode	Measure	YES	ResGuaP1	NO	GenericNumber
Diode	Measure	YES	ResGuaP2	NO	GenericNumber
Diode	Measure	YES	ResPar	NO	GenericNumber
Diode	Measure	YES	SenseAnode		GenericPinlist
Diode	Measure	YES	SenseCatode		GenericPinlist
Diode	Measure	YES	SenseGuardAnode		GenericPinlist
Diode	Measure	YES	SenseGuardCatode		GenericPinlist
Diode	Measure	YES	TestMode	Direct	GenericNumber
Diode	Measure	YES	TestResult	Default	GenericNumber
Diode	Measure	YES	Time	-1	GenericNumber
Diode	Measure	YES	TolNeg		GenericNumber
Diode	Measure	YES	TolPos		GenericNumber
Diode	Measure	YES	Volt		GenericNumber
Diode	Measure	YES	Wire	Default	GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
Discap	Clear	NO			
Discap	Measure	NO	Current	-1	GenericNumber
Discap	Measure	NO	Pin1		GenericPinlist
Discap	Measure	NO	Pin2		GenericPinlist
Discap	Measure	NO	TestResult	Default	GenericNumber
Discap	Measure	NO	Time	NO	GenericNumber
Discap	Measure	NO	Value		GenericNumber
DR1	Clear	NO			
DR1	Set	NO	mode	Current Generator	GenericNumber
DR1	Set	NO	pulse	No Pulse	GenericNumber
DR1	Set	NO	testPoint	NO	GenericPinlist
DR1	Set	NO	valI	NO	GenericNumber
DR1	Set	NO	valV	NO	GenericNumber
DR2	Clear	NO			
DR2	Set	NO	pulse	No Pulse	GenericNumber
DR2	Set	NO	RG	No	GenericNumber
DR2	Set	NO	testPoint	NO	GenericPinlist
DR2	Set	NO	valV	NO	GenericNumber
EB1	Clear	NO			
EB1	Set	NO	value	NOxFF	GenericNumber
EB2	Clear	NO			
EB2	Set	NO	value	NOxFF	GenericNumber
ELSE	Dummy	NO			
ELSE_IF	Evaluate	NO	condition		<unknown>
EXPRESSION	Assign	YES	expression		<unknown>
FAN	Clear	NO			
FAN	Measure	NO	BurstNr	-1	GenericNumber
FAN	Measure	NO	GuardPin1		GenericPinlist
FAN	Measure	NO	MeasType	Default	GenericNumber
FAN	Measure	NO	NumberWF	-1	GenericNumber
FAN	Measure	NO	Pin1		GenericPinlist
FAN	Measure	NO	Pin2		GenericPinlist
FAN	Measure	NO	PointWF	-1	GenericNumber
FAN	Measure	NO	Quality	-1	GenericNumber
FAN	Measure	NO	RangeI	-1	GenericNumber
FAN	Measure	NO	Recycle	1	GenericNumber
FAN	Measure	NO	Scale	NO	GenericNumber
FAN	Measure	NO	ScopeMode	Time	GenericNumber
FAN	Measure	NO	SettlingWave	-1	GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
FAN	Measure	NO	StartFreq	-1	GenericNumber
FAN	Measure	NO	StopFreq	-1	GenericNumber
FAN	Measure	NO	TestResult	Default	GenericNumber
FAN	Measure	NO	TolNegI		GenericNumber
FAN	Measure	NO	TolNegPha		GenericNumber
FAN	Measure	NO	TolPosI		GenericNumber
FAN	Measure	NO	TolPosPha		GenericNumber
FAN	Measure	NO	ValuePha	NO	GenericNumber
FAN	Measure	NO	ViewMode	Current	GenericNumber
FAN	Measure	NO	Volt	-1	GenericNumber
FAN	Measure	NO	Wire	Default	GenericNumber
FEMTO	Clear	NO			
FEMTO	Measure	YES	GND	NO	GenericPinlist
FEMTO	Measure	YES	testPoint	NO	GenericPinlist
FEMTO	Set	NO	behavior	Measure	GenericNumber
FEMTO	Set	NO	mode	Asynchronous	GenericNumber
FEMTO	Set	NO	scale	High	GenericNumber
FILE	Clear	NO			
FILE	Read	YES			
FILE	Set	NO	mode		GenericString
FILE	Set	NO	path		GenericString
FILE	Write	NO	string		GenericString
FNode	Clear	NO			
FNode	Measure	YES	BurstNr	-1	GenericNumber
FNode	Measure	YES	GuardPin1		GenericPinlist
FNode	Measure	YES	MeasType	Default	GenericNumber
FNode	Measure	YES	NumberWF	-1	GenericNumber
FNode	Measure	YES	Pin1		GenericPinlist
FNode	Measure	YES	Pin2		GenericPinlist
FNode	Measure	YES	PointWF	-1	GenericNumber
FNode	Measure	YES	Quality	-1	GenericNumber
FNode	Measure	YES	RangeI	-1	GenericNumber
FNode	Measure	YES	Recycle	1	GenericNumber
FNode	Measure	YES	RelMacro	NO	GenericNumber
FNode	Measure	YES	Scale	NO	GenericNumber
FNode	Measure	YES	ScopeMode	Time	GenericNumber
FNode	Measure	YES	SelectView	NO	GenericNumber
FNode	Measure	YES	SettlingWave	-1	GenericNumber
FNode	Measure	YES	StartFreq	-1	GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
FNode	Measure	YES	StopFreq	-1	GenericNumber
FNode	Measure	YES	TestResult	Default	GenericNumber
FNode	Measure	YES	TolNegI		GenericNumber
FNode	Measure	YES	TolNegPha		GenericNumber
FNode	Measure	YES	TolPosI		GenericNumber
FNode	Measure	YES	TolPosPha		GenericNumber
FNode	Measure	YES	ViewMode	Current	GenericNumber
FNode	Measure	YES	Volt	-1	GenericNumber
FNode	Measure	YES	Wire	Default	GenericNumber
Generic_S2NO	Measure	NO	ALM	NO	GenericNumber
Generic_S2NO	Measure	NO	CLE		GenericPinlist
Generic_S2NO	Measure	NO	D2P	1	GenericNumber
Generic_S2NO	Measure	NO	DR2	NO.NO	GenericNumber
Generic_S2NO	Measure	NO	DRI	NO.NO	GenericNumber
Generic_S2NO	Measure	NO	DRM	1	GenericNumber
Generic_S2NO	Measure	NO	DRV	NO.NO	GenericNumber
Generic_S2NO	Measure	NO	ELM	255	GenericNumber
Generic_S2NO	Measure	NO	HIG	NO.NO	GenericNumber
Generic_S2NO	Measure	NO	HLD	NO	GenericNumber
Generic_S2NO	Measure	NO	LI1		GenericPinlist
Generic_S2NO	Measure	NO	LI2		GenericPinlist
Generic_S2NO	Measure	NO	LI3		GenericPinlist
Generic_S2NO	Measure	NO	LI4		GenericPinlist
Generic_S2NO	Measure	NO	LN1	NO	GenericNumber
Generic_S2NO	Measure	NO	LN2	NO	GenericNumber
Generic_S2NO	Measure	NO	LN3	NO	GenericNumber
Generic_S2NO	Measure	NO	LN4	NO	GenericNumber
Generic_S2NO	Measure	NO	LOW	NO.NO	GenericNumber
Generic_S2NO	Measure	NO	MES	5NO.NO	GenericNumber
Generic_S2NO	Measure	NO	MMD	2	GenericNumber
Generic_S2NO	Measure	NO	MRC	1	GenericNumber
Generic_S2NO	Measure	NO	NCK	NO	GenericNumber
Generic_S2NO	Measure	NO	NER	NO	GenericNumber
Generic_S2NO	Measure	NO	NRC	NO	GenericNumber
Generic_S2NO	Measure	NO	OPC	255	GenericNumber
Generic_S2NO	Measure	NO	PUL	1	GenericNumber
Generic_S2NO	Measure	NO	PUM	1	GenericNumber
Generic_S2NO	Measure	NO	TIM	NO.NONO1	GenericNumber
Generic_S2NO	Measure	NO	VIM	1	GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
GND	Clear	NO			
GND	Set	NO	openChannels	NO	GenericPinlist
GND	Set	NO	testPoint	NO	GenericPinlist
GUA	Clear	NO			
GUA	Set	NO	mode	DR1	GenericNumber
GUA	Set	NO	testPoint	NO	GenericPinlist
HV	Clear	NO			
HV	Measure	YES	mode	M	GenericString
HV	Measure	YES	recycle	1	GenericNumber
HV	Measure	YES	time		GenericNumber
HV	Set	NO	time		GenericNumber
HV	Set	NO	valO		GenericNumber
HV	Set	NO	valV		GenericNumber
HVI	Clear	NO			
HVI	Measure	YES	Channel		GenericPinlist
HVI	Measure	YES	Current		GenericNumber
HVI	Measure	YES	Mode		GenericString
HVI	Measure	YES	time	NO.NONO1	GenericNumber
HVI	Set	NO	Channel		GenericPinlist
HVI	Set	NO	Voltage		GenericNumber
IEEE	Clear	NO			
IEEE	Read	YES			
IEEE	Set	NO	listen		GenericNumber
IEEE	Set	NO	mode		GenericString
IEEE	Set	NO	talk		GenericNumber
IEEE	Write	NO	string		GenericString
IF	Evaluate	NO	condition		<unknown>
Inductor	Clear	YES			
Inductor	Measure	NO	BoardOffset	NO	GenericNumber
Inductor	Measure	NO	CapGuaP1	NO	GenericNumber
Inductor	Measure	NO	CapGuaP2	NO	GenericNumber
Inductor	Measure	NO	Frequency	-1	GenericNumber
Inductor	Measure	NO	GuardPin1		GenericPinlist
Inductor	Measure	NO	GuardPin2		GenericPinlist
Inductor	Measure	NO	MeasType	Default	GenericNumber
Inductor	Measure	NO	Pin1		GenericPinlist
Inductor	Measure	NO	Pin2		GenericPinlist
Inductor	Measure	NO	Quality	-1	GenericNumber
Inductor	Measure	NO	Recycle	1	GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
Inductor	Measure	NO	ResGuaP1	NO	GenericNumber
Inductor	Measure	NO	ResGuaP2	NO	GenericNumber
Inductor	Measure	NO	ResPar	NO	GenericNumber
Inductor	Measure	NO	ResSer	NO	GenericNumber
Inductor	Measure	NO	SenseGuardPin1		GenericPinlist
Inductor	Measure	NO	SenseGuardPin2		GenericPinlist
Inductor	Measure	NO	SenseP1		GenericPinlist
Inductor	Measure	NO	SenseP2		GenericPinlist
Inductor	Measure	NO	SettlingWave	-1	GenericNumber
Inductor	Measure	NO	TestResult	Default	GenericNumber
Inductor	Measure	NO	TolNeg		GenericNumber
Inductor	Measure	NO	TolPos		GenericNumber
Inductor	Measure	NO	Value		GenericNumber
Inductor	Measure	NO	Volt	-1	GenericNumber
Inductor	Measure	NO	Wire	Default	GenericNumber
IOV5LOAD	Clear	NO			
IOV5LOAD_OUT_A	Clear	NO			
IOV5LOAD_OUT_ANA	Clear	NO			
IOV5LOAD_OUT_B	Clear	NO			
IOV5LOAD_OUT_LOAD	Clear	NO			
IOV5LOADOUT_OPTO	Clear	NO			
Isolation	Clear	NO			
Isolation	Measure	YES	CapGuaP1	NO	GenericNumber
Isolation	Measure	YES	CapGuaP1	NO	GenericNumber
Isolation	Measure	YES	CapPar	NO	GenericNumber
Isolation	Measure	YES	GuardPin1		GenericPinlist
Isolation	Measure	YES	GuardPin2		GenericPinlist
Isolation	Measure	YES	MeasType	Default	GenericNumber
Isolation	Measure	YES	Pin1		GenericPinlist
Isolation	Measure	YES	Pin2		GenericPinlist
Isolation	Measure	YES	Recycle	1	GenericNumber
Isolation	Measure	YES	ResGuaP1	NO	GenericNumber
Isolation	Measure	YES	ResGuaP1	NO	GenericNumber
Isolation	Measure	YES	TestResult	Default	GenericNumber
Isolation	Measure	YES	Time	-1	GenericNumber
Isolation	Measure	YES	Value		GenericNumber
Isolation	Measure	YES	Volt	-1	GenericNumber
Isolation	Measure	YES	Wire	Default	GenericNumber
JScan	Clear	NO			

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
JScan	Measure	YES	CapParRefPin	NO	GenericNumber
JScan	Measure	YES	CapParTestPin	NO	GenericNumber
JScan	Measure	YES	CurrentDRA	-1	GenericNumber
JScan	Measure	YES	CurrentDRB	-1	GenericNumber
JScan	Measure	YES	Frequency	-1	GenericNumber
JScan	Measure	YES	GndPin		GenericPinlist
JScan	Measure	YES	MeasType	Default	GenericNumber
JScan	Measure	YES	Quality	-1	GenericNumber
JScan	Measure	YES	Recycle	1	GenericNumber
JScan	Measure	YES	RefPin		GenericPinlist
JScan	Measure	YES	RefPinIso	NO	GenericNumber
JScan	Measure	YES	RelMacro	NO	GenericNumber
JScan	Measure	YES	ResParRefPin	NO	GenericNumber
JScan	Measure	YES	ResParTestPin	NO	GenericNumber
JScan	Measure	YES	SenseGndPin		GenericPinlist
JScan	Measure	YES	SettlingWave	-1	GenericNumber
JScan	Measure	YES	Split	NO	GenericNumber
JScan	Measure	YES	TestPin		GenericPinlist
JScan	Measure	YES	TestPinIso	NO	GenericNumber
JScan	Measure	YES	TestResult	Default	GenericNumber
JScan	Measure	YES	TolNeg		GenericNumber
JScan	Measure	YES	TolPos		GenericNumber
JScan	Measure	YES	Value		GenericNumber
LEN	Assign	YES	expression		GenericString
Mosfet	Clear	NO			
MOTION	Clear	NO			
MOTION	Read	YES			
MOTION	Set	NO	mode		GenericString
MOTION	Write	NO	string		GenericString
OPC	Clear	NO			
OPC	Set	NO	value	255	GenericNumber
OPENFIX	Clear	NO			
OPENFIX	Measure	YES	time		GenericNumber
OPENFIX	Set	NO	mode	FM	GenericString
OPENFIX	Set	NO	offset		GenericNumber
OPENFIX	Set	NO	scale	HIGH	GenericString
OPENFIX	Set	NO	valI		GenericNumber
OPENFIX	Set	NO	valV		GenericNumber
OpenFixture	Clear	NO			

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
OpenFixture	Measure	YES	Frequency	-1	GenericNumber
OpenFixture	Measure	YES	MeasMode	Default	GenericNumber
OpenFixture	Measure	YES	MeasType	Default	GenericNumber
OpenFixture	Measure	YES	Quality	-1	GenericNumber
OpenFixture	Measure	YES	Recycle	1	GenericNumber
OpenFixture	Measure	YES	RefPin		GenericPinlist
OpenFixture	Measure	YES	Sensor		GenericNumber
OpenFixture	Measure	YES	SensorCh		GenericPinlist
OpenFixture	Measure	YES	SettlingWave	-1	GenericNumber
OpenFixture	Measure	YES	TestPin		GenericPinlist
OpenFixture	Measure	YES	TestResult	Default	GenericNumber
OpenFixture	Measure	YES	TolNeg		GenericNumber
OpenFixture	Measure	YES	TolPos		GenericNumber
OpenFixture	Measure	YES	Value		GenericNumber
OpenFixture	Measure	YES	Volt	-1	GenericNumber
Opic	Clear	NO			
Opic	Measure	YES	CapParRefPin	NO	GenericNumber
Opic	Measure	YES	CapParTestPin	NO	GenericNumber
Opic	Measure	YES	CurrentDRA	-1	GenericNumber
Opic	Measure	YES	CurrentDRB	-1	GenericNumber
Opic	Measure	YES	Frequency	-1	GenericNumber
Opic	Measure	YES	GndPin		GenericPinlist
Opic	Measure	YES	MeasType	Default	GenericNumber
Opic	Measure	YES	Quality	-1	GenericNumber
Opic	Measure	YES	Recycle	1	GenericNumber
Opic	Measure	YES	RefPin		GenericPinlist
Opic	Measure	YES	RefPinIso	NO	GenericNumber
Opic	Measure	YES	ResParRefPin	NO	GenericNumber
Opic	Measure	YES	ResParTestPin	NO	GenericNumber
Opic	Measure	YES	SenseGndPin		GenericPinlist
Opic	Measure	YES	SettlingWave	-1	GenericNumber
Opic	Measure	YES	Split	NO	GenericNumber
Opic	Measure	YES	TestPin		GenericPinlist
Opic	Measure	YES	TestPinIso	NO	GenericNumber
Opic	Measure	YES	TestResult	Default	GenericNumber
Opic	Measure	YES	TolNeg		GenericNumber
Opic	Measure	YES	TolPos		GenericNumber
Opic	Measure	YES	Value		GenericNumber
PAN	Clear	NO			

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
PAN	Measure	NO	EnableC	TRUE	GenericNumber
PAN	Measure	NO	EnableD	TRUE	GenericNumber
PAN	Measure	NO	EnableL	TRUE	GenericNumber
PAN	Measure	NO	EnableR	TRUE	GenericNumber
PAN	Measure	NO	FreqC	-1	GenericNumber
PAN	Measure	NO	FreqL	-1	GenericNumber
PAN	Measure	NO	FreqR	-1	GenericNumber
PAN	Measure	NO	GuardPin1		GenericPinlist
PAN	Measure	NO	MeasType	Default	GenericNumber
PAN	Measure	NO	NodeType	NO	GenericNumber
PAN	Measure	NO	Pin1		GenericPinlist
PAN	Measure	NO	Pin2		GenericPinlist
PAN	Measure	NO	Quality	-1	GenericNumber
PAN	Measure	NO	Recycle	1	GenericNumber
PAN	Measure	NO	SettlingWave	-1	GenericNumber
PAN	Measure	NO	TestResult	Default	GenericNumber
PAN	Measure	NO	TolNegC		GenericNumber
PAN	Measure	NO	TolNegD		GenericNumber
PAN	Measure	NO	TolNegL		GenericNumber
PAN	Measure	NO	TolNegR		GenericNumber
PAN	Measure	NO	TolPosC		GenericNumber
PAN	Measure	NO	TolPosD		GenericNumber
PAN	Measure	NO	TolPosL		GenericNumber
PAN	Measure	NO	TolPosR		GenericNumber
PAN	Measure	NO	ValueC	NO	GenericNumber
PAN	Measure	NO	ValueD	NO	GenericNumber
PAN	Measure	NO	ValueL	NO	GenericNumber
PAN	Measure	NO	ValueR	NO	GenericNumber
PAN	Measure	NO	Volt	-1	GenericNumber
PAN	Measure	NO	Wire	Default	GenericNumber
POWERSCAN	Clear	NO			
PRINTER	Clear	NO			
PRINTER	Set	NO	mode		GenericString
PRINTER	Write	NO	string		GenericString
PROBE1	Clear	NO			
PROBE1	Set	NO	downPos	-NO.NONO2	GenericNumber
PROBE1	Set	NO	mode	UP	GenericString
PROBE1	Set	NO	speed	1NO	GenericNumber
PROBE1	Set	NO	upPos	NO.NO2NO	GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
PROBE2	Clear	NO			
PROBE2	Set	NO	downPos	-NO.NONO2	GenericNumber
PROBE2	Set	NO	mode	UP	GenericString
PROBE2	Set	NO	speed	1NO	GenericNumber
PROBE2	Set	NO	upPos	NO.NO2NO	GenericNumber
PROBE3	Clear	NO			
PROBE3	Set	NO	downPos	-NO.NONO2	GenericNumber
PROBE3	Set	NO	mode	UP	GenericString
PROBE3	Set	NO	speed	1NO	GenericNumber
PROBE3	Set	NO	upPos	NO.NO2NO	GenericNumber
PROBE4	Clear	NO			
PROBE4	Set	NO	downPos	-NO.NONO2	GenericNumber
PROBE4	Set	NO	mode	UP	GenericString
PROBE4	Set	NO	speed	1NO	GenericNumber
PROBE4	Set	NO	upPos	NO.NO2NO	GenericNumber
PUL	Clear	NO			
PUL	Set	NO	mode	Up	GenericNumber
PUL	Set	NO	valO	1NONONO	GenericNumber
PW1	Clear	NO			
PW1	Set	NO	sense	On	GenericNumber
PW1	Set	NO	state	On	GenericNumber
PW1	Set	NO	valV		GenericNumber
PW2	Clear	NO			
PW2	Set	NO	sense	On	GenericNumber
PW2	Set	NO	state	On	GenericNumber
PW2	Set	NO	valV		GenericNumber
PW3	Clear	NO			
PW3	Set	NO	sense	On	GenericNumber
PW3	Set	NO	state	On	GenericNumber
PW3	Set	NO	valV		GenericNumber
PW4	Clear	NO			
PW4	Set	NO	sense	On	GenericNumber
PW4	Set	NO	state	On	GenericNumber
PW4	Set	NO	valV		GenericNumber
PW5	Clear	NO			
PW5	Set	NO	sense	On	GenericNumber
PW5	Set	NO	state	On	GenericNumber
PW5	Set	NO	valV		GenericNumber
PW6	Clear	NO			

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
PW6	Set	NO	sense	On	GenericNumber
PW6	Set	NO	state	On	GenericNumber
PW6	Set	NO	valV		GenericNumber
REPORT	Clear	NO			
REPORT	Set	NO	mode		GenericString
REPORT	Write	NO	string		GenericString
Resistor	Clear	NO			
Resistor	Measure	YES	BoardOffset	NO	GenericNumber
Resistor	Measure	YES	CapGuaP1	NO	GenericNumber
Resistor	Measure	YES	CapGuaP2	NO	GenericNumber
Resistor	Measure	YES	CapPar	NO	GenericNumber
Resistor	Measure	YES	CapSer	NO	GenericNumber
Resistor	Measure	YES	Frequency	-1	GenericNumber
Resistor	Measure	YES	GuardPin1		GenericPinlist
Resistor	Measure	YES	GuardPin2		GenericPinlist
Resistor	Measure	YES	IndPar	NO	GenericNumber
Resistor	Measure	YES	IndSer	NO	GenericNumber
Resistor	Measure	YES	MeasMode	Default	GenericNumber
Resistor	Measure	YES	MeasType	Default	GenericNumber
Resistor	Measure	YES	Pin1		GenericPinlist
Resistor	Measure	YES	Pin2		GenericPinlist
Resistor	Measure	YES	Quality	-1	GenericNumber
Resistor	Measure	YES	Recycle	1	GenericNumber
Resistor	Measure	YES	ResGuaP1	NO	GenericNumber
Resistor	Measure	YES	ResGuaP2	NO	GenericNumber
Resistor	Measure	YES	SenseGuardPin1		GenericPinlist
Resistor	Measure	YES	SenseGuardPin2		GenericPinlist
Resistor	Measure	YES	SenseP1		GenericPinlist
Resistor	Measure	YES	SenseP2		GenericPinlist
Resistor	Measure	YES	SettlingWave	-1	GenericNumber
Resistor	Measure	YES	TestResult	Default	GenericNumber
Resistor	Measure	YES	Time	-1	GenericNumber
Resistor	Measure	YES	TolNeg		GenericNumber
Resistor	Measure	YES	TolPos		GenericNumber
Resistor	Measure	YES	Value		GenericNumber
Resistor	Measure	YES	Volt	-1	GenericNumber
Resistor	Measure	YES	Wire	Default	GenericNumber
RETURN	Dummy	NO	value		GenericNumber
SCAFP	Clear	NO			

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
SERIAL	Clear	NO			
SERIAL	Read	YES			
SERIAL	Set	NO	mode		GenericString
SERIAL	Write	NO	string		GenericString
Short	Clear	NO			
Short	Measure	YES	Family		GenericNumber
Short	Measure	YES	StartCh		GenericNumber
Short	Measure	YES	StopCh		GenericNumber
Short	Measure	YES	TestResult	Default	GenericNumber
Short	Measure	YES	Time	-1	GenericNumber
Short	Measure	YES	Value		GenericNumber
Short	Measure	YES	ViewCh	NO	GenericNumber
Short	Measure	YES	Volt	-1	GenericNumber
SYSTEM	AutoAdjustement	YES			
SYSTEM	Autodebug	YES			
SYSTEM	Autolearn	YES			
SYSTEM	AuxiliaryBox	YES			
SYSTEM	BarCode	YES			
SYSTEM	BoardDescription	YES			
SYSTEM	BoardStrayCapacitance	YES			
SYSTEM	BoardStrayImpedance	YES			
SYSTEM	BoardStrayInductance	YES			
SYSTEM	Capacitive	YES	action		GenericNumber
SYSTEM	Capacitor	YES	LParValue	NO	GenericNumber
SYSTEM	Capacitor	YES	ManGUA		GenericPinlist
SYSTEM	Capacitor	YES	Mode		GenericNumber
SYSTEM	Capacitor	YES	MRC		GenericNumber
SYSTEM	Capacitor	YES	NER		GenericNumber
SYSTEM	Capacitor	YES	NRC		GenericNumber
SYSTEM	Capacitor	YES	ParValue	NO	GenericNumber
SYSTEM	Capacitor	YES	Pin1		GenericPinlist
SYSTEM	Capacitor	YES	Pin1GUA		GenericPinlist
SYSTEM	Capacitor	YES	Pin2		GenericPinlist
SYSTEM	Capacitor	YES	Pin2GUA		GenericPinlist
SYSTEM	Capacitor	YES	Pin3		GenericPinlist
SYSTEM	Capacitor	YES	Pin4		GenericPinlist
SYSTEM	Capacitor	YES	Time		GenericNumber
SYSTEM	Capacitor	YES	TolN		GenericNumber
SYSTEM	Capacitor	YES	TolP		GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
SYSTEM	Capacitor	YES	Value		GenericNumber
SYSTEM	CapRead	YES	CapValue	1NONONONONONO	GenericNumber
SYSTEM	CapRead	YES	MeasureType	2	GenericNumber
SYSTEM	CapRead	YES	Pin1		GenericPinlist
SYSTEM	CapRead	YES	Pin2		GenericPinlist
SYSTEM	CapRead	YES	TolN	1NONO	GenericNumber
SYSTEM	CapRead	YES	TolP	1NONO	GenericNumber
SYSTEM	Cluster	YES	ClusterName		GenericString
SYSTEM	Cluster	YES	FileName		GenericString
SYSTEM	Cluster	YES	Flags	NO	GenericNumber
SYSTEM	Continuity	YES	Impedance		GenericNumber
SYSTEM	Continuity	YES	NER		GenericNumber
SYSTEM	Continuity	YES	Pin1		GenericPinlist
SYSTEM	Continuity	YES	Pin2		GenericPinlist
SYSTEM	Continuity	YES	Time		GenericNumber
SYSTEM	CurrentBoard	YES			
SYSTEM	DefaultWorkPath	YES			
SYSTEM	Dmasater	YES			
SYSTEM	F8LinesChannels	YES			
SYSTEM	Femto	YES			
SYSTEM	FixtureInvert	YES			
SYSTEM	FixtureType	YES			
SYSTEM	HighVoltage	YES			
SYSTEM	HVIvalue	YES			
SYSTEM	Inductor	YES	ManGUA		GenericPinlist
SYSTEM	Inductor	YES	Mode		GenericNumber
SYSTEM	Inductor	YES	MRC		GenericNumber
SYSTEM	Inductor	YES	NER		GenericNumber
SYSTEM	Inductor	YES	NRC		GenericNumber
SYSTEM	Inductor	YES	ParValue	NO	GenericNumber
SYSTEM	Inductor	YES	Pin1		GenericPinlist
SYSTEM	Inductor	YES	Pin1GUA		GenericPinlist
SYSTEM	Inductor	YES	Pin2		GenericPinlist
SYSTEM	Inductor	YES	Pin2GUA		GenericPinlist
SYSTEM	Inductor	YES	Pin3		GenericPinlist
SYSTEM	Inductor	YES	Pin4		GenericPinlist
SYSTEM	Inductor	YES	RParValue	NO	GenericNumber
SYSTEM	Inductor	YES	Time		GenericNumber
SYSTEM	Inductor	YES	TolN		GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
SYSTEM	Inductor	YES	TolP		GenericNumber
SYSTEM	Inductor	YES	Value		GenericNumber
SYSTEM	Isolation	YES	Impedance		GenericNumber
SYSTEM	Isolation	YES	NER		GenericNumber
SYSTEM	Isolation	YES	Pin1		GenericPinlist
SYSTEM	Isolation	YES	Pin2		GenericPinlist
SYSTEM	Isolation	YES	Time		GenericNumber
SYSTEM	Isolation	YES	Voltage		GenericNumber
SYSTEM	LamPresent	YES			
SYSTEM	LamScale	YES			
SYSTEM	MoveProbes	YES	Pin1		GenericPinlist
SYSTEM	MoveProbes	YES	Pin2		GenericPinlist
SYSTEM	MoveProbes	YES	Pin3		GenericPinlist
SYSTEM	MoveProbes	YES	Pin4		GenericPinlist
SYSTEM	MoveProbes	YES	ToolName1		GenericString
SYSTEM	MoveProbes	YES	ToolName2		GenericString
SYSTEM	MoveProbes	YES	ToolName3		GenericString
SYSTEM	MoveProbes	YES	ToolName4		GenericString
SYSTEM	OpenFix	YES			
SYSTEM	ParassiteLearn	YES			
SYSTEM	PowerSupply	YES			
SYSTEM	ProgrammingStation	YES			
SYSTEM	Resistor	YES	LParValue	NO	GenericNumber
SYSTEM	Resistor	YES	ManGUA		GenericPinlist
SYSTEM	Resistor	YES	Mode		GenericNumber
SYSTEM	Resistor	YES	MRC		GenericNumber
SYSTEM	Resistor	YES	NER		GenericNumber
SYSTEM	Resistor	YES	NRC		GenericNumber
SYSTEM	Resistor	YES	ParValue	NO	GenericNumber
SYSTEM	Resistor	YES	Pin1		GenericPinlist
SYSTEM	Resistor	YES	Pin1GUA		GenericPinlist
SYSTEM	Resistor	YES	Pin2		GenericPinlist
SYSTEM	Resistor	YES	Pin2GUA		GenericPinlist
SYSTEM	Resistor	YES	Pin3		GenericPinlist
SYSTEM	Resistor	YES	Pin4		GenericPinlist
SYSTEM	Resistor	YES	Time		GenericNumber
SYSTEM	Resistor	YES	TolN		GenericNumber
SYSTEM	Resistor	YES	TolP		GenericNumber
SYSTEM	Resistor	YES	Value		GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
SYSTEM	RestoreParameters	YES			
SYSTEM	SaveParameters	YES			
SYSTEM	ScaFP	YES			
SYSTEM	Scampo	YES			
SYSTEM	ScannerBoards	YES			
SYSTEM	Side To Test	YES			
SYSTEM	Soldering	YES	HeadRotation		GenericNumber
SYSTEM	Soldering	YES	PCBDistance		GenericNumber
SYSTEM	Soldering	YES	PinDiameter		GenericNumber
SYSTEM	Soldering	YES	PreHeatPower		GenericNumber
SYSTEM	Soldering	YES	PreHeatTime		GenericNumber
SYSTEM	Soldering	YES	ProcessSimul		GenericNumber
SYSTEM	Soldering	YES	PSSoldPower		GenericNumber
SYSTEM	Soldering	YES	PSSoldTime		GenericNumber
SYSTEM	Soldering	YES	SHeadRotation		GenericNumber
SYSTEM	Soldering	YES	SolderJoint		GenericPinlist
SYSTEM	Soldering	YES	SolderPower		GenericNumber
SYSTEM	Soldering	YES	SolderStep		GenericNumber
SYSTEM	Soldering	YES	SolderTime		GenericNumber
SYSTEM	Soldering	YES	SpotDiameter		GenericNumber
SYSTEM	Soldering	YES	WireLength		GenericNumber
SYSTEM	Soldering	YES	WireNear		GenericNumber
SYSTEM	Soldering1	YES	HeadRotation		GenericNumber
SYSTEM	Soldering1	YES	PCBDistance		GenericNumber
SYSTEM	Soldering1	YES	PinDiameter		GenericNumber
SYSTEM	Soldering1	YES	PreHeatPw1		GenericNumber
SYSTEM	Soldering1	YES	PreHeatPw2		GenericNumber
SYSTEM	Soldering1	YES	PreHeatTime		GenericNumber
SYSTEM	Soldering1	YES	PreWireNear		GenericNumber
SYSTEM	Soldering1	YES	ProcessSimul		GenericNumber
SYSTEM	Soldering1	YES	PSSoldPw1		GenericNumber
SYSTEM	Soldering1	YES	PSSoldPw2		GenericNumber
SYSTEM	Soldering1	YES	PSSoldTime		GenericNumber
SYSTEM	Soldering1	YES	PwReductionF		GenericNumber
SYSTEM	Soldering1	YES	SHeadRotation		GenericNumber
SYSTEM	Soldering1	YES	SolderJoint		GenericPinlist
SYSTEM	Soldering1	YES	SolderPw1		GenericNumber
SYSTEM	Soldering1	YES	SolderPw2		GenericNumber
SYSTEM	Soldering1	YES	SolderTime		GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
SYSTEM	Soldering1	YES	SoldWireNear		GenericNumber
SYSTEM	Soldering1	YES	SpotDiameter		GenericNumber
SYSTEM	Soldering1	YES	WireLenAdj		GenericNumber
SYSTEM	Soldering1	YES	WireLength		GenericNumber
SYSTEM	Soldering2	YES	Feedback		GenericNumber
SYSTEM	Soldering2	YES	HeadRotation		GenericNumber
SYSTEM	Soldering2	YES	PinDiameter		GenericNumber
SYSTEM	Soldering2	YES	Power		GenericNumber
SYSTEM	Soldering2	YES	PreHeatPw1		GenericNumber
SYSTEM	Soldering2	YES	PreHeatPw2		GenericNumber
SYSTEM	Soldering2	YES	PreHeatTemp		GenericNumber
SYSTEM	Soldering2	YES	PreHeatTime		GenericNumber
SYSTEM	Soldering2	YES	PSSoldPw1		GenericNumber
SYSTEM	Soldering2	YES	PSSoldPw2		GenericNumber
SYSTEM	Soldering2	YES	PSSoldTemp		GenericNumber
SYSTEM	Soldering2	YES	PSSoldTime		GenericNumber
SYSTEM	Soldering2	YES	SHeadRotation		GenericNumber
SYSTEM	Soldering2	YES	SolderJoint		GenericPinlist
SYSTEM	Soldering2	YES	SolderPw1		GenericNumber
SYSTEM	Soldering2	YES	SolderPw2		GenericNumber
SYSTEM	Soldering2	YES	SolderTemp		GenericNumber
SYSTEM	Soldering2	YES	SolderTime		GenericNumber
SYSTEM	Soldering2	YES	SpotDiameter		GenericNumber
SYSTEM	Soldering2	YES	WireLength		GenericNumber
SYSTEM	StrayCapacitance	YES			
SYSTEM	StrayImpedance	YES			
SYSTEM	StrayInductance	YES			
SYSTEM	SystemName	YES			
SYSTEM	SystemType	YES			
SYSTEM	UserBoard	YES			
SYSTEM	UserBoard2	YES			
SYSTEM	UserBoard3	YES			
SYSTEM	UserBoard4	YES			
SYSTEM	UserGen	YES			
SYSTEM	UserGen2	YES			
SYSTEM	UserGen3	YES			
SYSTEM	UserGen4	YES			
SYSTEM	Vision	YES	Component		GenericString
TEST	Clear	NO			

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
TEST	GetParN	YES	Default Value		GenericNumber
TEST	GetParN	YES	Name		GenericString
TEST	GetParP	YES	Default Value		GenericPinlist
TEST	GetParP	YES	Name		GenericString
TEST	GetParS	YES	Default Value		GenericString
TEST	GetParS	YES	Name		GenericString
TEST	Measure	YES	error	Set Error	GenericNumber
TEST	Measure	YES	value	NO	GenericNumber
TEST	Measure	YES	view	None	GenericNumber
TEST	Set	NO	high	NO	GenericNumber
TEST	Set	NO	low	NO	GenericNumber
TEST	Set	NO	meas	NO	GenericNumber
TEST	Set	NO	unit	V	GenericString
TEST	SetParN	NO	Name		GenericString
TEST	SetParN	NO	Value		GenericNumber
TEST	SetParP	NO	Name		GenericString
TEST	SetParP	NO	Value		GenericPinlist
TEST	SetParS	NO	Name		GenericString
TEST	SetParS	NO	Value		GenericString
TEST	WriteMessage	YES	Message		GenericString
TEST	WriteReport	YES	Message		GenericString
Transistor	Clear	NO			
Transistor	Measure	YES	Base_Gate		GenericPinlist
Transistor	Measure	YES	CapGuabase	NO	GenericNumber
Transistor	Measure	YES	CapGuaCollector	NO	GenericNumber
Transistor	Measure	YES	CapPar	NO	GenericNumber
Transistor	Measure	YES	Collector_Drain		GenericPinlist
Transistor	Measure	YES	Current	-1	GenericNumber
Transistor	Measure	YES	Emitter_Source		GenericPinlist
Transistor	Measure	YES	GuardBase		GenericPinlist
Transistor	Measure	YES	GuardCollector		GenericPinlist
Transistor	Measure	YES	MeasMode	NO	GenericNumber
Transistor	Measure	YES	MeasType	Default	GenericNumber
Transistor	Measure	YES	Quality	-1	GenericNumber
Transistor	Measure	YES	Recycle	1	GenericNumber
Transistor	Measure	YES	ResGuaBase	NO	GenericNumber
Transistor	Measure	YES	ResGuaCollector	NO	GenericNumber
Transistor	Measure	YES	ResPar	NO	GenericNumber
Transistor	Measure	YES	Technology		GenericNumber

Tool.Name	Method.Name	Have RetValue	Param.Name	DefaultValue	ParameterClass
Transistor	Measure	YES	TestMode	Default	GenericNumber
Transistor	Measure	YES	TestResult	Default	GenericNumber
Transistor	Measure	YES	Time	-1	GenericNumber
Transistor	Measure	YES	TolNeg		GenericNumber
Transistor	Measure	YES	TolPos		GenericNumber
Transistor	Measure	YES	Type		GenericNumber
Transistor	Measure	YES	Volt	-1	GenericNumber
Transistor	Measure	YES	Wire	Default	GenericNumber
USERBOARD	Clear	NO			
VBS	Clear	NO			
VBS	Execute	NO	code		GenericString
VBS	Execute	NO	run		<unknown>
VISION	Clear	NO			
VISION	Measure	YES	mode	AUTO	GenericString
VISION	Measure	YES	tol	3NO	GenericNumber
VOLT	Clear	NO			
VOLT	Measure	YES	mode	Average	GenericNumber
VOLT	Measure	YES	recycle	1	GenericNumber
VOLT	Measure	YES	testPoint	NO	GenericPinlist
VOLT	Measure	YES	time	NO.NONO1	GenericNumber
VOLT	Set	NO	inShortWith	None	GenericNumber
VOLT	Set	NO	maxVal	*5NO	GenericNumber
VOLT	Set	NO	measType	Volt	GenericString
WAIT	Set	NO	time		GenericNumber

5. Bibliography

[1] VIVA Macro di sistema
Codice: MA-VIVAMSIT-03 (28-APR-2008) Seica S.p.A.

VIVA System macro
Codice: MA-VI-VIVAMSEN-02 (11-JAN-2008) Seica S.p.A.

[2] Modulo ACL - Strumenti interni
Codice: MA-VI-ACLI00IT-08 (3-MAR-2009) Seica S.p.A.

ACL Module
Codice: MA-VI-ACLI00EN-06 (23-JUL-2009) Seica S.p.A.

[3] VIVA Language MA-VI-VIVAVLIT-01
Codice: MA-VIVAVLIT-NONO (27-AUG-2007) Seica S.p.A.

VIVA Language MA-VIVAVLEN-00
Codice: MA-VIVAVLEN-00 (31-1-2006) Seica S.p.A.

[4] VIVA Device Complessi MA-VI-VIVADCIT-00
Codice: MA-VI-VIVADCIT-00 (9-7-2007) Seica S.p.A.

VIVA Complex Devices MA-VI-VIVADCEN-00
Codice: MA-VI-VIVADCEN-00 (19-12-2007) Seica S.p.A.

[5] ICT Programming Language MA-LAFPICEN-2
Codice: MA-LAFPICEN-NO2 (23-NOV-200474) Seica S.p.A.

Programmazione ICI 311 MA-LAFPICIT-2
Codice: MA-LAFPICIT-2x (18-1-2004) Seica S.p.A.